MarutiNovaTech

Data and Technology Training Institute, Jaipur

SQL Server Interview Playbook

The 51 most-asked SQL Server interview questions, from fresher to senior.
Every question comes with the ways to solve it, why you would choose each way, and exactly when each way fails.

Fresher | 1 to 3 yrs | 3 to 5 yrs | 5+ yrs

By Deepankar Sir

Instagram @datawithdeepankar
YouTube @deepankarpathak983

October 2026 edition

How to use this playbook

You do not need to memorise 51 answers. You need to understand 15 to 20 ideas and know how to say them. This playbook is built so you can read the question, say the opening line, and then walk through the ways in the same order a strong candidate would.

The four tags

Tag	What it means
MOST ASKED	Shows up in almost every list, video and reel we reviewed. Learn these first.
IMPORTANT	A core concept. A weak answer here usually ends the round.
LOGICAL	Tests reasoning, a trap or an output-prediction puzzle, not memory.
VARIANT FAMILY	One pattern, many questions. Learn the approach once, then practise every variant listed under it. Second highest salary is the classic example: the same idea gives you third highest, Nth highest, per department and more.

What the percentage means

The percentage is our estimate of how likely it is that this question, or a direct variant of it, is asked in an SQL Server round at that experience level. It comes from how often each topic appeared across interview lists, written guides and Indian interview-prep posts we reviewed. It is a judgement, not a measured statistic. Use it to decide the order of your revision.

The answer method: say it in five moves

> **1. Restate and clarify.** Repeat the question in your own words and ask one smart question. For example: "Should ties count as one salary or two?"

> **2. Give the simplest working way first.** A correct simple answer beats a clever broken one.

> **3. Name its weak spot yourself.** "This works, but it only handles the second highest, not the Nth." Interviewers love this.

> **4. Offer the stronger way and say why.** This is the part most candidates skip, and it is what this playbook trains.

> **5. Say what you would do in a real project.** Test on a copy, check the plan, mind the NULLs.

How every question page is built

- **How the interviewer asks it:** the different ways the same question is worded.
- **Say this first:** a spoken opening you can say in 20 seconds.
- **Way 1, Way 2, Way 3 ...:** each with the code, **why you would use it** and **when it fails**.
- **Picture it:** a diagram (Venn, table walk-through, timeline) that shows what the query does.
- **Execution order:** the order SQL Server really runs each clause, step by step on our practice data.
- **Which one should you pick:** the decision rule.
- **The interviewer's trap:** the mistake they are waiting for.
- **More questions hiding inside this one:** the follow-ups, so a new version of the question never surprises you.

> **Language and version.** Everything is T-SQL for Microsoft SQL Server. Where a feature needs a newer version (for example OFFSET FETCH needs SQL Server 2012 or later), the page says so. Use the practice script on the next pages to build the same tables and run every query yourself.

Practice setup: build the tables once

Every query in this playbook runs on these five small tables. Copy the script into SSMS or Azure Data Studio and run it once. The data is designed on purpose: the top salary is shared by two people, two departments have no employees, and one customer is duplicated. That is what makes the traps visible.

SETUP SCRIPT

```
CREATE DATABASE InterviewPrep;
GO
USE InterviewPrep;
GO

CREATE TABLE Departments (
    DeptID   INT PRIMARY KEY,
    DeptName VARCHAR(30) NOT NULL
);
INSERT INTO Departments VALUES
 (1,'Sales'), (2,'IT'), (3,'HR'), (4,'Finance'), (5,'Legal');
 -- Finance and Legal have no employees on purpose

CREATE TABLE Employees (
    EmpID     INT PRIMARY KEY,
    Name      VARCHAR(30) NOT NULL,
    Gender    CHAR(1),
    Salary    INT,
    DeptID    INT,
    ManagerID INT NULL,
    JoinDate  DATE
);
INSERT INTO Employees VALUES
 (1,'Asha','F',95000,2,NULL,'2018-01-10'),
 (2,'Vikram','M',90000,2,1,'2019-03-15'),
 (3,'Neha','F',90000,1,1,'2019-07-01'),
 (4,'Rahul','M',80000,1,3,'2020-02-20'),
 (5,'Priya','F',80000,2,2,'2021-05-11'),
 (6,'Karan','M',70000,3,1,'2020-09-09'),
 (7,'Sneha','F',60000,3,6,'2022-01-17'),
 (8,'Amit','M',60000,1,3,'2022-06-30'),
 (9,'Pooja','F',55000,2,2,'2023-02-14'),
 (10,'Rajesh','M',50000,3,6,'2023-08-01'),
 (11,'Meera','F',95000,1,3,'2021-11-05');

CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,
    Name  VARCHAR(30),
    Email VARCHAR(50),
    City  VARCHAR(20)
);
INSERT INTO Customers VALUES
 (1,'Aman','aman@mail.com','Jaipur'),
 (2,'Bhavna','bhavna@mail.com','Delhi'),
 (3,'Chirag','chirag@mail.com','Pune'),
 (4,'Aman','aman@mail.com','Jaipur'),   -- duplicate email
 (5,'Divya','divya@mail.com','Jaipur'),
 (6,'Esha','esha@mail.com','Mumbai');

CREATE TABLE Orders (
    OrderID    INT PRIMARY KEY,
    CustomerID INT NULL,
    Amount     INT,
    OrderDate  DATE
);
INSERT INTO Orders VALUES
 (1,1,2000,'2026-01-05'), (2,1,3500,'2026-01-20'),
 (3,2,1500,'2026-01-22'), (4,3,4000,'2026-02-03'),
 (5,2,2500,'2026-02-14'), (6,1,3000,'2026-02-28'),
 (7,3,1000,'2026-03-10'), (8,5,5000,'2026-03-15');

CREATE TABLE Logins (UserID INT, LoginDate DATE);
INSERT INTO Logins VALUES
 (1,'2026-03-01'),(1,'2026-03-02'),(1,'2026-03-03'),(1,'2026-03-04'),
 (1,'2026-03-06'),(1,'2026-03-07'),
 (2,'2026-03-01'),(2,'2026-03-03'),(2,'2026-03-04'),(2,'2026-03-05'),
 (3,'2026-03-02'),(3,'2026-03-05');
```

Sample tables at a glance. Employees has 11 rows (salaries 95000, 95000, 90000, 90000, 80000, 80000, 70000, 60000, 60000, 55000, 50000). Customers has 6 rows. Orders has 8 rows from January to March 2026. Logins has 12 rows for the streak questions.

Index of all 51 questions

An asterisk marks MOST ASKED questions. The percentage is the estimated chance of the question being asked. The last column is the page number.

Fresher 0 to 1 year

Q1	Find the second highest salary *	95%	6
Q2	Types of JOINs in SQL Server *	95%	12
Q3	How many rows will each join return? *	70%	15
Q4	DELETE vs TRUNCATE vs DROP *	90%	17
Q5	WHERE vs HAVING *	85%	19
Q6	Find duplicate records *	85%	22
Q7	Clustered vs non-clustered index *	85%	25
Q8	Stored procedure vs function	80%	26
Q9	Primary key vs unique key	80%	28
Q10	Logical order of query execution	70%	29
Q11	UNION vs UNION ALL (and INTERSECT, EXCEPT) *	70%	32
Q12	NULL handling: ISNULL vs COALESCE	65%	34
Q13	COUNT(*) vs COUNT(column) vs COUNT(DISTINCT column)	60%	36

Junior to Mid 1 to 3 years

Q14	ROW_NUMBER vs RANK vs DENSE_RANK *	90%	40
Q15	Top N per group (top 3 earners in each department) *	80%	42
Q16	Delete duplicates but keep one row *	85%	46
Q17	Employees who earn more than their manager *	75%	49
Q18	Customers who never placed an order	70%	52
Q19	Running total (cumulative sum)	70%	55
Q20	LAG and LEAD: compare with the previous or next row	65%	58
Q21	CTE vs temp table vs table variable	75%	61
Q22	Rows to columns: CASE and PIVOT	60%	62
Q23	Swap M and F in one UPDATE	55%	65
Q24	Correlated subquery, and EXISTS vs IN	60%	67
Q25	Triggers: AFTER vs INSTEAD OF	55%	70
Q26	Error handling and transactions (TRY...CATCH)	55%	72
Q27	Views: can you update data through a view?	50%	74

Mid to Senior 3 to 5 years

Q28	A query is slow. How do you troubleshoot it? *	85%	77
Q29	Index seek vs scan, key lookup and covering index *	80%	80
Q30	Why is my index not used? (SARGability)	70%	82
Q31	Isolation levels in SQL Server	75%	85
Q32	Deadlock vs blocking	75%	88
Q33	Consecutive days (gaps and islands)	60%	91
Q34	Recursive CTE: employee hierarchy	60%	94
Q35	MERGE: insert, update and delete in one statement	55%	98
Q36	Parameter sniffing	55%	101
Q37	Index fragmentation: REBUILD vs REORGANIZE	50%	103
Q38	Dynamic SQL and SQL injection	50%	106
Q39	Median salary	45%	108
Q40	Date logic questions	50%	111
Q41	Normalization, and when to denormalize	50%	113

Senior 5+ years

Q42	It was fast yesterday and 40 seconds today. What changed? *	70%	118
Q43	Indexing strategy for a big table	65%	120
Q44	Choosing the clustered index key: IDENTITY vs GUID	55%	122
Q45	Statistics and cardinality estimation	55%	125
Q46	Recovery models, backups and point-in-time restore *	55%	127
Q47	Table partitioning	50%	129
Q48	Columnstore vs rowstore indexes	50%	131
Q49	High availability and disaster recovery options	50%	133
Q50	Query Store and plan regression	45%	136
Q51	tempdb contention and usage	40%	138

LEVEL 1 OF 4

Fresher

0 to 1 year

13 questions in this level

WHAT THE INTERVIEWER IS REALLY CHECKING

Can you explain the basics clearly and write a clean query without syntax errors?

HOW TO WIN THIS LEVEL

Be precise. Interviewers at this level are not looking for tricks. They want correct definitions, correct syntax, and one sentence on the edge case (ties, NULLs). If you mention the edge case before they ask, you stand out from most freshers.

THREE HABITS FOR THIS LEVEL

- Never start typing before you restate the question and name the table.
- Always say the word DISTINCT out loud when salary or ranking is involved.
- If you do not know a feature, say what you would try. Guessing silently is worse than saying 'I would check the documentation for this'.

Q1 Find the second highest salary

MOST ASKED | VARIANT FAMILY

Chance of being asked 95%

HOW THE INTERVIEWER ASKS IT

- *"Write a query to find the second highest salary."*
- *"Find the second highest salary without using a window function."*
- *"Find the employee name who earns the second highest salary."*
- *"Same question, but two people can share the top salary. What happens now?"*

Say this first

"Second highest means the second highest **distinct** salary. Before I write anything, can two people share the top salary? That decides which way is safe. I know seven ways. Three use window functions: DENSE_RANK, ROW_NUMBER and RANK, and only one of the three is correct when salaries repeat. The other four are a MAX subquery, OFFSET FETCH, TOP twice and a correlated subquery. I will show the window function first because it is the easiest to read and it solves the whole family of questions."

Picture it

1. Employees, highest salary first

Name	Salary
Asha	95000
Meera	95000
Vikram	90000
Neha	90000
Rahul	80000
Priya	80000
Karan	70000
Sneha	60000
Amit	60000
Pooja	55000
Rajesh	50000

Red rows: two people share the top salary.

DISTINCT → remove repeats

2. Each salary once, numbered

Rank	Salary
1	95000
2	**90000**
3	80000
4	70000
5	60000
6	55000
7	50000

Rank 1 is ONE row, not two.

pick rank 2 →

Second highest salary

90000

earned by Vikram and Neha

Ties stay together:
both 90000 rows share rank 2

Read the picture left to right: sort, remove repeats, number, pick rank 2.

WAY 1 Window function: DENSE_RANK (the one to trust)

T-SQL

```
WITH r AS (
    SELECT Name, Salary,
           DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
    FROM Employees)
SELECT DISTINCT Salary AS SecondHighest
FROM r
WHERE rnk = 2;
```

RESULT

```
Step 1: what the CTE r produces          Step 2: WHERE rnk = 2
Name    Salary  rnk                      SecondHighest
------  ------  ---                      -------------
Asha    95000   1                        90000
Meera   95000   1
Vikram  90000   2   <- kept
Neha    90000   2   <- kept
Rahul   80000   3
Priya   80000   3
Karan   70000   4
Sneha   60000   5
Amit    60000   5
Pooja   55000   6
Rajesh  50000   7
```

WHY USE IT

DENSE_RANK gives people with the same salary the same rank and never skips a number, so rank 2 always means the second highest distinct salary. Change 2 to N for the Nth highest, add PARTITION BY DeptID for per-department, and remove DISTINCT to see the employee names. One pattern solves the whole family.

WHEN IT FAILS

It ranks the whole table just to get one number, which is overkill on a huge table with no index on Salary. When there is no second value it returns an empty result, not NULL. A window function cannot be filtered directly in WHERE, so you always need the extra layer (a CTE or subquery).

EXECUTION ORDER Way 1: DENSE_RANK in a CTE

```
WITH r AS (SELECT Name, Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk FROM Employees)
SELECT DISTINCT Salary AS SecondHighest FROM r WHERE rnk = 2;
```

1 FROM (inside r) SQL Server runs the inner query first. `FROM Employees` reads all 11 rows. There is no WHERE or GROUP BY inside it.

2 SELECT (window) `DENSE_RANK() OVER (ORDER BY Salary DESC)` is calculated now. The 11 rows are sorted by salary and numbered, ties share a number and no number is skipped. The result is the CTE r.

rnk	Salary	People with this salary
1	95000	Asha, Meera
2	90000	Vikram, Neha
3	80000	Rahul, Priya
4	70000	Karan
5	60000	Sneha, Amit
6	55000	Pooja
7	50000	Rajesh

3	`FROM (outer)`	The outer query reads the 11 numbered rows of r.
4	`WHERE`	`rnk = 2` keeps only Vikram and Neha. This is why the CTE exists: a window function is calculated in the SELECT step, which runs AFTER WHERE, so you cannot filter on it in the same query level.
5	`SELECT`	Salary AS SecondHighest returns two rows, both 90000.
6	`DISTINCT`	Removes the duplicate, leaving one row: **90000**.

WAY 2 Window function: ROW_NUMBER (wrong when salaries repeat)

T-SQL

```
WITH r AS (
    SELECT Name, Salary,
           ROW_NUMBER() OVER (ORDER BY Salary DESC) AS rn
    FROM Employees)
SELECT Salary AS SecondHighest
FROM r
WHERE rn = 2;
```

RESULT

```
SecondHighest
-------------
95000      <- WRONG. This is the other person at the top, not the second salary.
```

WHY USE IT

ROW_NUMBER numbers every row 1, 2, 3 … with no ties, so it is correct for "the second ROW" or when you have already de-duplicated the salaries. It is also the right tool for "keep one row per group" problems (Q16).

WHEN IT FAILS

Asha and Meera both earn 95000. ROW_NUMBER gives them 1 and 2, so rn = 2 returns 95000. To fix it you must first select the distinct salaries, then number those. Without a tie-breaker in ORDER BY, which of the two people gets 1 is also not guaranteed.

WAY 3 Window function: RANK (returns nothing when salaries repeat)

T-SQL

```
WITH r AS (
    SELECT Name, Salary,
           RANK() OVER (ORDER BY Salary DESC) AS rnk
    FROM Employees)
SELECT Salary AS SecondHighest
FROM r
WHERE rnk = 2;
```

RESULT

```
SecondHighest
-------------
(0 rows)      <- WRONG. Rank 2 does not exist.
```

WHY USE IT

RANK is the right choice when you want competition-style ranking, where a tie takes two places and the next person is correctly in 3rd place, as in a sports table. Use it when skipped ranks are the answer you want.

WHEN IT FAILS

Asha and Meera share rank 1, so the next rank jumps to 3. Rank 2 never appears, and the query returns an empty result. In this data RANK also skips 2, 4, 6 and 9.

WAY 4 MAX with a subquery (no window function)

T-SQL

```
SELECT MAX(Salary) AS SecondHighest
FROM Employees
WHERE Salary < (SELECT MAX(Salary) FROM Employees);
```

RESULT

```
SecondHighest
-------------
90000
```

WHY USE IT

Works on every SQL Server version. The inner query finds 95000, the outer query finds the biggest salary below it. Because the filter is strictly less than, two people sharing 95000 do not cause any problem. It returns NULL (not an error) when there is no second salary.

WHEN IT FAILS

Only good for the second highest. For the third highest you must nest again (MAX below the MAX below the MAX), and for the Nth it becomes impossible. It returns only the salary number, not the employee name, unless you wrap it in another query.

EXECUTION ORDER Way 4: MAX with a subquery

```
SELECT MAX(Salary) AS SecondHighest
FROM Employees
WHERE Salary < (SELECT MAX(Salary) FROM Employees);
```

1 Subquery The inner `SELECT MAX(Salary) FROM Employees` runs first, once, because it does not depend on the outer query. It returns 95000.

2	FROM	The outer query reads all 11 rows of Employees.
3	WHERE	`Salary < 95000` removes Asha and Meera. 9 rows are left.
4	SELECT	With an aggregate and no GROUP BY, the 9 rows form one group. MAX(Salary) of that group is **90000**.

WAY 5 OFFSET FETCH (SQL Server 2012 and later)

T-SQL

```
SELECT DISTINCT Salary
FROM Employees
ORDER BY Salary DESC
OFFSET 1 ROWS FETCH NEXT 1 ROWS ONLY;
```

WHY USE IT

One short statement, and it is easy to change: OFFSET N-1 gives the Nth highest. It reads almost like English, so it is good for a quick check.

WHEN IT FAILS

Remove DISTINCT and it returns 95000, because the second row is the other person with 95000. Duplicates break it. When there is no second value it returns an empty result, not NULL. Needs SQL Server 2012 or later.

WAY 6 TOP twice (the classic pre-2012 answer)

T-SQL

```
SELECT TOP 1 Salary
FROM (SELECT DISTINCT TOP 2 Salary
      FROM Employees
      ORDER BY Salary DESC) AS t
ORDER BY Salary ASC;
```

WHY USE IT

Works on very old versions where OFFSET does not exist. Interviewers like it because it shows you understand TOP and a derived table. The inner query takes the two highest distinct salaries, the outer query takes the smaller of them.

WHEN IT FAILS

If the table has only one distinct salary, the inner query returns one row and the outer query returns that salary, which is the highest, not the second. You get a wrong answer instead of NULL. It is also hard to extend to the Nth.

WAY 7 Correlated subquery (no TOP, no window function)

T-SQL

```
SELECT DISTINCT e1.Salary
FROM Employees e1
WHERE 1 = (SELECT COUNT(DISTINCT e2.Salary)
           FROM Employees e2
           WHERE e2.Salary > e1.Salary);
```

WHY USE IT

Pure standard SQL, runs anywhere. It reads as "a salary that has exactly one distinct salary above it". Change 1 to N-1 and you get the Nth highest.

WHEN IT FAILS

Slow. For every outer row it scans the table again, so the work grows with the square of the row count. Fine for 11 rows, painful for a million.

The window function trap: ROW_NUMBER vs RANK vs DENSE_RANK

Our table has two people at 95000 and two at 90000. This is what each function does with them. Salaries are listed from highest to lowest.

Name	Salary	ROW_NUMBER	RANK	DENSE_RANK
Asha	95000	1	1	1
Meera	95000	2	1	1
Vikram	90000	3	3	2
Neha	90000	4	3	2
Rahul	80000	5	5	3
Priya	80000	6	5	3

- `ROW_NUMBER() = 2` returns **95000**. Wrong. It numbers rows, so the second row is simply the other person at the top.
- `RANK() = 2` returns **nothing**. Wrong. After two people share rank 1, the next rank jumps to 3, so rank 2 does not exist.
- `DENSE_RANK() = 2` returns **90000**. Correct. It never skips a number, so ties share a rank and the next value is always rank 2.

WHICH ONE SHOULD YOU PICK?

- Default in an interview: DENSE_RANK in a CTE. It handles ties, the Nth, per department and names.
- Quick one-liner for "second highest only" without a window function: the MAX subquery.
- If they say "without OFFSET or window functions": TOP twice or the correlated subquery.
- Mention OFFSET FETCH to show you know modern SQL Server.

THE INTERVIEWER'S TRAP

Ask what "second highest" means. If they want the second highest **row** (counting duplicates as separate people), ROW_NUMBER is correct. If they want the second highest **value**, you need DISTINCT or DENSE_RANK. Asking this single question shows you think about data, not just syntax.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Third highest salary**
  Same pattern. Use `rnk = 3` in the DENSE_RANK query or `OFFSET 2` in OFFSET FETCH. The MAX subquery needs another nesting level, which is why it does not scale. Answer on our data: 80000.
- **Nth highest salary**
  Put the DENSE_RANK query inside a function or procedure with an `@N` parameter and use `WHERE rnk = @N`.
- **Names of employees with the second highest salary**
  Remove DISTINCT and select Name and Salary from the DENSE_RANK subquery. Vikram and Neha both appear. ROW_NUMBER would return only one of them, which is wrong.
- **Second highest salary in each department**
  Add `PARTITION BY DeptID` inside OVER. Result on our data: Sales 90000 (Neha), IT 90000 (Vikram), HR 60000 (Sneha).
- **Second lowest salary**
  Change `DESC` to `ASC` (or `OFFSET 1` on an ascending sort).
- **What if there is no second highest salary**
  Wrap the query: `SELECT (SELECT ...) AS SecondHighest` returns NULL instead of an empty result.

ONE LINE TO REMEMBER

Second highest means second highest DISTINCT value. For ties, DENSE_RANK is the safe window function. ROW_NUMBER and RANK both go wrong.

Q2 Types of JOINs in SQL Server

MOST ASKED IMPORTANT Chance of being asked 95%

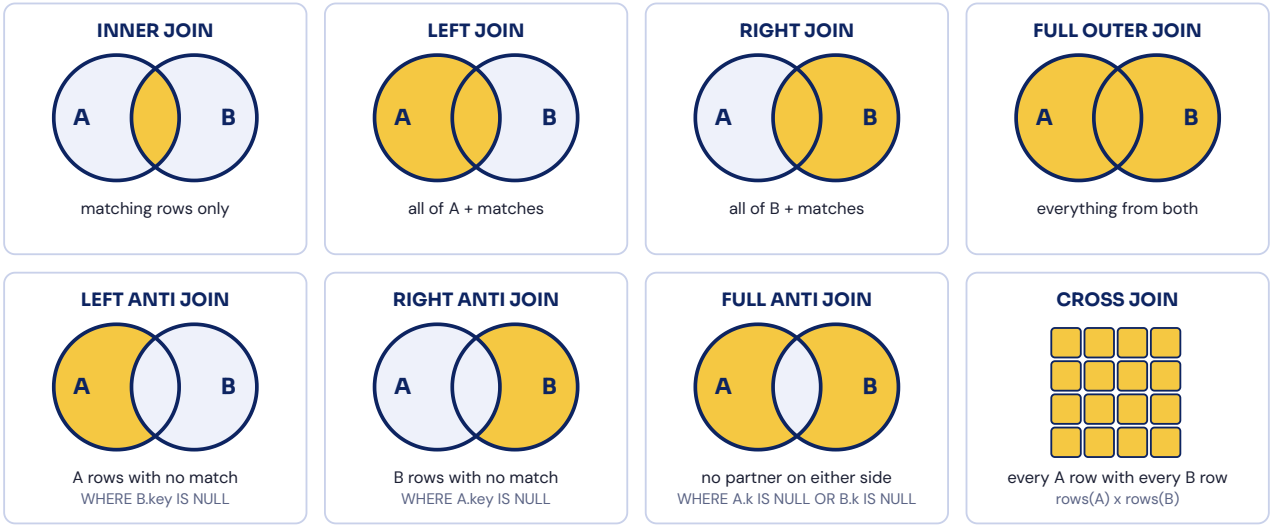
HOW THE INTERVIEWER ASKS IT

- *"What are the types of joins? Explain with an example."*
- *"What is the difference between INNER JOIN and LEFT JOIN?"*
- *"Which join would you use to find rows in one table that are missing in another?"*

Say this first

"A join combines rows from two tables on a matching condition. INNER keeps only matches. LEFT and RIGHT keep every row from one side and fill NULL where the other side has no match. FULL keeps everything from both. CROSS pairs every row with every row. A SELF join is simply a table joined to itself using two aliases."

Picture it



Gold = the rows that come back. A and B are the two tables.

Join	What it returns	Rows with no match
INNER JOIN	Only rows that match in both tables	Dropped
LEFT OUTER JOIN	All rows from the left table, plus matches from the right	Right side shows NULL
RIGHT OUTER JOIN	All rows from the right table, plus matches from the left	Left side shows NULL
FULL OUTER JOIN	All rows from both tables	NULL on whichever side is missing
CROSS JOIN	Every left row paired with every right row (A x B)	No ON clause at all
SELF JOIN	A table joined to itself with two aliases	Example: employee and manager

WAY 1 LEFT JOIN: keep every department, even empty ones

```T-SQL
SELECT d.DeptName, e.Name
FROM Departments d
LEFT JOIN Employees e ON e.DeptID = d.DeptID;
```

WHY USE IT

Finance and Legal have no employees, but they still appear with NULL in the Name column. Use this whenever the left table is the "must keep" side, such as reports that must list every department or every customer.

WHEN IT FAILS

If you filter the right table in the WHERE clause, the NULL rows are removed and the LEFT JOIN silently behaves like an INNER JOIN. `WHERE e.Salary > 60000` throws away Finance and Legal.

EXECUTION ORDER Way 1: LEFT JOIN from Departments to Employees

```
SELECT d.DeptName, e.Name
FROM Departments d
```

```
LEFT JOIN Employees e ON e.DeptID = d.DeptID;
```

1	FROM	Departments is the left table. 5 rows: Sales, IT, HR, Finance, Legal.
2	JOIN ... ON	For each department, SQL Server finds employees with the same DeptID. Sales 4, IT 4, HR 3, Finance 0, Legal 0. That makes 11 matched rows.
3	Outer rows added	A LEFT JOIN now adds one extra row for every left row that found no match, with NULL on the right side. Finance and Legal each get one row. Total 13 rows.
4	WHERE	There is none in this query. If you wrote `WHERE e.Salary > 60000` here, those two NULL rows would be removed, because NULL > 60000 is not true.
5	SELECT	Picks DeptName and Name. 13 rows returned.

DeptName	Name
Finance	NULL
Legal	NULL

WAY 2 Fix: put the filter in ON, not in WHERE

T-SQL

```
SELECT d.DeptName, e.Name
FROM Departments d
LEFT JOIN Employees e
       ON e.DeptID = d.DeptID
      AND e.Salary > 60000;
```

WHY USE IT

The condition now decides which employees match, not which rows survive. Every department still appears, and departments with no high earners show NULL.

WHEN IT FAILS

Do not move a filter on the left table into ON. For an outer join, a left-table condition in ON does not remove left rows. If you want to filter the left table, keep it in WHERE.

EXECUTION ORDER Way 2: the filter moved into ON

```
SELECT d.DeptName, e.Name
FROM Departments d
LEFT JOIN Employees e ON e.DeptID = d.DeptID AND e.Salary > 60000;
```

1	FROM	Departments, 5 rows.
2	JOIN ... ON	A match now needs BOTH conditions. Employees earning more than 60000: Asha, Vikram, Priya (IT), Neha, Rahul, Meera (Sales), Karan (HR). That is 7 matched rows.
3	Outer rows added	Departments with no match get a NULL row. Finance and Legal. Total 9 rows. Nothing is removed afterwards, so every department survives.
4	SELECT	9 rows returned. Compare with the WHERE version, which would return only 7.

WHICH ONE SHOULD YOU PICK?

- Need only matching rows: INNER.
- Need "all of A, with B if it exists": LEFT (put the main table on the left, it reads more naturally than RIGHT).
- Need rows in A that are missing in B: LEFT JOIN then `WHERE b.key IS NULL`.
- Need every combination (calendar x products): CROSS.

THE INTERVIEWER'S TRAP

The classic: "My LEFT JOIN is returning only matched rows." The cause is almost always a WHERE condition on the right-hand table. Say the fix (move it to ON) before they ask.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Find rows in A that are not in B**
  LEFT JOIN with `IS NULL` on the right key. See Q18 for all four ways.
- **Join three or more tables**
  Join them one after another. The logical order follows the ON clauses, so a LEFT JOIN followed by an INNER JOIN on the right table can silently drop rows.
- **Old comma join vs ANSI join**
  `FROM A, B WHERE A.id = B.id` works, but forgetting the WHERE creates an accidental CROSS JOIN. Always use JOIN ... ON.
- **When do you actually use CROSS JOIN**
  Generating combinations, such as every store with every month, or a calendar table with a list of products.
- **Does join order matter**
  For INNER joins the optimizer decides the order. For outer joins the written order changes the result.

ONE LINE TO REMEMBER

INNER keeps matches, LEFT keeps all of the left table. A WHERE filter on the right table turns a LEFT JOIN into an INNER JOIN. Put it in ON.

Q3 How many rows will each join return?

LOGICAL MOST ASKED Chance of being asked 70%

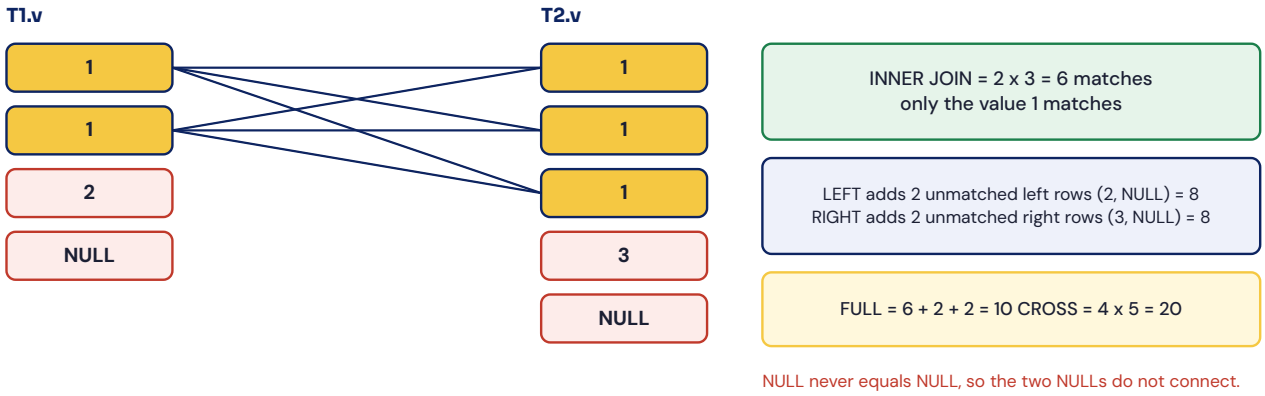
HOW THE INTERVIEWER ASKS IT

- *"Table A has the values 1, 1, 2, NULL. Table B has 1, 1, 1, 3, NULL. How many rows does an INNER JOIN return? LEFT? RIGHT? FULL? CROSS?"*
- *"Both tables have only the value 1, three times and two times. How many rows in each join?"*

Say this first

"I count in two steps. First I count the matching combinations: every row on the left pairs with every equal row on the right, so 2 ones times 3 ones is 6. NULL never equals NULL, so NULLs do not match. Second, I add the rows with no match, depending on the join type."

Picture it



Every 1 on the left pairs with every 1 on the right.

```T-SQL
CREATE TABLE T1 (v INT); INSERT INTO T1 VALUES (1),(1),(2),(NULL);
CREATE TABLE T2 (v INT); INSERT INTO T2 VALUES (1),(1),(1),(3),(NULL);

SELECT COUNT(*) FROM T1 INNER JOIN T2 ON T1.v = T2.v;      -- 6
SELECT COUNT(*) FROM T1 LEFT  JOIN T2 ON T1.v = T2.v;      -- 8
SELECT COUNT(*) FROM T1 RIGHT JOIN T2 ON T1.v = T2.v;      -- 8
SELECT COUNT(*) FROM T1 FULL  JOIN T2 ON T1.v = T2.v;      -- 10
SELECT COUNT(*) FROM T1 CROSS JOIN T2;                     -- 20
```

Join	Rows	How to count it
INNER	6	2 ones on the left x 3 ones on the right. 2, NULL, 3 and NULL find no partner.
LEFT	8	6 matches + 2 unmatched left rows (the value 2 and the NULL)
RIGHT	8	6 matches + 2 unmatched right rows (the value 3 and the NULL)
FULL	10	6 matches + 2 unmatched left + 2 unmatched right
CROSS	20	4 rows x 5 rows. No condition at all, so NULLs do not matter.

Other versions of the same puzzle

Situation	INNER	LEFT	RIGHT	FULL	CROSS
Left has 1,1,1 and right has 1,1	6	6	6	6	6
Left has NULL,NULL and right has NULL,NULL,NULL	0	2	3	5	6
Right table is empty (left has 4 rows)	0	4	0	4	0

WHICH ONE SHOULD YOU PICK?

Use this routine for any puzzle: (1) count matches as left count x right count for each equal value, (2) ignore NULLs for matching, (3) add unmatched rows for the outer side, (4) CROSS is always rows x rows.

THE INTERVIEWER'S TRAP

The interviewer hopes you say NULL matches NULL. It does not, because NULL = NULL is unknown, not true. Also remember: an empty string `''` does match another empty string, because it is a real value. NULL and blank are different.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **All values identical**
  Matches become left count x right count, so every join type gives the same number when nothing is unmatched.
- **Add a WHERE on the right table**
  A LEFT JOIN followed by `WHERE T2.v IS NOT NULL` becomes an INNER JOIN in effect. Recount.
- **Duplicate keys causing inflated totals**
  A join on a non-unique key multiplies rows, and a SUM taken after the join is too high. This is a real production bug, so mention it.

ONE LINE TO REMEMBER

Count matches as left x right per value. NULL never matches NULL. Then add unmatched rows for the outer side.

Q4 DELETE vs TRUNCATE vs DROP

MOST ASKED

Chance of being asked 90%

HOW THE INTERVIEWER ASKS IT

- *"What is the difference between DELETE, TRUNCATE and DROP?"*
- *"Can you roll back a TRUNCATE?"*
- *"Which is faster, DELETE or TRUNCATE, and why?"*

Say this first

"DELETE removes the rows I choose with a WHERE clause and is fully logged. TRUNCATE removes all rows quickly and resets the identity counter. DROP removes the whole table, structure and data. In SQL Server, TRUNCATE can be rolled back inside a transaction, which many people get wrong."

Picture it

DELETE ... WHERE

ID	Name
1	Asha
~~2~~	~~Vikram~~
3	Neha
~~4~~	~~Rahul~~

Removes the rows you choose
WHERE allowed
Logs every row
Identity keeps counting
Triggers fire

TRUNCATE TABLE

ID	Name
0 rows, columns kept	

Removes ALL rows
No WHERE
Logs page deallocation, fast
Identity restarts at 1
No triggers

DROP TABLE

table no longer exists

Removes the table itself
Indexes, constraints
and permissions go too
Cannot undo without a backup

Remember

DELETE = DML
TRUNCATE = DDL
DROP = DDL

All three can be rolled back inside a transaction in SQL Server.

Three commands, three different amounts of damage.

Feature	DELETE	TRUNCATE	DROP
Type of command	DML	DDL	DDL
WHERE clause	Yes	No	No
What it removes	Chosen rows	All rows	The whole table
IDENTITY counter	Not reset	Reset to the seed	Table is gone
Triggers	DELETE triggers fire	Do not fire	Do not fire
Logging	Every row logged	Only page deallocations logged	Metadata change
Rollback inside a transaction	Yes	**Yes, in SQL Server**	Yes
Table referenced by a foreign key	Allowed (rows must be free to delete)	**Not allowed**	Blocked until the FK is dropped
Speed on a big table	Slowest	Very fast	Very fast

WAY 1 Proof that TRUNCATE can be rolled back

```T-SQL
BEGIN TRAN;
    TRUNCATE TABLE Orders;
    SELECT COUNT(*) FROM Orders;    -- 0
ROLLBACK;
SELECT COUNT(*) FROM Orders;        -- 8 again
```

WHY USE IT

Shows the interviewer you have actually tried it. In Oracle and MySQL a TRUNCATE commits automatically and cannot be rolled back. In SQL Server, as long as you are inside an explicit transaction, it can.

WHEN IT FAILS

Rollback only works if you opened the transaction first. A bare TRUNCATE outside a transaction commits immediately.

WHICH ONE SHOULD YOU PICK?

- Remove some rows or fire a trigger: DELETE.
- Empty a staging table fast and reset the identity: TRUNCATE.
- Remove the table itself: DROP.

THE INTERVIEWER'S TRAP

"TRUNCATE cannot be rolled back" is the most common wrong answer. It is true in other databases, not in SQL Server. The other trap: TRUNCATE fails if another table has a foreign key pointing to it, even when that child table is empty.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Empty a table that is referenced by a foreign key**
  Use DELETE, or drop the foreign key, truncate, and recreate the key.
- **Delete millions of rows without filling the log**
  Delete in batches: `DELETE TOP (10000) FROM T WHERE ...` in a loop with a short transaction each time.
- **Truncate only one partition**
  SQL Server 2016 and later: `TRUNCATE TABLE T WITH (PARTITIONS (1))`.
- **DELETE without WHERE vs TRUNCATE**
  Same result, different cost: DELETE logs every row, keeps the identity value, and fires triggers.
- **Drop a table only if it exists**
  `DROP TABLE IF EXISTS dbo.T;` (SQL Server 2016 and later).

ONE LINE TO REMEMBER

DELETE = choose rows, logged, triggers fire. TRUNCATE = all rows, fast, resets identity, CAN be rolled back in SQL Server. DROP = the table is gone.

Q5 WHERE vs HAVING

MOST ASKED

Chance of being asked 85%

HOW THE INTERVIEWER ASKS IT

- *"What is the difference between WHERE and HAVING?"*
- *"Can you use an aggregate function in WHERE?"*
- *"Find departments with more than 3 high-salary employees."*

Say this first

"WHERE filters individual rows before grouping. HAVING filters groups after the aggregation is done. That is why COUNT or SUM can appear in HAVING but not in WHERE."

Picture it



WHERE works on rows BEFORE grouping. HAVING works on groups AFTER grouping.

The row counts are from our Employees table.

WAY 1 WHERE and HAVING together

```tsql
SELECT DeptID, COUNT(*) AS Headcount, AVG(Salary) AS AvgSalary
FROM Employees
WHERE Salary > 50000          -- step 1: filter rows
GROUP BY DeptID               -- step 2: make groups
HAVING COUNT(*) >= 3;         -- step 3: filter groups
```

RESULT

DeptID	Headcount	AvgSalary
1	4	81250
2	4	80000

WHY USE IT

Each filter does its own job. WHERE removes Rajesh (50000) before grouping, which makes HR drop to 2 rows, and then HAVING removes HR because 2 is less than 3. Filtering early with WHERE also means fewer rows to group, so it is faster.

WHEN IT FAILS

Swap them and you break the query. `WHERE COUNT(*) >= 3` gives the error "An aggregate may not appear in the WHERE clause".

EXECUTION ORDER Way 1: WHERE and HAVING together

```tsql
SELECT DeptID, COUNT(*) AS Headcount, AVG(Salary) AS AvgSalary
FROM Employees
WHERE Salary > 50000
GROUP BY DeptID
HAVING COUNT(*) >= 3;
```

1 FROM Reads Employees: 11 rows.

2 WHERE `Salary > 50000` removes Rajesh (50000). 10 rows are left.

3 GROUP BY The 10 rows are put into one group per DeptID.

DeptID	Rows in group
1 (Sales)	4
2 (IT)	4
3 (HR)	2

4	HAVING	`COUNT(*) >= 3` is checked on each group. HR has 2 rows, so the whole HR group is removed. Two groups are left.
5	SELECT	Calculates the output columns for each remaining group.

DeptID	Headcount	AvgSalary
1	4	81250
2	4	80000

WAY 2 Using HAVING for a non-aggregate condition

```T-SQL
SELECT DeptID, COUNT(*) AS Headcount
FROM Employees
GROUP BY DeptID
HAVING DeptID <> 3;
```

WHY USE IT

It works, because DeptID is a grouping column. Good to know so you are not surprised if you see it.

WHEN IT FAILS

It is slower and poor style. SQL Server groups all departments first, then discards HR. A WHERE would have discarded HR rows before grouping. Rule: if the condition does not need an aggregate, use WHERE.

THE INTERVIEWER'S TRAP

The interviewer asks: "Which runs first?" The answer is WHERE, then GROUP BY, then HAVING. See Q10 for the full order. Also, the SELECT alias is not available in HAVING in SQL Server, so repeat the expression.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Departments with average salary above the company average**
  `HAVING AVG(Salary) > (SELECT AVG(Salary) FROM Employees)`.
- **Customers with more than one order**
  `GROUP BY CustomerID HAVING COUNT(*) > 1`.
- **Duplicates**
  The same pattern: `GROUP BY col HAVING COUNT(*) > 1` (see Q6).
- **Can HAVING be used without GROUP BY**
  Yes. The whole table is treated as one group, for example `SELECT COUNT(*) FROM Employees HAVING COUNT(*) > 5`.

ONE LINE TO REMEMBER

WHERE filters rows before grouping. HAVING filters groups after aggregation. Aggregates are not allowed in WHERE.

Q6 Find duplicate records

MOST ASKED | VARIANT FAMILY

Chance of being asked 85%

HOW THE INTERVIEWER ASKS IT

- *"Find duplicate emails in a table."*
- *"Show the complete duplicate rows, not only the repeated value."*
- *"How will you find rows that appear exactly twice?"*

Say this first

"I will first decide what makes two rows duplicates, for example the same email. Then I group by that column and keep only the groups with a count above 1. If you want the full rows, I will use a window function instead."

Picture it

Customers

ID	Name	Email
1	Aman	aman@mail.com
2	Bhavna	bhavna@mail.com
3	Chirag	chirag@mail.com
4	Aman	aman@mail.com
5	Divya	divya@mail.com
6	Esha	esha@mail.com

GROUP BY Email →

After GROUP BY

Email	COUNT(*)
aman@mail.com	2
bhavna@mail.com	1
chirag@mail.com	1
divya@mail.com	1
esha@mail.com	1

HAVING COUNT(*) > 1 keeps only this

aman@mail.com is duplicated

GROUP BY collapses the six rows into five groups. Only one group has a count above 1.

WAY 1 GROUP BY with HAVING

T-SQL

```
SELECT Email, COUNT(*) AS Times
FROM Customers
GROUP BY Email
HAVING COUNT(*) > 1;
```

RESULT

```
Email           Times
--------------  -----
aman@mail.com   2
```

WHY USE IT

The fastest to write and easiest to explain. It shows which values repeat and how many times.

WHEN IT FAILS

It shows only the repeated value, not the full rows. You cannot see CustomerID or City, so you cannot decide which row to keep.

EXECUTION ORDER Way 1: GROUP BY with HAVING

```
SELECT Email, COUNT(*) AS Times
FROM Customers
```

```
GROUP BY Email
HAVING COUNT(*) > 1;
```

1	FROM	Reads Customers: 6 rows.
2	GROUP BY	Rows with the same Email form one group. 5 groups: aman@mail.com (2 rows), and bhavna, chirag, divya, esha (1 row each).
3	HAVING	`COUNT(*) > 1` keeps only the aman@mail.com group.
4	SELECT	Returns Email and COUNT(*): **aman@mail.com, 2**.

WAY 2 COUNT(*) OVER (window function)

T-SQL

```
SELECT *
FROM (SELECT *, COUNT(*) OVER (PARTITION BY Email) AS Cnt
      FROM Customers) AS t
WHERE Cnt > 1;
```

WHY USE IT

Returns every column of every duplicate row, so you can compare them side by side. Both Aman rows (CustomerID 1 and 4) appear.

WHEN IT FAILS

It computes the count over the whole table, so it is heavier on large tables. A window function cannot be used directly in WHERE, so you need the subquery.

WAY 3 Self join

T-SQL

```
SELECT DISTINCT c1.*
FROM Customers c1
JOIN Customers c2
  ON c1.Email = c2.Email
 AND c1.CustomerID <> c2.CustomerID;
```

WHY USE IT

No window functions needed. Works on very old versions and shows you understand joins.

WHEN IT FAILS

If an email repeats many times the join explodes (n x n rows). It also needs a unique column like CustomerID to tell the rows apart.

WAY 4 ROW_NUMBER: show only the extra copies

T-SQL

```
SELECT *
FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY Email ORDER BY CustomerID) AS rn
      FROM Customers) AS t
WHERE rn > 1;
```

WHY USE IT

This returns only the rows you would delete, because the first copy of each email has rn = 1. It is the preparation step for deleting duplicates (Q16).

WHEN IT FAILS

It hides the first copy, so you cannot see the pair together. Use the COUNT OVER way when you want to compare both rows.

WHICH ONE SHOULD YOU PICK?

- Just need to know which values repeat: GROUP BY with HAVING.
- Need full rows to inspect: COUNT(*) OVER.
- Need the rows to delete: ROW_NUMBER with `rn > 1`.

THE INTERVIEWER'S TRAP

"Duplicate" must be defined. Same email? Same name and city? Same whole row? Ask first. Also remember that NULLs are grouped together by GROUP BY, so many NULL emails count as one duplicate group.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Duplicates on more than one column**
  `GROUP BY Name, City HAVING COUNT(*) > 1`.
- **Rows that appear exactly twice**
  `HAVING COUNT(*) = 2`.
- **Rows with no duplicates (unique only)**
  `HAVING COUNT(*) = 1`.
- **Count of duplicate groups**
  Wrap the GROUP BY query and count its rows.
- **Delete duplicates but keep one**
  See Q16.

ONE LINE TO REMEMBER

GROUP BY + HAVING COUNT(*) > 1 finds the value. COUNT(*) OVER (PARTITION BY ...) finds the full rows. ROW_NUMBER finds the extras.

Q7 Clustered vs non-clustered index

MOST ASKED IMPORTANT

Chance of being asked 85%

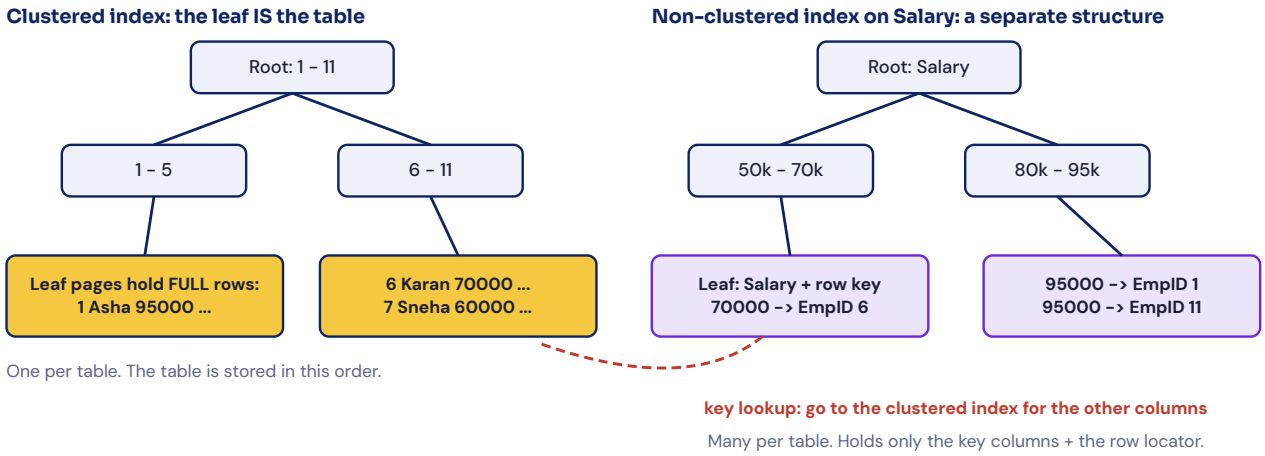
HOW THE INTERVIEWER ASKS IT

- *"What is the difference between a clustered and a non-clustered index?"*
- *"How many clustered indexes can a table have?"*
- *"What is a heap?"*

Say this first

"A clustered index is the table itself, stored in the order of the key, so there can be only one. A non-clustered index is a separate structure with the key and a pointer back to the row, so a table can have many. Think of a clustered index as a dictionary and a non-clustered index as the index at the back of a book."

Picture it



In a clustered index the leaf pages are the table. A non-clustered index only points to it.

Feature	Clustered index	Non-clustered index
What it is	The table data, sorted by the key	A separate B-tree with the key and a row locator
How many per table	**Only 1**	Up to 999
Leaf level holds	The actual data pages	Key values plus a pointer to the row
Created by default for	PRIMARY KEY (if no clustered index exists)	UNIQUE constraint
Extra storage	None	Yes, its own space
Best for	Range scans and ORDER BY on the key	Selective lookups on other columns

```T-SQL
CREATE TABLE Demo (
    ID    INT PRIMARY KEY,        -- creates the clustered index
    Email VARCHAR(100),
    City  VARCHAR(30)
);

CREATE NONCLUSTERED INDEX IX_Demo_Email
ON Demo (Email) INCLUDE (City);   -- separate structure
```

A table with no clustered index is called a **heap**. Its rows are stored in no particular order, and non-clustered indexes point to rows by a physical row ID (RID) instead of a key.

WHICH ONE SHOULD YOU PICK?

Choose the clustered key carefully: it is copied into every non-clustered index. A narrow, unique, steadily increasing key (like an INT identity) is usually best. See Q44.

THE INTERVIEWER'S TRAP

"Primary key and clustered index are the same thing" is wrong. The primary key is a constraint. SQL Server happens to build a clustered index for it by default, but you can write `PRIMARY KEY NONCLUSTERED` and cluster on another column.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Can you have two clustered indexes**
  No. The data can be physically sorted in only one order.
- **What happens when a query needs a column that is not in the non-clustered index**
  A key lookup (or RID lookup) back to the table. See Q29.
- **Index seek vs index scan**
  A seek jumps straight to matching rows, a scan reads the whole index. See Q29.
- **Why does a wide clustered key hurt**
  Every non-clustered index stores a copy of it, so all of them get bigger.
- **Unique index vs unique constraint**
  Functionally the same. The constraint is a logical rule, the index is how it is enforced.

ONE LINE TO REMEMBER

Clustered = the table, sorted by key, only one. Non-clustered = separate structure with a pointer, many. PK is a constraint, not the same as a clustered index.

Q8 Stored procedure vs function

IMPORTANT

Chance of being asked 80%

HOW THE INTERVIEWER ASKS IT

- *"What is the difference between a stored procedure and a function?"*
- *"Can you call a procedure inside a SELECT?"*
- *"Can a function modify data?"*

Say this first

"A function returns a value and can be used inside a SELECT, but it cannot change data. A stored procedure can do anything: insert, update, delete, use transactions and return many result sets, but you call it with EXEC and cannot use it inside a SELECT."

Feature	Stored procedure	User-defined function
Return value	Optional. Can return result sets, OUTPUT parameters and a status code	**Must return a value (scalar) or a table**
How you call it	`EXEC usp_Name`	Inside SELECT, WHERE, JOIN
Can modify tables	Yes	**No**
TRY...CATCH and transactions	Yes	No
Use in a SELECT	No	Yes
Cached plan	Yes	Inline table function: yes. Scalar function: can be slow row by row (improved in SQL Server 2019)

T-SQL

```
-- Procedure
CREATE PROCEDURE dbo.usp_GetDeptEmployees @DeptID INT
AS
BEGIN
    SET NOCOUNT ON;
    SELECT Name, Salary FROM Employees WHERE DeptID = @DeptID;
END;
GO
EXEC dbo.usp_GetDeptEmployees @DeptID = 2;
GO
-- Scalar function
CREATE FUNCTION dbo.fn_Bonus (@Salary INT) RETURNS INT
AS
BEGIN
    RETURN @Salary / 10;
END;
GO
SELECT Name, dbo.fn_Bonus(Salary) AS Bonus FROM Employees;
```

There are three kinds of function in SQL Server. A **scalar** function returns one value. An **inline table-valued** function returns a table from a single SELECT and is usually fast, because the optimizer expands it like a view. A **multi-statement table-valued** function builds a table in steps and often performs poorly because the optimizer cannot see inside it.

THE INTERVIEWER'S TRAP

"Can a function call a procedure?" Generally no. "Can a procedure call a function?" Yes. If they ask why a function cannot change data: functions must have no side effects, because the optimizer may run them many times or in any order.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Inline vs multi-statement table-valued function**
  Inline is almost always faster. Prefer it.

- **Why can a scalar function slow a query**
  Before SQL Server 2019 it ran once per row and blocked parallelism. 2019 can inline simple ones automatically.
- **OUTPUT parameter vs RETURN value**
  RETURN gives only an integer status code. OUTPUT parameters can return any data type.
- **Can a procedure return a table**
  Yes, as a result set, or you capture it with `INSERT INTO #T EXEC proc`.
- **Why use `SET NOCOUNT ON`**
  Stops the "n rows affected" message for every statement, which reduces network chatter.

ONE LINE TO REMEMBER

Function: returns a value, usable in SELECT, cannot change data. Procedure: does anything, called with EXEC, cannot sit inside a SELECT.

Q9 Primary key vs unique key

IMPORTANT

Chance of being asked 80%

HOW THE INTERVIEWER ASKS IT

- *"What is the difference between a primary key and a unique key?"*
- *"Can a unique key have NULL? How many?"*
- *"Can a table have more than one primary key?"*

Say this first

"Both stop duplicates. A table has only one primary key and it cannot hold NULL. A table can have many unique keys, and in SQL Server a unique key allows exactly one NULL."

Feature	Primary key	Unique key
Duplicates	Not allowed	Not allowed
NULL values	**Not allowed**	**One NULL allowed**
How many per table	1	Many
Index created by default	Clustered	Non-clustered
Can be referenced by a foreign key	Yes	Yes
Purpose	Identifies each row	Protects an alternate column from duplicates

T-SQL

```
CREATE TABLE Student (
    RollNo INT PRIMARY KEY,
    Email  VARCHAR(100) UNIQUE,
    Phone  VARCHAR(15)
);

INSERT INTO Student VALUES (1, NULL, '98000');   -- ok
INSERT INTO Student VALUES (2, NULL, '97000');   -- fails: second NULL in a unique column
```

WAY 1 Workaround: allow many NULLs with a filtered unique index

```T-SQL
CREATE UNIQUE INDEX UX_Student_Email
ON Student (Email) WHERE Email IS NOT NULL;
```

WHY USE IT

Uniqueness is enforced only on real emails. Any number of students can have NULL email. This is the standard answer to "I want many NULLs in a unique column".

WHEN IT FAILS

It is an index, not a constraint, so tools that look only for UNIQUE constraints will not see it. It needs SQL Server 2008 or later.

THE INTERVIEWER'S TRAP

MySQL and PostgreSQL allow many NULLs in a unique column. SQL Server allows only one. If you answer from memory of another database, you will lose the point.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Composite primary key**
  A key made of two or more columns together, for example (OrderID, ProductID).
- **Can a primary key be non-clustered**
  Yes: `PRIMARY KEY NONCLUSTERED (ID)`, then create the clustered index elsewhere.
- **Add a primary key to an existing table**
  All values must be unique and NOT NULL first, then `ALTER TABLE ... ADD CONSTRAINT PK ... PRIMARY KEY (...)`.
- **Natural key vs surrogate key**
  A natural key comes from the real world (email, PAN). A surrogate key is a generated number (identity).
- **Foreign key rules**
  A foreign key must point to a primary or unique key. Know ON DELETE CASCADE and what happens without it.

ONE LINE TO REMEMBER

One primary key per table, no NULL. Many unique keys, and SQL Server allows exactly ONE NULL in a unique column.

Q10 Logical order of query execution

LOGICAL

Chance of being asked 70%

HOW THE INTERVIEWER ASKS IT

- *"In what order does SQL Server execute a SELECT query?"*
- *"Why can't I use a column alias in the WHERE clause?"*
- *"Why does ORDER BY accept the alias but WHERE does not?"*

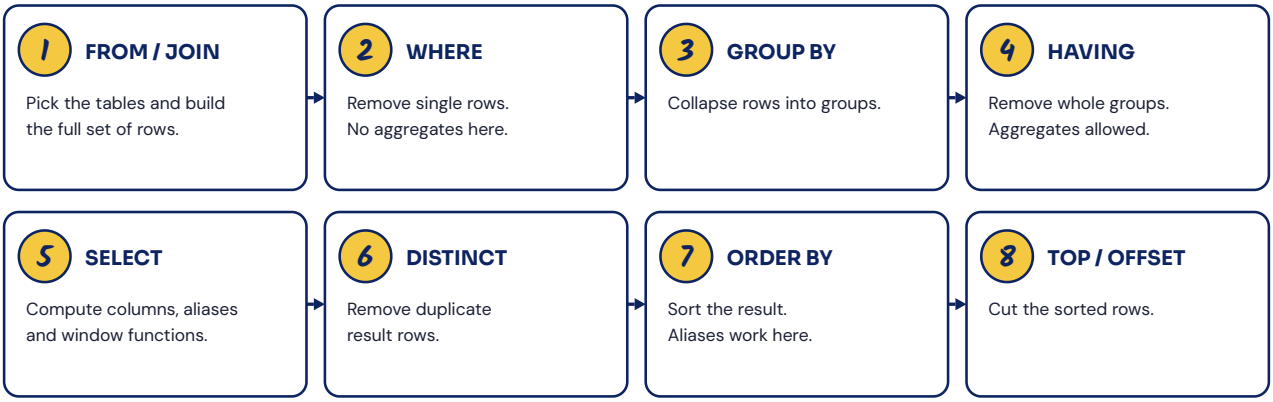
Say this first

"A query is not run in the order we write it. SQL Server logically processes FROM and JOIN first, then WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, and finally TOP or OFFSET. The alias is created in the SELECT step, which comes after WHERE, so WHERE cannot see it. ORDER BY comes after SELECT, so it can."

Picture it

You WRITE it like this: `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY`

SQL Server RUNS it like this: `FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> TOP`



Learn this picture. Every "why does this not work" question is answered by it.

EXECUTION ORDER A full example that uses every clause

```
SELECT TOP 1 DeptID, AVG(Salary) AS AvgSal
FROM Employees
WHERE Salary > 50000
GROUP BY DeptID
HAVING COUNT(*) >= 3
ORDER BY AvgSal DESC;
```

1	FROM	Employees: 11 rows.
2	WHERE	`Salary > 50000` removes Rajesh. 10 rows.
3	GROUP BY	3 groups: DeptID 1 (4 rows), DeptID 2 (4 rows), DeptID 3 (2 rows).
4	HAVING	`COUNT(*) >= 3` removes DeptID 3. 2 groups left.
5	SELECT	Calculates DeptID and AvgSal. The alias AvgSal is born here.
6	DISTINCT	Not used in this query. If it were, it would run here.
7	ORDER BY	Sorts by AvgSal DESC. The alias works here because SELECT has already run.
8	TOP	`TOP 1` keeps the first row: **DeptID 1, 81250**.

DeptID	AvgSal
1	81250
2	80000

Step	Clause	What happens
1	FROM and JOIN	Pick the tables and combine them
2	WHERE	Filter individual rows
3	GROUP BY	Make groups
4	HAVING	Filter the groups
5	SELECT	Compute columns, window functions and aliases
6	DISTINCT	Remove duplicate rows
7	ORDER BY	Sort the result
8	TOP / OFFSET FETCH	Cut the number of rows

T-SQL

```
-- Fails: alias does not exist yet when WHERE runs
SELECT Salary * 12 AS Annual FROM Employees WHERE Annual > 600000;
-- Msg 207: Invalid column name 'Annual'

-- Works: ORDER BY runs after SELECT
SELECT Salary * 12 AS Annual FROM Employees ORDER BY Annual;
```

WAY 1 Fix 1: repeat the expression

T-SQL

```
SELECT Salary * 12 AS Annual
FROM Employees
WHERE Salary * 12 > 600000;
```

WHY USE IT

Simple and fast. No extra layers. Good for short expressions.

WHEN IT FAILS

For a long expression you repeat the logic in two places, and one copy can get edited while the other is forgotten.

WAY 2 Fix 2: CTE or derived table

T-SQL

```
WITH x AS (SELECT Name, Salary * 12 AS Annual FROM Employees)
SELECT Name, Annual FROM x WHERE Annual > 600000;
```

WHY USE IT

The alias is now a real column of the inner query, so the outer query can filter it. Readable and reusable. This is also how you filter on a window function result.

WHEN IT FAILS

Adds a layer of text. It makes no difference to speed. Beginners sometimes think a CTE is faster, and it is not.

WAY 3 Fix 3: CROSS APPLY with VALUES

```tsql
SELECT e.Name, x.Annual
FROM Employees e
CROSS APPLY (VALUES (e.Salary * 12)) AS x(Annual)
WHERE x.Annual > 600000;
```

WHY USE IT

A neat SQL Server idiom that defines the expression once and lets you reuse it in WHERE, GROUP BY and ORDER BY. Shows senior-level knowledge.

WHEN IT FAILS

Less familiar to many interviewers, so be ready to explain it in one sentence.

THE INTERVIEWER'S TRAP

Window functions run in the SELECT step, so you can never write `WHERE ROW_NUMBER() OVER (...) = 1`. This is exactly why Q1 and Q15 need a CTE or subquery.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Why can HAVING use aggregates but WHERE cannot**
  Aggregates exist only after GROUP BY, and WHERE runs before it.
- **Why does ORDER BY accept aliases**
  It runs after SELECT.
- **Where do window functions run**
  In SELECT, after WHERE, GROUP BY and HAVING. They see only the rows that survived those steps.
- **Does the optimizer follow this order**
  No. This is the logical order for meaning. The optimizer may physically reorder steps as long as the result is the same.

ONE LINE TO REMEMBER

FROM, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, TOP. Aliases are born in SELECT, so WHERE cannot see them.

Q11 UNION vs UNION ALL (and INTERSECT, EXCEPT)

MOST ASKED

Chance of being asked 70%

HOW THE INTERVIEWER ASKS IT

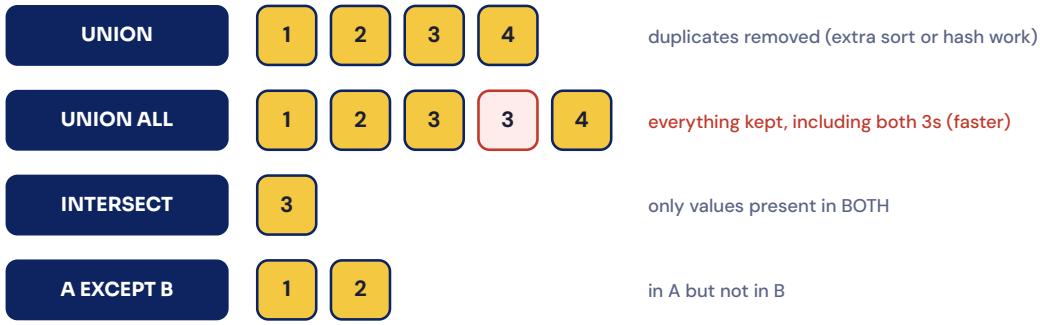
- *"What is the difference between UNION and UNION ALL?"*
- *"Which is faster and why?"*
- *"What are INTERSECT and EXCEPT?"*

Say this first

"UNION combines two result sets and removes duplicates. UNION ALL combines them and keeps everything. Removing duplicates needs an extra sort or hash step, so UNION ALL is faster. I use UNION ALL unless I really need duplicates removed."

Picture it

A = {1, 2, 3} B = {3, 4}



A = {1, 2, 3} and B = {3, 4}. See what each operator returns.

Operator	Result	Duplicates
UNION	All rows from both queries	Removed
UNION ALL	All rows from both queries	**Kept, so it is faster**
INTERSECT	Only rows found in both queries	Removed
EXCEPT	Rows in the first query that are not in the second	Removed

T-SQL

```
-- Customer cities: Jaipur, Delhi, Pune, Jaipur, Jaipur, Mumbai (6 rows)
SELECT City FROM Customers
UNION
SELECT 'Jaipur';
-- 4 rows: Delhi, Jaipur, Mumbai, Pune

SELECT City FROM Customers
UNION ALL
SELECT 'Jaipur';
-- 7 rows: the 6 city rows plus one more Jaipur
```

Rules: both queries need the same number of columns, with compatible data types. Column names come from the first query. ORDER BY can appear only once, at the very end. The output of UNION is often sorted, but that is a side effect, never a guarantee.

THE INTERVIEWER'S TRAP

Columns are matched by **position**, not by name. If you write `SELECT Name, City ...` UNION `SELECT City, Name ...`, SQL Server happily mixes cities into the name column without any error. Always check the column order.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Rows in A that are not in B**
  `A EXCEPT B`. It is NULL-safe and treats NULLs as equal, which is a neat alternative to NOT IN (see Q18).
- **Compare two tables for differences**
  `(A EXCEPT B) UNION ALL (B EXCEPT A)`.
- **Highest and lowest salary employees in one result**
  Two queries joined by UNION ALL, each returning a `Type` column ('Highest' or 'Lowest'). If several people tie on the lowest salary, all of them appear.
- **Combine two tables with different column names**
  Alias them to the same names in each SELECT.
- **UNION vs OR in WHERE**
  UNION ALL of two seeks can beat one OR across two columns (see Q30).

ONE LINE TO REMEMBER

UNION removes duplicates (slower). UNION ALL keeps them (faster). Columns match by position, not name.

Q12 NULL handling: ISNULL vs COALESCE

IMPORTANT

Chance of being asked 65%

HOW THE INTERVIEWER ASKS IT

- *"What is the difference between ISNULL and COALESCE?"*
- *"How do you replace NULL with a default value?"*
- *"Why does `WHERE col = NULL` return nothing?"*

Say this first

"NULL means unknown, not zero and not blank. Any comparison with NULL, even NULL = NULL, is unknown, so I must use IS NULL. To replace NULL I can use ISNULL, COALESCE or CASE. ISNULL is SQL Server only and takes two arguments. COALESCE is standard SQL and takes many."

WAY 1 ISNULL

T-SQL

```
SELECT ISNULL(NULL, 0);                 -- 0

DECLARE @a VARCHAR(3) = NULL;
SELECT ISNULL(@a, 'Hello World');       -- 'Hel'  (cut to VARCHAR(3))
```

WHY USE IT

Short, fast, and well known to SQL Server developers. Good for a simple "if NULL use this".

WHEN IT FAILS

Takes exactly two arguments. The result takes the data type of the first argument, so a longer replacement value gets silently truncated, as shown above. T-SQL only, so it is not portable.

WAY 2 COALESCE

```T-SQL
DECLARE @a VARCHAR(3) = NULL;
SELECT COALESCE(@a, 'Hello World');      -- 'Hello World'
SELECT COALESCE(Phone1, Phone2, Phone3, 'No phone') FROM Contacts;
```

WHY USE IT

Takes any number of arguments and returns the first non-NULL one. Standard SQL, so it works on other databases. The result type follows data type precedence, so no accidental truncation.

WHEN IT FAILS

If one argument is a subquery, it can be evaluated more than once. Be careful when every argument has a different data type: it may raise a conversion error.

WAY 3 CASE expression

```T-SQL
SELECT CASE WHEN Salary IS NULL THEN 0 ELSE Salary END FROM Employees;
```

Situation	Result
`5 + NULL`	NULL
`NULL = NULL`	Unknown (not true)
`WHERE col = NULL`	No rows
`WHERE col IS NULL`	**Correct way**
`10 / NULLIF(0, 0)`	NULL (a safe way to avoid divide by zero)

WHY USE IT

The most flexible option. You can handle several conditions at once, not only NULL.

WHEN IT FAILS

Verbose for a simple replacement. Easy to forget the ELSE, which silently produces NULL.

THE INTERVIEWER'S TRAP

`NOT IN` with a NULL in the list returns nothing at all (see Q18). Also, SUM and COUNT ignore NULLs, but `COUNT(*)` does not (see Q13).

MORE QUESTIONS HIDING INSIDE THIS ONE

- **NULLIF**
  Returns NULL if the two values are equal. Used to avoid a divide-by-zero error.
- **Where do NULLs sort**
  First in ascending order and last in descending order in SQL Server.
- **NULL in joins**
  NULL never matches NULL (see Q3).

- **Does the empty string equal NULL**
  No. They are different values.
- **Concatenating with NULL**
  `'a' + NULL` is NULL by default. `CONCAT('a', NULL)` returns 'a'.

ONE LINE TO REMEMBER

Use IS NULL, never = NULL. ISNULL is two arguments and takes the first argument's type. COALESCE is standard, takes many, and does not truncate.

Q13 COUNT(*) vs COUNT(column) vs COUNT(DISTINCT column)

LOGICAL

Chance of being asked 60%

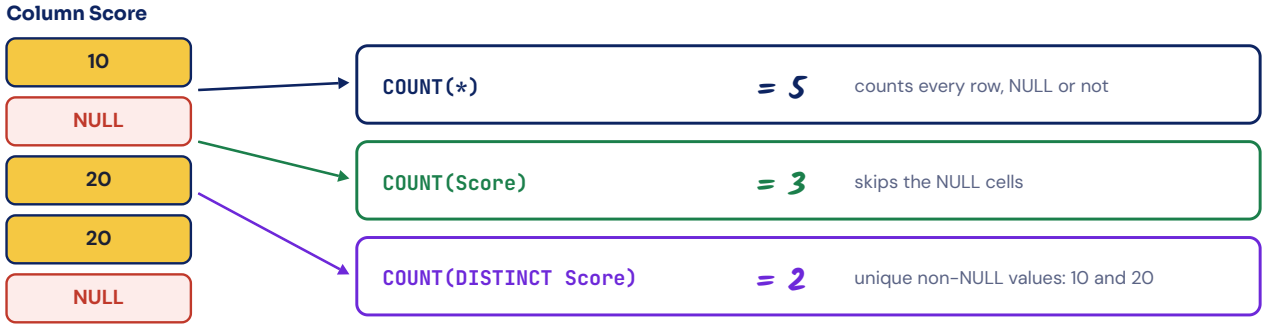
HOW THE INTERVIEWER ASKS IT

- *"What is the difference between COUNT(*), COUNT(col) and COUNT(DISTINCT col)?"*
- *"A column holds 1, NULL, 1, 2. What does each function return?"*
- *"What does SUM return on an empty table?"*

Say this first

"COUNT(*) counts rows, including rows full of NULLs. COUNT(column) counts only the rows where that column is not NULL. COUNT(DISTINCT column) counts the different non-NULL values."

Picture it



Three counts on the same five cells.

T-SQL

```
CREATE TABLE X (col INT);
INSERT INTO X VALUES (1), (NULL), (1), (2);

SELECT COUNT(*)                          AS cnt_star,
       COUNT(col)                        AS cnt_col,
       COUNT(DISTINCT col)               AS cnt_distinct,
       SUM(col)                          AS total,
       AVG(col)                          AS avg_int,
       AVG(CAST(col AS DECIMAL(10,2)))   AS avg_dec
FROM X;
```

RESULT

```
cnt_star  cnt_col  cnt_distinct  total  avg_int  avg_dec
--------  -------  ------------  -----  -------  --------
4         3        2             4      1        1.333333
```

Expression	Result	Why
COUNT(*)	4	Counts all rows, including the one with NULL
COUNT(col)	3	The NULL row is skipped
COUNT(DISTINCT col)	2	Only the values 1 and 2
SUM(col)	4	1 + 1 + 2, the NULL is ignored
AVG(col)	**1**	AVG ignores the NULL, so 4 / 3 = 1.33, but an INT column returns an INT, so SQL Server truncates it to 1
AVG(CAST(col AS DECIMAL(10,2)))	**1.333333**	Cast first to keep the decimals

WAY 1 The real-life use: customers with zero orders

T-SQL

```
SELECT c.Name, COUNT(o.OrderID) AS OrderCount
FROM Customers c
LEFT JOIN Orders o ON o.CustomerID = c.CustomerID
GROUP BY c.CustomerID, c.Name;
```

WHY USE IT

`COUNT(o.OrderID)` ignores the NULL produced by the LEFT JOIN, so a customer with no orders correctly shows 0.

WHEN IT FAILS

Use `COUNT(*)` here and a customer with no orders shows **1**, because the LEFT JOIN still creates one row for them. This is a very common production bug.

THE INTERVIEWER'S TRAP

`SUM` of a column where every value is NULL returns NULL, not 0. On an empty table `COUNT(*)` is 0 but `SUM` is NULL. Wrap with `ISNULL(SUM(col), 0)` when you need a number.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **COUNT(1) vs COUNT(*)**
  Identical result and cost. The 1 is just a constant.
- **Customers and their order count including zero**
  LEFT JOIN with `COUNT(o.OrderID)`, as above.
- **Count of NULLs in a column**
  `COUNT(*) - COUNT(col)`, or `SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END)`.
- **Percentage of NULL values**
  `100.0 * (COUNT(*) - COUNT(col)) / COUNT(*)`.
- **Why is my AVG a whole number**
  Integer division. Multiply by 1.0 or cast to DECIMAL.

ONE LINE TO REMEMBER

COUNT(*) counts rows. COUNT(col) skips NULLs. COUNT(DISTINCT col) counts unique values. AVG on an INT column truncates.

LEVEL 2 OF 4

Junior to Mid

1 to 3 years

14 questions in this level

WHAT THE INTERVIEWER IS REALLY CHECKING

Can you solve a live query with window functions, joins and duplicates, and explain why you chose that approach?

HOW TO WIN THIS LEVEL

This is the live-query round. You will get two or three tables and a business question. Win it by saying your plan first, writing the simplest correct query, and then naming where it would break.

THREE HABITS FOR THIS LEVEL

- Talk while you type. Silence looks like confusion.
- State one assumption out loud: 'I am assuming a customer can have many orders.'
- After your answer, volunteer the second way. It shows range.

Q14 ROW_NUMBER vs RANK vs DENSE_RANK

MOST ASKED IMPORTANT

Chance of being asked 90%

HOW THE INTERVIEWER ASKS IT

- *"What is the difference between ROW_NUMBER, RANK and DENSE_RANK?"*
- *"Salaries are 100, 100, 90. What does each function return?"*
- *"Which one would you use for the Nth highest salary, and why?"*

Say this first

"All three number rows inside an ordered window. ROW_NUMBER gives every row a unique number, so tied rows get different numbers. RANK gives tied rows the same number but then skips ahead. DENSE_RANK gives tied rows the same number and never skips. So ROW_NUMBER is for picking exactly one row, RANK is for sports-style rankings with gaps, and DENSE_RANK is for Nth highest value."

Picture it

Name	Salary	ROW_NUMBER	RANK	DENSE_RANK
Asha	95000	1	1	1
Meera	95000	2	1	1
Vikram	90000	3	3	2
Neha	90000	4	3	2
Rahul	80000	5	5	3
Priya	80000	6	5	3
Karan	70000	7	7	4
Sneha	60000	8	8	5
Amit	60000	9	8	5
Pooja	55000	10	10	6
Rajesh	50000	11	11	7

ROW_NUMBER
Unique numbers. Ties are broken arbitrarily unless you add a tie-breaker.

RANK
Ties share a rank, then SKIP.
Ranks 2, 4, 6 and 9 never appear.
Rank = 2 returns nothing.

DENSE_RANK
Ties share a rank, NO skipping.
Rank 2 always exists.

Ordered by Salary DESC, EmpID

Same data, three functions. The red cells are the numbers RANK skips.

T-SQL

```
SELECT Name, Salary,
       ROW_NUMBER() OVER (ORDER BY Salary DESC, EmpID) AS rn,
       RANK()       OVER (ORDER BY Salary DESC)        AS rnk,
       DENSE_RANK() OVER (ORDER BY Salary DESC)        AS drnk
FROM Employees;
```

EXECUTION ORDER The query above, step by step

```
SELECT Name, Salary,
ROW_NUMBER() OVER (ORDER BY Salary DESC, EmpID) AS rn,
RANK()       OVER (ORDER BY Salary DESC)        AS rnk,
DENSE_RANK() OVER (ORDER BY Salary DESC)        AS drnk
FROM Employees;
```

1	FROM	Reads Employees: 11 rows. There is no WHERE, GROUP BY or HAVING.
2	SELECT (window step)	Each OVER clause is calculated on the full set of 11 rows. For each function SQL Server orders the rows as the OVER clause says, then numbers them. ROW_NUMBER uses (Salary DESC, EmpID), the other two use Salary DESC.

Name	Salary	rn	rnk	drnk
Asha	95000	1	1	1
Meera	95000	2	1	1
Vikram	90000	3	3	2

3 Result

There is no ORDER BY, so SQL Server may return the rows in any order. Add `ORDER BY Salary DESC` if the order matters.

Name	Salary	ROW_NUMBER	RANK	DENSE_RANK
Asha	95000	1	1	1
Meera	95000	2	1	1
Vikram	90000	3	**3**	**2**
Neha	90000	4	**3**	**2**
Rahul	80000	5	5	3
Priya	80000	6	5	3
Karan	70000	7	7	4
Sneha	60000	8	8	5
Amit	60000	9	8	5
Pooja	55000	10	10	6
Rajesh	50000	11	11	7

Look at the red cells: after two people share rank 1, RANK jumps to 3. DENSE_RANK goes to 2. That one jump is the entire reason RANK fails for "second highest".

WAY 1 ROW_NUMBER

WHY USE IT

Always unique, so it is perfect when you need exactly one row: de-duplicating, "latest order per customer", pagination, top 1 per group.

WHEN IT FAILS

It splits ties arbitrarily. Two people with the same salary get different numbers, so "Nth highest" returns the wrong person or the wrong value. Without a tie-breaker in the ORDER BY the numbering can change between runs.

WAY 2 RANK

WHY USE IT

Right when ties should share a position and you want the real position: leaderboards, "top 3 including ties".

WHEN IT FAILS

Leaves gaps (1, 1, 3). A rank that you ask for, such as 2, may not exist at all.

WAY 3 DENSE_RANK

WHY USE IT

No gaps, so the Nth rank always exists if there are N distinct values. The right tool for "Nth highest salary".

WHEN IT FAILS

Not unique: a filter like `drnk <= 3` can return more than 3 rows when there are ties. If the business wants exactly 3 rows, use ROW_NUMBER.

WHICH ONE SHOULD YOU PICK?

- Need exactly one row per group: ROW_NUMBER.
- Need ties to share a place and the true position: RANK.
- Need the Nth distinct value: DENSE_RANK.
- Need to split into equal buckets (quartiles, deciles): NTILE(n).

THE INTERVIEWER'S TRAP

Quick mental test they use: "Values 100, 100, 90. What are the ranks?" ROW_NUMBER 1, 2, 3. RANK 1, 1, 3. DENSE_RANK 1, 1, 2. Practise this until you can say it without thinking.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **NTILE**
  `NTILE(4) OVER (ORDER BY Salary)` splits rows into 4 roughly equal groups (quartiles).
- **Pagination with ROW_NUMBER**
  Number the rows, then filter `rn BETWEEN 21 AND 30`. Today OFFSET FETCH is simpler.
- **Is ORDER BY mandatory inside OVER**
  Yes for all three ranking functions. PARTITION BY is optional.
- **What decides the order of tied rows in ROW_NUMBER**
  Nothing, unless you add a tie-breaker such as `ORDER BY Salary DESC, EmpID`.
- **Top 1 per group**
  ROW_NUMBER with PARTITION BY and `rn = 1` (see Q15).

ONE LINE TO REMEMBER

ROW_NUMBER = unique. RANK = ties share, then gaps. DENSE_RANK = ties share, no gaps. For 100, 100, 90: ROW_NUMBER 1,2,3. RANK 1,1,3. DENSE_RANK 1,1,2.

Q15 Top N per group (top 3 earners in each department)

MOST ASKED | VARIANT FAMILY

Chance of being asked 80%

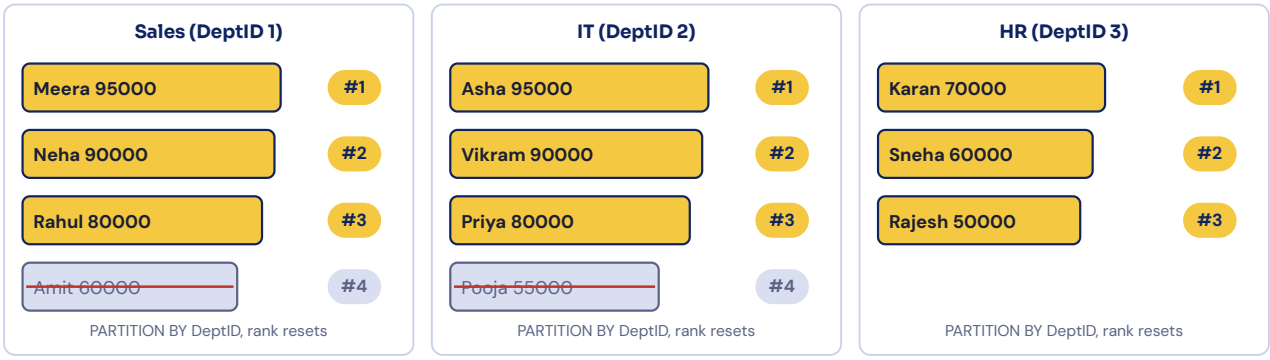
HOW THE INTERVIEWER ASKS IT

- *"Find the top 3 salaries in each department."*
- *"Find the highest paid employee in every department."*
- *"Get the latest order for every customer."*

Say this first

"This is a top-N-per-group problem. I partition the rows by department, order them by salary, rank them inside each partition, and keep ranks up to N. The only decision is which ranking function: DENSE_RANK if I want the top 3 salary values including ties, ROW_NUMBER if I need exactly 3 rows."

Picture it



WHERE rnk <= 3 keeps the gold bars. The grey bars (rank 4) are dropped.

PARTITION BY restarts the ranking inside every department.

WAY 1 DENSE_RANK (top 3 salary values, ties included)

T-SQL

```
WITH r AS (
    SELECT DeptID, Name, Salary,
           DENSE_RANK() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS rnk
    FROM Employees)
SELECT DeptID, Name, Salary
FROM r
WHERE rnk <= 3
ORDER BY DeptID, Salary DESC;
```

RESULT

DeptID	Name	Salary
1	Meera	95000
1	Neha	90000
1	Rahul	80000
2	Asha	95000
2	Vikram	90000
2	Priya	80000
3	Karan	70000
3	Sneha	60000
3	Rajesh	50000

WHY USE IT

The business question is usually "top 3 salaries", not "top 3 people". If two people tie for third, both are shown, which is fair.

WHEN IT FAILS

It can return more than 3 rows per department when there are ties. Amit (Sales, 60000) is excluded here only because his salary is the fourth value in Sales.

EXECUTION ORDER Way 1: DENSE_RANK per department

```
WITH r AS (SELECT DeptID, Name, Salary,
DENSE_RANK() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS rnk
```

```
FROM Employees)
SELECT DeptID, Name, Salary FROM r WHERE rnk <= 3 ORDER BY DeptID, Salary DESC;
```

1	`FROM (inside r)`	Employees: 11 rows.
2	`SELECT (window)`	`PARTITION BY DeptID` splits the rows into 3 partitions. The ranking starts again at 1 inside each one.
3	`WHERE (outer)`	`rnk <= 3` removes Amit and Pooja (rank 4). 9 rows remain.
4	`SELECT`	Returns DeptID, Name, Salary.
5	`ORDER BY`	Sorts the 9 rows by department, then salary.

DeptID	Ranks inside the partition
1 Sales	Meera 1, Neha 2, Rahul 3, Amit 4
2 IT	Asha 1, Vikram 2, Priya 3, Pooja 4
3 HR	Karan 1, Sneha 2, Rajesh 3

WAY 2 ROW_NUMBER (exactly 3 rows)

T-SQL

```
WITH r AS (
    SELECT DeptID, Name, Salary,
           ROW_NUMBER() OVER (PARTITION BY DeptID
                              ORDER BY Salary DESC, EmpID) AS rn
    FROM Employees)
SELECT DeptID, Name, Salary FROM r WHERE rn <= 3;
```

WHY USE IT

You always get at most 3 rows per group. Right for "top 3 orders per customer" or any report with a fixed row count.

WHEN IT FAILS

If two people tie for third, one of them is dropped, decided only by your tie-breaker. That can be unfair, so explain the tie-breaker you chose.

WAY 3 CROSS APPLY with TOP (SQL Server idiom)

T-SQL

```
SELECT d.DeptID, x.Name, x.Salary
FROM Departments d
CROSS APPLY (SELECT TOP 3 e.Name, e.Salary
             FROM Employees e
             WHERE e.DeptID = d.DeptID
             ORDER BY e.Salary DESC) AS x;
```

WHY USE IT

With an index on (DeptID, Salary DESC) SQL Server seeks only 3 rows per department instead of ranking the whole table. Excellent on very large tables and a sign of senior knowledge.

WHEN IT FAILS

CROSS APPLY behaves like an inner join: departments with no employees (Finance, Legal) vanish. Use OUTER APPLY to keep them. Needs a driver table (Departments) to loop over. Ties are cut arbitrarily, like ROW_NUMBER.

WAY 4 Correlated COUNT (no window function)

T-SQL

```
SELECT e1.DeptID, e1.Name, e1.Salary
FROM Employees e1
WHERE (SELECT COUNT(DISTINCT e2.Salary)
       FROM Employees e2
       WHERE e2.DeptID = e1.DeptID
         AND e2.Salary > e1.Salary) < 3;
```

WHY USE IT

Pure standard SQL and works on any database. Reads as "fewer than 3 higher salaries in my own department".

WHEN IT FAILS

Slow on big tables because the subquery runs for every row. Harder to read than the window version.

WHICH ONE SHOULD YOU PICK?

- Top N values, ties allowed: DENSE_RANK.
- Exactly N rows: ROW_NUMBER with a clear tie-breaker.
- Huge table with a good index: CROSS APPLY with TOP.
- Window functions not allowed: correlated COUNT.

THE INTERVIEWER'S TRAP

Do not use `TOP 3 ... GROUP BY DeptID`. TOP applies to the whole result, not to each department. This is why per-group problems need a window function or APPLY.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Highest salary in each department**
  Use `rnk = 1`. If two people tie, both appear. Without windows: join to `(SELECT DeptID, MAX(Salary) mx FROM Employees GROUP BY DeptID)` on both columns.
- **Latest order per customer**
  ROW_NUMBER with `PARTITION BY CustomerID ORDER BY OrderDate DESC, OrderID DESC` and `rn = 1`.
- **Second highest in each department**
  `rnk = 2` (see Q1).
- **Lowest salary per department**
  ORDER BY Salary ASC.
- **Top 2 products in each category**
  Same query with `PARTITION BY CategoryID` and `rnk <= 2`.
- **Departments where the top earner earns more than 90000**
  Filter on `rnk = 1 AND Salary > 90000`.

ONE LINE TO REMEMBER

Top N per group = PARTITION BY group, ORDER BY the measure, filter the rank. DENSE_RANK keeps ties, ROW_NUMBER gives exactly N, CROSS APPLY TOP is the fast SQL Server way.

Q16 Delete duplicates but keep one row

MOST ASKED | VARIANT FAMILY

Chance of being asked 85%

HOW THE INTERVIEWER ASKS IT

- *"Delete the duplicate rows from a table but keep one copy."*
- *"How do you delete duplicates when the table has no primary key?"*
- *"Keep the latest record and delete the older duplicates."*

Say this first

"First I decide what makes a row a duplicate and which copy to keep. Then I number the rows inside each duplicate group and delete everything numbered above 1. I always run it as a SELECT first, and I run the real delete inside a transaction."

Picture it

`ROW_NUMBER() OVER (PARTITION BY Email ORDER BY CustomerID) AS rn`

CustomerID	Name	Email	rn	Action
1	Aman	aman@mail.com	1	keep
2	Bhavna	bhavna@mail.com	1	keep
3	Chirag	chirag@mail.com	1	keep
4	Aman	aman@mail.com	2	DELETE
5	Divya	divya@mail.com	1	keep
6	Esha	esha@mail.com	1	keep

rn = 1 is the first copy.
rn = 2, 3, ... are extra copies.

DELETE FROM d WHERE rn > 1 removes exactly 1 row (ID 4).

Only the copy numbered 2 or more is removed.

WAY 1 CTE with ROW_NUMBER, delete from the CTE

T-SQL

```
WITH d AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY Email
                                 ORDER BY CustomerID) AS rn
    FROM Customers)
DELETE FROM d WHERE rn > 1;
```

RESULT

```
(1 row affected)    -- CustomerID 4, the second 'aman@mail.com'
```

WHY USE IT

One statement, no temporary tables. It works even when the table has no unique key (use `ORDER BY (SELECT NULL)` if you do not care which copy stays). SQL Server lets you delete through a CTE straight into the base table.

WHEN IT FAILS

Deleting from a CTE is T-SQL only, so it is not portable. If the ORDER BY column has ties, which copy survives is arbitrary, so order by a unique column. Always preview with SELECT first.

EXECUTION ORDER **Way 1: delete through a CTE**

```
WITH d AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY Email ORDER BY CustomerID) AS rn FROM Customers)
DELETE FROM d WHERE rn > 1;
```

1	`FROM (inside d)`	Customers: 6 rows.
2	`SELECT (window)`	`PARTITION BY Email` puts the two aman@mail.com rows in one partition. `ORDER BY CustomerID` numbers them 1 and 2. Every other email is alone in its partition and gets 1.
3	`DELETE ... WHERE`	`rn > 1` matches only CustomerID 4. Because the CTE reads one base table, SQL Server deletes that row from Customers.
4	`Result`	**(1 row affected)**. Run the SELECT part alone first to preview what will be deleted.

WAY 2 DELETE with a MIN subquery

T-SQL

```
DELETE FROM Customers
WHERE CustomerID NOT IN (SELECT MIN(CustomerID)
                         FROM Customers
                         GROUP BY Email);
```

WHY USE IT

Standard SQL, easy to explain: "keep the smallest ID per email, delete the rest". To keep the latest copy use MAX.

WHEN IT FAILS

Needs a unique ID column. NOT IN breaks if the subquery can return NULL (see Q18). It is slower on large tables.

WAY 3 Self join delete

T-SQL

```
DELETE c1
FROM Customers c1
JOIN Customers c2
  ON c1.Email = c2.Email
 AND c1.CustomerID > c2.CustomerID;
```

WHY USE IT

Reads as "delete a row if an older row with the same email exists". No window function needed.

WHEN IT FAILS

Needs a unique ID. With many duplicates the join creates many row pairs, so it can be slow.

WAY 4 Copy distinct rows to a new table and swap

```tsql
SELECT DISTINCT * INTO Customers_clean FROM Customers;
DROP TABLE Customers;
EXEC sp_rename 'Customers_clean', 'Customers';
```

WHY USE IT

Good for huge tables where the rows are identical in every column and there is no key. Bulk copy is fast.

WHEN IT FAILS

In our table CustomerID differs, so DISTINCT * would keep both Aman rows. It also loses indexes, constraints, permissions and triggers, and needs downtime. Use it only for fully identical rows.

WHICH ONE SHOULD YOU PICK?

Default answer: the CTE with ROW_NUMBER. Mention MIN or MAX subquery as the portable alternative, and the copy-and-swap for fully identical rows on a huge table.

THE INTERVIEWER'S TRAP

The interviewer waits to see if you ask "which copy should I keep?" and "is this production?". Say you would take a backup or run the delete inside `BEGIN TRAN` and check the row count before `COMMIT`. On millions of rows you would delete in batches to protect the log.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Keep the latest row**
  Change the ROW_NUMBER order to `ORDER BY CustomerID DESC` (or by a date column).
- **Delete every row that has a duplicate, including the original**
  Delete where the email appears in `GROUP BY Email HAVING COUNT(*) > 1`.
- **Duplicates on several columns**
  `PARTITION BY Name, City`.
- **Table has no unique column**
  The CTE method still works with `ORDER BY (SELECT NULL)`.
- **Stop duplicates coming back**
  Add a UNIQUE constraint or unique index on the column.

ONE LINE TO REMEMBER

Number the rows in each duplicate group with ROW_NUMBER, delete where rn > 1. Preview first, delete in a transaction, and decide which copy survives.

Q17 Employees who earn more than their manager

MOST ASKED | VARIANT FAMILY

Chance of being asked 75%

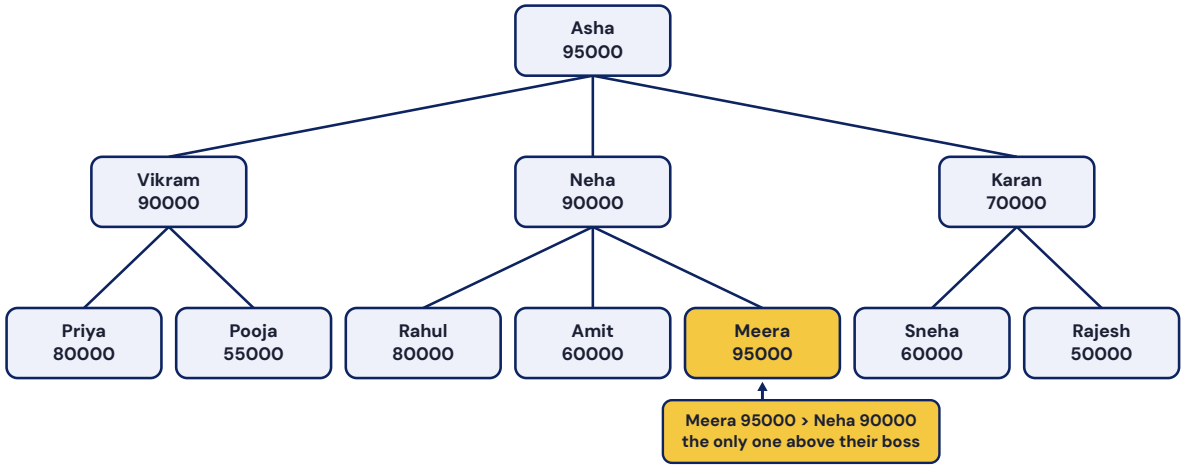
HOW THE INTERVIEWER ASKS IT

- *"Find employees whose salary is higher than their manager's salary."*
- *"Show every employee with the name of their manager."*
- *"Which employees have no manager?"*

Say this first

"Employees and managers sit in the same table, so I will join the table to itself using two aliases. One alias plays the employee, the other plays the manager, and I match the employee's ManagerID to the manager's EmpID."

Picture it



Each employee is compared with the person directly above them in the tree.

WAY 1 Self join

T-SQL

```
SELECT e.Name AS Employee, e.Salary,
       m.Name AS Manager,  m.Salary AS ManagerSalary
FROM Employees e
JOIN Employees m ON e.ManagerID = m.EmpID
WHERE e.Salary > m.Salary;
```

RESULT

```
Employee  Salary  Manager  ManagerSalary
--------  ------  -------  -------------
Meera     95000   Neha     90000
```

WHY USE IT

The standard answer. An INNER JOIN is right here because the top person (Asha, ManagerID NULL) has nobody to compare against and should not appear.

WHEN IT FAILS

Employees whose ManagerID points to a missing row are silently dropped. Sloppy aliases like `e1` and `e2` make it hard for the interviewer to follow, so name them `e` and `m`.

EXECUTION ORDER Way 1: self join

```
SELECT e.Name AS Employee, e.Salary, m.Name AS Manager, m.Salary AS ManagerSalary
FROM Employees e
JOIN Employees m ON e.ManagerID = m.EmpID
WHERE e.Salary > m.Salary;
```

1 FROM — Employees as e: 11 rows.

2 JOIN ... ON — Each e row is matched to the m row whose EmpID equals e.ManagerID. Asha has ManagerID NULL, so she finds no match and an INNER JOIN drops her. 10 rows remain.

Employee	Salary	Manager	ManagerSalary
Vikram	90000	Asha	95000
Neha	90000	Asha	95000
Rahul	80000	Neha	90000
Priya	80000	Vikram	90000
Karan	70000	Asha	95000
Sneha	60000	Karan	70000
Amit	60000	Neha	90000
Pooja	55000	Vikram	90000
Rajesh	50000	Karan	70000
Meera	95000	Neha	90000

3 WHERE — `e.Salary > m.Salary` is true only for Meera (95000 > 90000). 1 row remains.

4 SELECT — Returns Meera, 95000, Neha, 90000.

WAY 2 Correlated subquery

T-SQL

```
SELECT e.Name, e.Salary
FROM Employees e
WHERE e.Salary > (SELECT m.Salary
                  FROM Employees m
                  WHERE m.EmpID = e.ManagerID);
```

WHY USE IT

Reads like the sentence itself and does not need a join. For the top person the subquery finds nothing, the comparison becomes unknown, and the row is correctly left out.

WHEN IT FAILS

If EmpID were not unique the subquery would return several rows and the query would fail. It only returns the employee's columns, so showing the manager's name needs a second subquery.

THE INTERVIEWER'S TRAP

If they ask "show all employees with their manager name, including the top person", an INNER JOIN drops Asha. Switch to LEFT JOIN so the top person appears with a NULL manager, and use `COALESCE(m.Name, 'No manager')` for display.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Every employee with the manager's name**
  LEFT JOIN, same aliases, select `m.Name`.
- **Employees with no manager**
  `WHERE ManagerID IS NULL`. Our data: Asha.
- **Managers who have 3 or more direct reports**
  `SELECT ManagerID FROM Employees GROUP BY ManagerID HAVING COUNT(*) >= 3`. Our data: Asha (1) and Neha (3).
- **Employee with their manager's manager**
  Join the table three times.
- **Whole chain up to the top**
  A recursive CTE (see Q34).
- **Employees earning more than their department average**
  See Q24.

ONE LINE TO REMEMBER

Self join: the same table twice with two aliases, matching ManagerID to EmpID. LEFT JOIN when the top person must still appear.

Q18 Customers who never placed an order

LOGICAL IMPORTANT

Chance of being asked 70%

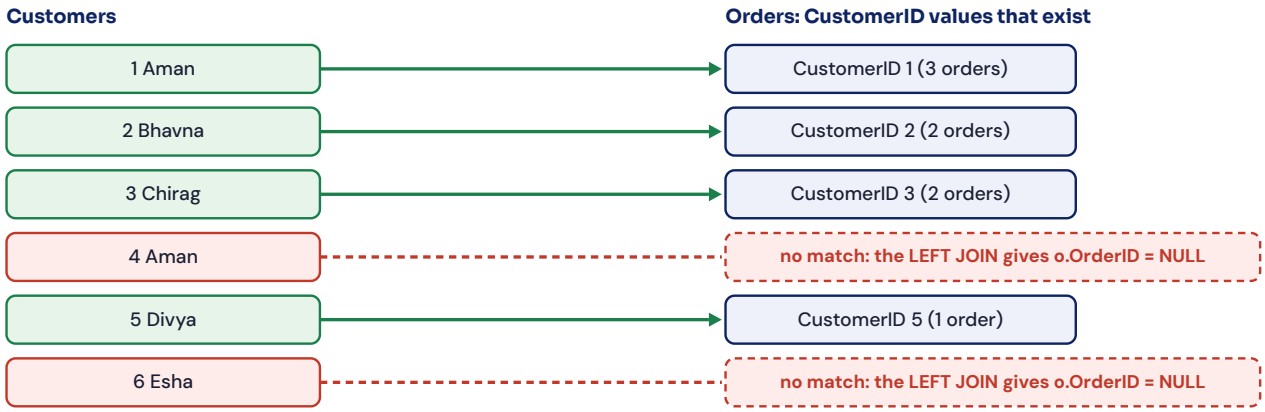
HOW THE INTERVIEWER ASKS IT

- *"Find customers who have not placed any order."*
- *"Find departments that have no employees."*
- *"What is wrong with NOT IN here?"*

Say this first

"This is an anti-join: rows in one table with no match in another. I can write it with LEFT JOIN and IS NULL, NOT EXISTS, NOT IN or EXCEPT. NOT EXISTS is the safest, and I will show you why NOT IN is dangerous when NULLs are present."

Picture it



Green customers have orders. Red customers have none.

WAY 1 LEFT JOIN ... IS NULL

T-SQL

```
SELECT c.CustomerID, c.Name
FROM Customers c
LEFT JOIN Orders o ON o.CustomerID = c.CustomerID
WHERE o.OrderID IS NULL;
```

RESULT

```
CustomerID  Name
----------  ----
4           Aman
6           Esha
```

WHY USE IT

Very readable and fast: the optimizer turns it into an anti-join. Test a column that can never be NULL in a matching row (the Orders primary key).

WHEN IT FAILS

If you test a nullable column on the right, you may filter wrongly. If you also put other conditions on the right table in WHERE, the logic changes (see Q2).

EXECUTION ORDER Way 1: LEFT JOIN ... IS NULL

```
SELECT c.CustomerID, c.Name
FROM Customers c
LEFT JOIN Orders o ON o.CustomerID = c.CustomerID
WHERE o.OrderID IS NULL;
```

1	FROM	Customers: 6 rows.
2	JOIN ... ON	Matches by CustomerID. Customer 1 matches 3 orders, customer 2 matches 2, customer 3 matches 2, customer 5 matches 1. That is 8 matched rows.
3	Outer rows added	Customers 4 and 6 found nothing, so each gets one row with NULL in every Orders column. Total 10 rows.
4	WHERE	`o.OrderID IS NULL` keeps only the rows SQL Server invented in step 3. 2 rows remain.
5	SELECT	Returns **4 Aman** and **6 Esha**.

CustomerID	Name	o.OrderID
4	Aman	NULL
6	Esha	NULL

WAY 2 NOT EXISTS

T-SQL

```
SELECT c.CustomerID, c.Name
FROM Customers c
WHERE NOT EXISTS (SELECT 1 FROM Orders o
                  WHERE o.CustomerID = c.CustomerID);
```

WHY USE IT

The safest and the one most seniors choose. It stops at the first matching row, and it is not affected by NULLs in the Orders table.

WHEN IT FAILS

The correlated syntax looks scary to beginners, and returning rows from only one table needs no join, so there is nothing to fix, only to explain.

WAY 3 NOT IN

T-SQL

```
SELECT CustomerID, Name
FROM Customers
WHERE CustomerID NOT IN (SELECT CustomerID FROM Orders);
```

WHY USE IT

The easiest to read for a beginner. Works correctly while Orders.CustomerID has no NULLs.

WHEN IT FAILS

Add one order with a NULL CustomerID and the query returns **zero rows**. NOT IN (2, 3, NULL) can never be true, because comparing with NULL is unknown. This is the number one reason to avoid NOT IN on nullable columns.

WAY 4 EXCEPT

```tsql
SELECT CustomerID FROM Customers
EXCEPT
SELECT CustomerID FROM Orders;
```

WHY USE IT

Short and set-based. EXCEPT treats NULLs as equal, so a NULL in Orders does not break it.

WHEN IT FAILS

Returns only the columns you compare, so to show names you must join back. It also removes duplicates.

Prove the NOT IN problem in 3 lines

```tsql
INSERT INTO Orders (OrderID, CustomerID, Amount, OrderDate)
VALUES (9, NULL, 100, '2026-03-20');
-- Re-run the NOT IN query above: it now returns no rows at all.
```

WHICH ONE SHOULD YOU PICK?

Say NOT EXISTS first. Say LEFT JOIN IS NULL as the readable alternative. Mention NOT IN only to explain the NULL trap.

THE INTERVIEWER'S TRAP

The interviewer may ask: "Why does NOT IN return nothing when there is a NULL?" Answer: `x NOT IN (2, 3, NULL)` means `x <> 2 AND x <> 3 AND x <> NULL`, and the last part is unknown, so the whole condition is never true.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Departments with no employees**
  Same pattern with Departments and Employees. Our data: Finance and Legal.
- **Products that were never sold**
  Same pattern with Products and OrderItems.
- **Customers who ordered in January but not in February**
  Combine EXISTS for January with NOT EXISTS for February, or use EXCEPT on two filtered queries.
- **Employees who are not managers**
  `NOT EXISTS (SELECT 1 FROM Employees x WHERE x.ManagerID = e.EmpID)`.

ONE LINE TO REMEMBER

Anti-join: NOT EXISTS is safest, LEFT JOIN IS NULL is readable. NOT IN returns nothing if the subquery contains a single NULL.

Q19 Running total (cumulative sum)

VARIANT FAMILY

Chance of being asked 70%

HOW THE INTERVIEWER ASKS IT

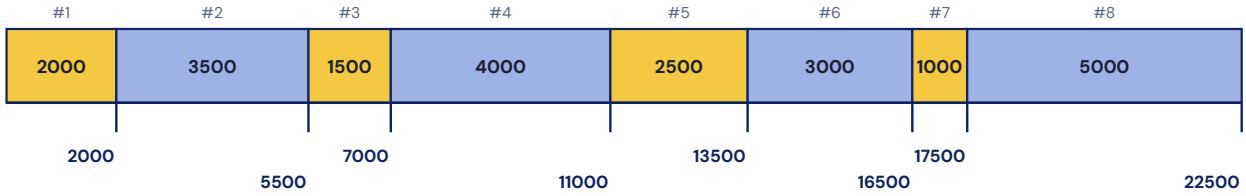
- *"Calculate a running total of sales by date."*
- *"Show each order with the total of all orders so far."*
- *"Calculate a 3-order moving average."*

Say this first

"A running total is a SUM over a window ordered by date. I tell the window to include every row from the start up to the current row. In SQL Server I write the frame explicitly with ROWS, because the default frame is slower and treats equal dates differently."

Picture it

Each order adds a slice. The tick under it is the running total after that order.




```
SUM(Amount) OVER (ORDER BY OrderDate, OrderID ROWS UNBOUNDED PRECEDING)
```

Every tick is the sum of all slices up to that point.

WAY 1 SUM() OVER with an explicit ROWS frame

T-SQL

```
SELECT OrderDate, Amount,
       SUM(Amount) OVER (ORDER BY OrderDate, OrderID
                         ROWS BETWEEN UNBOUNDED PRECEDING
                                  AND CURRENT ROW) AS RunningTotal
FROM Orders;
```

RESULT

```
OrderDate   Amount  RunningTotal
----------  ------  ------------
2026-01-05  2000    2000
2026-01-20  3500    5500
2026-01-22  1500    7000
2026-02-03  4000    11000
2026-02-14  2500    13500
2026-02-28  3000    16500
2026-03-10  1000    17500
2026-03-15  5000    22500
```

WHY USE IT

Fastest method (it uses an in-memory window). The explicit ROWS frame and the OrderID tie-breaker make the result stable even when two orders share a date.

WHEN IT FAILS

Needs SQL Server 2012 or later (ORDER BY inside an aggregate window). Without a tie-breaker, rows with the same date can appear in any order.

EXECUTION ORDER Way 1: SUM() OVER with a ROWS frame

```
SELECT OrderDate, Amount,
SUM(Amount) OVER (ORDER BY OrderDate, OrderID
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS RunningTotal
FROM Orders;
```

1 FROM — Orders: 8 rows. No WHERE or GROUP BY.

2 SELECT (window) — SQL Server sorts the rows by OrderDate, OrderID. For each row the frame runs from the first row down to the current row, and SUM adds the Amounts inside the frame.

Row	Amount	Frame covers	RunningTotal
1	2000	row 1	2000
2	3500	rows 1 to 2	5500
3	1500	rows 1 to 3	7000
8	5000	rows 1 to 8	22500

3 Result — 8 rows, one per order. The aggregate did NOT collapse the rows, which is the difference between a window function and GROUP BY.

WAY 2 SUM() OVER without a frame (default)

```tsql
SELECT OrderDate, Amount,
       SUM(Amount) OVER (ORDER BY OrderDate) AS RunningTotal
FROM Orders;
```

WHY USE IT

Shortest to type. Gives the right numbers when every date is unique.

WHEN IT FAILS

The default frame is RANGE, not ROWS. Rows that share a date are treated as one block, so they all show the same total. On large data RANGE is also slower because it can spill to tempdb on disk. Prefer ROWS.

WAY 3 Correlated subquery (old versions)

```tsql
SELECT o1.OrderDate, o1.Amount,
       (SELECT SUM(o2.Amount)
        FROM Orders o2
        WHERE o2.OrderDate <= o1.OrderDate) AS RunningTotal
FROM Orders o1;
```

WHY USE IT

Works on SQL Server 2008 and earlier, where windowed ORDER BY is not available. Easy to understand.

WHEN IT FAILS

Reads the table again for every row, so the work grows with the square of the rows. Same-date rows get the same total. Avoid on big data.

WHICH ONE SHOULD YOU PICK?

Use the ROWS frame version. Show the subquery only to prove you know the old way, and name its cost.

THE INTERVIEWER'S TRAP

Never use a cursor or loop for a running total. It works but is the slowest option by a wide margin, and interviewers use it as a red flag.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Running total per customer**
  Add `PARTITION BY CustomerID`. The total restarts for each customer.
- **3-order moving average**
  `AVG(Amount) OVER (ORDER BY OrderDate, OrderID ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)`.
- **Percentage of the grand total**
  `Amount * 100.0 / SUM(Amount) OVER ()`.
- **Monthly running total**
  First aggregate to months in a CTE, then apply the window.

- **Reset the total every month**
  `PARTITION BY YEAR(OrderDate), MONTH(OrderDate)`.

ONE LINE TO REMEMBER

SUM(x) OVER (ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW). Always write ROWS, add a tie-breaker.

Q20 LAG and LEAD: compare with the previous or next row

VARIANT FAMILY

Chance of being asked 65%

HOW THE INTERVIEWER ASKS IT

- *"Show each order with the change from the previous order."*
- *"Find month-on-month sales growth."*
- *"Find the days between two consecutive orders of the same customer."*

Say this first

"LAG looks back at an earlier row and LEAD looks ahead, without needing a self join. I give it the column, how many rows to move, and a default for the first row. The window is ordered by date, and partitioned if I need it per customer."

Picture it

Same query, same sort order, every row looks sideways

Order	Amount	LAG(Amount)	LEAD(Amount)
1	2000	NULL	3500
2	3500	2000	1500
3	1500	3500	4000
4	4000	1500	2500
5	2500	4000	3000
6	3000	2500	1000
7	1000	3000	5000
8	5000	1000	NULL

LAG looks one row UP (the previous row).
The first row has nothing above, so NULL.

LEAD looks one row DOWN (the next row).
The last row has nothing below, so NULL.

Change = Amount - LAG(Amount)

LAG looks up, LEAD looks down, in the sort order you choose.

WAY 1 LAG

T-SQL

```
SELECT OrderDate, Amount,
       LAG(Amount) OVER (ORDER BY OrderDate, OrderID) AS PrevAmount,
       Amount - LAG(Amount) OVER (ORDER BY OrderDate, OrderID) AS Change
FROM Orders;
```

RESULT

```
OrderDate   Amount  PrevAmount  Change
----------  ------  ----------  ------
2026-01-05  2000    NULL        NULL
2026-01-20  3500    2000        1500
2026-01-22  1500    3500        -2000
2026-02-03  4000    1500        2500
2026-02-14  2500    4000        -1500
2026-02-28  3000    2500        500
2026-03-10  1000    3000        -2000
2026-03-15  5000    1000        4000
```

WHY USE IT

Short, fast and clear. The first row has no previous row, so LAG returns NULL (or a default you provide: `LAG(Amount, 1, 0)`).

WHEN IT FAILS

Needs SQL Server 2012 or later. If you forget PARTITION BY on a per-customer question, you compare one customer's order with another customer's.

EXECUTION ORDER Way 1: LAG

```
SELECT OrderDate, Amount,
LAG(Amount) OVER (ORDER BY OrderDate, OrderID) AS PrevAmount,
Amount - LAG(Amount) OVER (ORDER BY OrderDate, OrderID) AS Change
FROM Orders;
```

1 FROM — Orders: 8 rows.

2 SELECT (window) — Rows are sorted by OrderDate, OrderID. LAG(Amount) copies the Amount of the previous row. The first row has no previous row, so it gets NULL.

3 SELECT (expression) — `Amount - LAG(Amount)` is then calculated. Any number minus NULL is NULL.

OrderDate	Amount	PrevAmount	Change
2026-01-05	2000	NULL	NULL
2026-01-20	3500	2000	1500
2026-01-22	1500	3500	-2000

WAY 2 Self join on ROW_NUMBER (before 2012)

T-SQL

```
WITH o AS (
    SELECT OrderDate, Amount,
           ROW_NUMBER() OVER (ORDER BY OrderDate, OrderID) AS rn
    FROM Orders)
SELECT cur.OrderDate, cur.Amount,
       cur.Amount - prev.Amount AS Change
FROM o AS cur
LEFT JOIN o AS prev ON prev.rn = cur.rn - 1;
```

WHY USE IT

Works where LAG is missing and proves you understand what LAG does internally.

WHEN IT FAILS

More code, a join, and slower than LAG. It is easy to make an off-by-one mistake.

WHICH ONE SHOULD YOU PICK?

LAG or LEAD in every normal case. Mention the join version only if the interviewer says "without LAG".

THE INTERVIEWER'S TRAP

Month-on-month growth needs a two-step query: first total each month in a CTE, then apply LAG on the monthly totals. Applying LAG on raw orders gives order-to-order change, not month-to-month.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Month-on-month growth %**
  Totals are Jan 7000, Feb 9500, Mar 6000. Growth: Feb +35.7% and Mar -36.8%. Formula: `(Sales - LAG(Sales)) * 100.0 / LAG(Sales)`.
- **Days between consecutive orders of a customer**
  `DATEDIFF(DAY, LAG(OrderDate) OVER (PARTITION BY CustomerID ORDER BY OrderDate), OrderDate)`.
- **Next order date**
  Use LEAD instead of LAG.
- **Look back 2 rows or 12 months**
  `LAG(Amount, 2)` or `LAG(Sales, 12)` on monthly data (year-over-year).
- **First and last value in a group**
  FIRST_VALUE and LAST_VALUE. For LAST_VALUE you must set `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`, otherwise it returns the current row.

ONE LINE TO REMEMBER

LAG looks back, LEAD looks ahead. First row gives NULL unless you give a default. Add PARTITION BY for per-customer comparisons.

Q21 CTE vs temp table vs table variable

IMPORTANT

Chance of being asked 75%

HOW THE INTERVIEWER ASKS IT

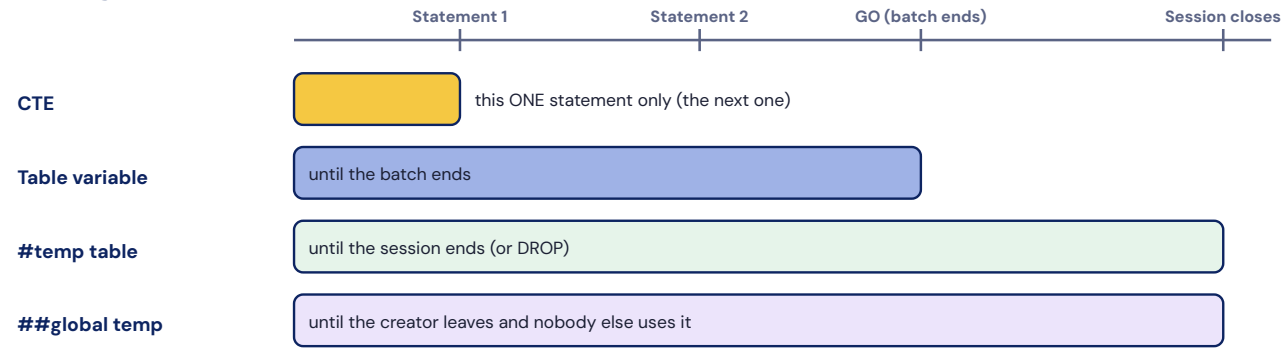
- *"What is the difference between a CTE, a temp table and a table variable?"*
- *"Is a CTE faster than a subquery?"*
- *"When would you use a temp table instead of a CTE?"*

Say this first

"A CTE is just a named query that lasts for one statement. A temp table is a real table in tempdb that lasts for the session. A table variable is like a temp table with a smaller scope and fewer statistics. I use a CTE for readability or recursion, and a temp table when I need to reuse a large intermediate result."

Picture it

How long does each one live?



The shorter the life, the less cleanup you have to think about.

Feature	CTE	#Temp table	@Table variable
Lives for	One statement	The session (or procedure)	The batch or procedure
Stored in	Nowhere. It is only a query definition	tempdb	tempdb
Statistics	Uses the base tables	**Yes, auto-updated**	Limited. Older versions assume 1 row
Indexes	Not applicable	Yes, any time	Only defined with the declaration
Rolled back with the transaction	Not applicable	Yes	**No**
Can be recursive	Yes	No	No
Best for	Readability, recursion	Large results reused several times	Small row counts

```T-SQL
-- CTE
WITH hi AS (SELECT * FROM Employees WHERE Salary > 80000)
SELECT * FROM hi;

-- Temp table
SELECT * INTO #hi FROM Employees WHERE Salary > 80000;
CREATE INDEX ix ON #hi (DeptID);
SELECT * FROM #hi;

-- Table variable
DECLARE @hi TABLE (EmpID INT, Name VARCHAR(30), Salary INT);
INSERT INTO @hi SELECT EmpID, Name, Salary FROM Employees WHERE Salary > 80000;
```

THE INTERVIEWER'S TRAP

"A CTE stores its result in memory and is faster" is a myth. SQL Server does not store a CTE. If you reference it twice, the query inside runs twice. If the result is large and used many times, load it into a temp table once.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Recursive CTE**
  A CTE that references itself, used for hierarchies and number series (see Q34).
- **Why is a table variable not rolled back**
  It is not part of the transaction log in the same way, so its data survives a ROLLBACK.
- **Local vs global temp table**
  `#T` is visible to your session. `##T` is visible to all sessions until the last one disconnects.
- **Can you UPDATE or DELETE through a CTE**
  Yes, in SQL Server (see Q16).
- **Table-valued parameter**
  A table variable passed into a stored procedure, ideal for sending a list of IDs.

ONE LINE TO REMEMBER

CTE = named query for one statement, not stored. #Temp = real table in tempdb with statistics. @Table = lighter, not rolled back.

Q22 Rows to columns: CASE and PIVOT

LOGICAL VARIANT FAMILY

Chance of being asked 60%

HOW THE INTERVIEWER ASKS IT

- *"Show the number of male and female employees in each department as columns."*
- *"Convert rows to columns using PIVOT."*
- *"What if the column values are not known in advance?"*

Say this first

"I can turn rows into columns in two fixed ways: conditional aggregation with CASE, or the PIVOT operator. If the list of columns is not known until runtime, I build the PIVOT query with dynamic SQL."

Picture it

Rows (long)

DeptID	Gender	Count
1	M	2
1	F	2
2	M	1
2	F	3
3	M	2
3	F	1

CASE or PIVOT →

Columns (wide)

DeptID	Male	Female
1	2	2
2	1	3
3	2	1

Each value of Gender becomes a column. PIVOT needs the values written in IN ( ).

Rows become columns. The counts are the same, only the shape changes.

WAY 1 Conditional aggregation with CASE

T-SQL

```
SELECT DeptID,
       SUM(CASE WHEN Gender = 'M' THEN 1 ELSE 0 END) AS Male,
       SUM(CASE WHEN Gender = 'F' THEN 1 ELSE 0 END) AS Female
FROM Employees
GROUP BY DeptID;
```

RESULT

```
DeptID  Male  Female
------  ----  ------
1       2     2
2       1     3
3       2     1
```

WHY USE IT

Works on every database, not only SQL Server. Each column can have its own condition, not just an equality, so it is the most flexible.

WHEN IT FAILS

The column list is typed by hand. A new value (say 'O') needs a new line. With many columns it becomes long.

EXECUTION ORDER Way 1: conditional aggregation with CASE

```
SELECT DeptID,
SUM(CASE WHEN Gender = 'M' THEN 1 ELSE 0 END) AS Male,
SUM(CASE WHEN Gender = 'F' THEN 1 ELSE 0 END) AS Female
FROM Employees
GROUP BY DeptID;
```

1 FROM — Employees: 11 rows.

2 GROUP BY — 3 groups by DeptID: 1 (4 rows), 2 (4 rows), 3 (3 rows).

3 SELECT — Inside each group, CASE turns every row into 1 or 0 for each column, and SUM adds those numbers up.

DeptID	Male (sum of 1s for M)	Female (sum of 1s for F)
1	2	2
2	1	3
3	2	1

WAY 2 PIVOT operator

```tsql
SELECT DeptID, [M] AS Male, [F] AS Female
FROM (SELECT DeptID, Gender FROM Employees) AS src
PIVOT (COUNT(Gender) FOR Gender IN ([M], [F])) AS p;
```

WHY USE IT

Compact and easy to read once you know the pattern: aggregate, the column to spread, and the values.

WHEN IT FAILS

SQL Server only. The IN list is still hard-coded. Always feed PIVOT from a small derived table as above, because any extra column in the source silently becomes a grouping column and splits your rows.

WAY 3 Dynamic PIVOT (column list unknown)

```tsql
DECLARE @cols NVARCHAR(MAX) =
  (SELECT STRING_AGG(QUOTENAME(Gender), ',')
   FROM (SELECT DISTINCT Gender FROM Employees) g);

DECLARE @sql NVARCHAR(MAX) =
  N'SELECT DeptID, ' + @cols + N'
    FROM (SELECT DeptID, Gender FROM Employees) s
    PIVOT (COUNT(Gender) FOR Gender IN (' + @cols + N')) p;';

EXEC sp_executesql @sql;
```

WHY USE IT

The columns follow the data. When a new category appears, the report adapts without anyone editing the query.

WHEN IT FAILS

It is dynamic SQL, so it is harder to debug and a risk if any value comes from user input (QUOTENAME helps). It cannot be used inside a view or function. STRING_AGG needs SQL Server 2017 or later.

WHICH ONE SHOULD YOU PICK?

Fixed small list: CASE (portable) or PIVOT (SQL Server style). Unknown columns: dynamic PIVOT. Say which one you prefer and why.

THE INTERVIEWER'S TRAP

The opposite direction, UNPIVOT, is also asked. And when someone says "I need totals too", mention `GROUP BY ROLLUP` or `GROUPING SETS` rather than a second query.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **UNPIVOT (columns to rows)**
  Use the UNPIVOT operator, or `CROSS APPLY (VALUES ...)` which is more flexible.
- **Monthly sales by region as columns**
  PIVOT with the months in the IN list.

- **Combine values into one comma separated cell**
  `STRING_AGG(Name, ', ')` (SQL Server 2017 and later), or `FOR XML PATH` on older versions.
- **Add a total row**
  `GROUP BY ROLLUP (DeptID)`.

ONE LINE TO REMEMBER

Rows to columns: CASE for any database, PIVOT for T-SQL, dynamic PIVOT when the columns are not fixed.

Q23 Swap M and F in one UPDATE

LOGICAL

Chance of being asked 55%

HOW THE INTERVIEWER ASKS IT

- *"Swap the values 'M' and 'F' in the Gender column using one UPDATE."*
- *"Swap the values of two columns."*
- *"Give different hikes to different departments in one statement."*

Say this first

"I must do this in a single statement. If I ran two updates, the second one would undo the first. A CASE expression decides the new value for every row at once, so nothing is touched twice."

Picture it

Before

Name	Gender
Asha	F
Vikram	M
Neha	F
Rahul	M

```
SET Gender = CASE Gender
WHEN 'M' THEN 'F' WHEN 'F' THEN 'M'
```

After (one UPDATE)

Name	Gender
Asha	**M**
Vikram	**F**
Neha	**M**
Rahul	**F**

CASE reads the OLD value of each row, so both directions change in the same pass. No temporary value needed.

One pass over the table, no temporary value.

WAY 1 CASE expression

T-SQL

```
UPDATE Employees
SET Gender = CASE Gender
                WHEN 'M' THEN 'F'
                WHEN 'F' THEN 'M'
                ELSE Gender
             END;
```

WHY USE IT

One statement, so it is atomic: either all rows change or none do. The `ELSE Gender` keeps any other value as it is.

WHEN IT FAILS

If you forget the ELSE, any value that is not 'M' or 'F' (including NULL) becomes NULL, which is silent data loss.

EXECUTION ORDER Way 1: one UPDATE with CASE

```
UPDATE Employees
SET Gender = CASE Gender WHEN 'M' THEN 'F' WHEN 'F' THEN 'M' ELSE Gender END;
```

1	Target	UPDATE Employees. There is no WHERE, so all 11 rows are targeted.
2	SET (evaluate)	For each row SQL Server evaluates the CASE using the value the row had BEFORE the statement. M becomes F and F becomes M at the same moment, so nothing is swapped twice.
3	Write	The new values are written. **(11 rows affected)**.

WAY 2 Two updates with a temporary value

T-SQL

```
BEGIN TRAN;
UPDATE Employees SET Gender = 'X' WHERE Gender = 'M';
UPDATE Employees SET Gender = 'M' WHERE Gender = 'F';
UPDATE Employees SET Gender = 'F' WHERE Gender = 'X';
COMMIT;
```

WHY USE IT

Easy to understand. Shows you know why the naive two-update version fails.

WHEN IT FAILS

Three passes over the table, and without the transaction a failure halfway leaves the data corrupted. It can also clash with a CHECK constraint when 'X' is not allowed.

WAY 3 ASCII arithmetic trick

```tsql
UPDATE Employees
SET Gender = CHAR(ASCII('M') + ASCII('F') - ASCII(Gender));
```

WHY USE IT

A clever one-liner: 77 + 70 = 147, so 147 minus M gives F, and 147 minus F gives M.

WHEN IT FAILS

Breaks on any other value (NULL, 'O', lowercase). Unreadable to the next developer. Treat it as a party trick, not a production solution.

WHICH ONE SHOULD YOU PICK?

CASE with an ELSE. Always.

THE INTERVIEWER'S TRAP

The interviewer may add: "Why can't you simply run two updates?" Because after the first update every row is 'F', so the second update turns them all back to 'M'.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Swap the values of two columns**
  `UPDATE T SET a = b, b = a`. In SQL Server the right side uses the old values, so this works in one statement.
- **Different hike per department**
  `SET Salary = CASE DeptID WHEN 1 THEN Salary * 1.10 WHEN 2 THEN Salary * 1.05 ELSE Salary END`.
- **Flip a bit column**
  `SET IsActive = 1 - IsActive`.
- **Update using another table**
  `UPDATE e SET e.Salary = n.NewSalary FROM Employees e JOIN NewSalaries n ON n.EmpID = e.EmpID`.
- **Update only the top N rows**
  `UPDATE TOP (100) ...`.

ONE LINE TO REMEMBER

One UPDATE with CASE and an ELSE that keeps the old value. Two separate updates would undo each other.

Q24 Correlated subquery, and EXISTS vs IN

IMPORTANT

Chance of being asked 60%

HOW THE INTERVIEWER ASKS IT

- *"Find employees who earn more than the average salary of their own department."*
- *"What is a correlated subquery?"*
- *"Difference between EXISTS and IN?"*

Say this first

"A correlated subquery refers to a column of the outer query, so it is evaluated once for every outer row. Here, for each employee I compare their salary to the average of their own department. I can also write it with a join to a pre-calculated average, or with a window function."

WAY 1 Correlated subquery

T-SQL

```
SELECT e.Name, e.DeptID, e.Salary
FROM Employees e
WHERE e.Salary > (SELECT AVG(1.0 * e2.Salary)
                  FROM Employees e2
                  WHERE e2.DeptID = e.DeptID);
```

RESULT

```
Name    DeptID  Salary
------  ------  ------
Asha    2       95000
Vikram  2       90000
Neha    1       90000
Karan   3       70000
Meera   1       95000
```

WHY USE IT

A direct translation of the sentence, so it is easy to explain and easy to get right.

WHEN IT FAILS

Logically it runs once per employee. The optimizer often rewrites it into a join, but not always, and it can be slow on large tables without an index on DeptID.

EXECUTION ORDER Way 1: correlated subquery

```
SELECT e.Name, e.DeptID, e.Salary
FROM Employees e
WHERE e.Salary > (SELECT AVG(1.0 * e2.Salary) FROM Employees e2 WHERE e2.DeptID = e.DeptID);
```

1 FROM — Employees as e: 11 rows.

2 WHERE — For each e row the subquery is evaluated with that row's DeptID (logically once per row, although the optimizer often calculates each department average only once).

DeptID	Department average	Who is above it
1	81250	Neha 90000, Meera 95000
2	80000	Asha 95000, Vikram 90000
3	60000	Karan 70000

3 SELECT — Returns 5 rows: Asha, Vikram, Neha, Karan, Meera. Priya (80000) is not above 80000, and Sneha (60000) is not above 60000.

WAY 2 Join to a derived table of averages

```tsql
SELECT e.Name, e.DeptID, e.Salary
FROM Employees e
JOIN (SELECT DeptID, AVG(1.0 * Salary) AS AvgSal
      FROM Employees GROUP BY DeptID) a
  ON a.DeptID = e.DeptID
WHERE e.Salary > a.AvgSal;
```

WHY USE IT

Each department average is computed once, and then compared. Usually the fastest and still very readable.

WHEN IT FAILS

Write `AVG(Salary)` without `1.0 *` and an INT average is truncated (for example 81250.5 becomes 81250), which can include or exclude the wrong people. Always multiply by 1.0.

WAY 3 Window function

```tsql
SELECT Name, DeptID, Salary
FROM (SELECT Name, DeptID, Salary,
             AVG(1.0 * Salary) OVER (PARTITION BY DeptID) AS AvgSal
      FROM Employees) t
WHERE Salary > AvgSal;
```

WHY USE IT

A single pass over the table. No join and no subquery that repeats per row.

WHEN IT FAILS

You cannot filter on the window result in the same SELECT, so you need the derived table or a CTE (see Q10).

EXISTS vs IN in one table

Topic	EXISTS	IN
What it tests	Whether at least one row exists	Whether a value is in a list
Stops early	Yes, at the first match	Not always
NULL in the subquery	**Safe**	NOT IN returns nothing (see Q18)
Typical use	Correlated checks: "has any order"	Short lists or a simple subquery
Performance	Usually the same plan in modern versions	Usually the same plan in modern versions

WHICH ONE SHOULD YOU PICK?

Dept average problems: join to a derived table or use the window version. Existence checks: EXISTS. Say that the optimizer often converts IN and EXISTS into the same plan, so correctness (NULLs) matters more than speed.

THE INTERVIEWER'S TRAP

Department averages on INT salaries. Dept 1 is exactly 81250 here, so everything looks fine, but the trap appears the moment an average has a fraction. Multiply by 1.0.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Employees earning more than the company average**
  `WHERE Salary > (SELECT AVG(1.0 * Salary) FROM Employees)` . A plain subquery, not correlated.
- **Departments whose average is above the company average**
  `GROUP BY DeptID HAVING AVG(1.0 * Salary) > (SELECT AVG(1.0 * Salary) FROM Employees)` .
- **Nth highest with a correlated subquery**
  See Way 5 of Q1.
- **Rewrite a correlated subquery as a join**
  As in Way 2 above.

ONE LINE TO REMEMBER

Correlated subquery = refers to the outer row, runs per row logically. Faster: join to pre-computed averages or use AVG OVER. Multiply by 1.0 to avoid integer averages.

Q25 Triggers: AFTER vs INSTEAD OF

IMPORTANT

Chance of being asked 55%

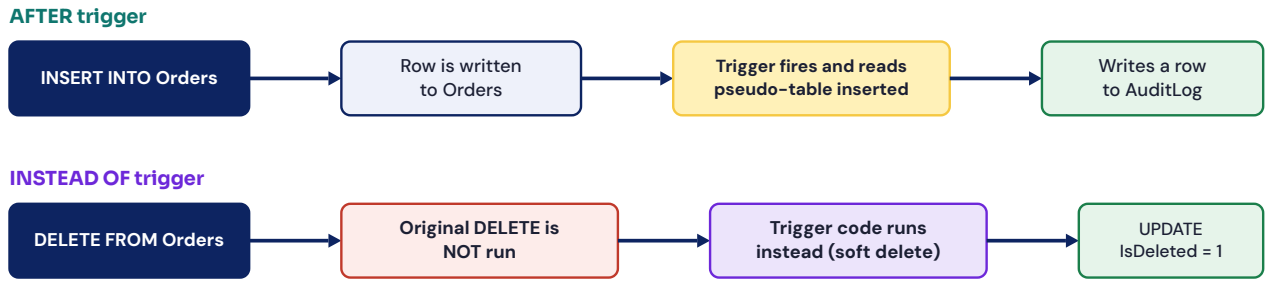
HOW THE INTERVIEWER ASKS IT

- *"What is a trigger? What types are there?"*
- *"What are the inserted and deleted tables?"*
- *"Write a trigger to log salary changes."*

Say this first

"A trigger is code that runs automatically when an INSERT, UPDATE or DELETE happens. An AFTER trigger runs once the statement has succeeded. An INSTEAD OF trigger replaces the statement, and is mostly used on views. Inside a trigger two special tables, inserted and deleted, hold the new and the old rows."

Picture it



AFTER runs after the change. INSTEAD OF replaces the change.

Feature	AFTER trigger	INSTEAD OF trigger
When it runs	After the statement and constraint checks	Instead of the statement
Typical use	Audit logs, enforcing rules	Making a complex view updatable
On a view	No	Yes
Several per table	Yes	Only one per action

T-SQL

```
CREATE TABLE SalaryAudit (EmpID INT, OldSalary INT, NewSalary INT, ChangedOn DATETIME);
GO
CREATE TRIGGER trg_Employees_SalaryAudit
ON Employees
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO SalaryAudit (EmpID, OldSalary, NewSalary, ChangedOn)
    SELECT d.EmpID, d.Salary, i.Salary, GETDATE()
    FROM deleted  d
    JOIN inserted i ON i.EmpID = d.EmpID
    WHERE d.Salary <> i.Salary;
END;
```

In an UPDATE trigger, **deleted** holds the old version of each changed row and **inserted** holds the new version. For an INSERT only inserted has rows, for a DELETE only deleted has rows.

THE INTERVIEWER'S TRAP

A trigger fires **once per statement, not once per row**. If one UPDATE changes 500 rows, `inserted` has 500 rows. Code written for a single row, such as `DECLARE @id INT = (SELECT EmpID FROM inserted)`, fails with "Subquery returned more than 1 value". Always write the trigger as a set-based INSERT … SELECT, as above.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **DDL trigger**
  Fires on CREATE, ALTER or DROP. Used to block or log schema changes.
- **Trigger vs constraint**
  Use a constraint when it can express the rule. Constraints are faster and cannot be forgotten.
- **Disable a trigger**
  `DISABLE TRIGGER trg_Employees_SalaryAudit ON Employees;`
- **Nested and recursive triggers**
  A trigger that fires another trigger. Know that they exist and can cause surprises.
- **Modern alternatives for auditing**
  Temporal tables, Change Data Capture, or the OUTPUT clause.

ONE LINE TO REMEMBER

Triggers fire per statement, not per row. Use the inserted and deleted tables with set-based code. AFTER for auditing, INSTEAD OF for views.

Q26 Error handling and transactions (TRY...CATCH)

IMPORTANT

Chance of being asked 55%

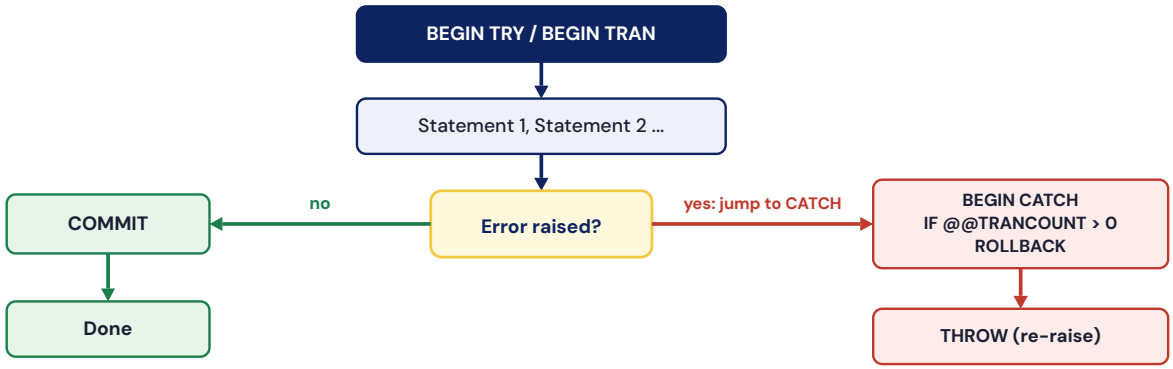
HOW THE INTERVIEWER ASKS IT

- *"How do you handle errors in T-SQL?"*
- *"What is the difference between THROW and RAISERROR?"*
- *"How do you make a bank transfer all-or-nothing?"*

Say this first

"I wrap the work in a transaction inside a TRY block. If anything fails, control jumps to CATCH, where I roll back and re-throw the error so the caller knows. I also set XACT_ABORT ON so that a runtime error rolls back the whole transaction automatically."

Picture it



An error jumps straight from TRY to CATCH. COMMIT is skipped.

WAY 1 TRY...CATCH with a transaction (the modern pattern)

T-SQL

```
SET XACT_ABORT ON;
BEGIN TRY
    BEGIN TRAN;
        UPDATE Accounts SET Balance = Balance - 500 WHERE AccountID = 1;
        UPDATE Accounts SET Balance = Balance + 500 WHERE AccountID = 2;
    COMMIT;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK;
    THROW;      -- re-raise the original error
END CATCH;
```

WHY USE IT

The standard, readable pattern. Both updates succeed or neither does. THROW with no arguments re-raises the original error number, message and line.

WHEN IT FAILS

Some errors are not caught by TRY...CATCH in the same scope, such as compile errors and severity 20 or higher errors that kill the connection. Without XACT_ABORT, some errors leave the transaction open, so check `XACT_STATE()` in CATCH for complex code.

WAY 2 @@ERROR after each statement (old style)

```tsql
BEGIN TRAN;
UPDATE Accounts SET Balance = Balance - 500 WHERE AccountID = 1;
IF @@ERROR <> 0 BEGIN ROLLBACK; RETURN; END
UPDATE Accounts SET Balance = Balance + 500 WHERE AccountID = 2;
IF @@ERROR <> 0 BEGIN ROLLBACK; RETURN; END
COMMIT;
```

WHY USE IT

Works on SQL Server 2000 and earlier. You will see it in legacy code.

WHEN IT FAILS

@@ERROR is reset by every statement, so it is easy to read the wrong one. You must repeat the check after every statement, and there is no central place to handle errors.

WAY 3 SET XACT_ABORT ON on its own

```tsql
SET XACT_ABORT ON;
BEGIN TRAN;
    UPDATE Accounts SET Balance = Balance - 500 WHERE AccountID = 1;
    UPDATE Accounts SET Balance = Balance + 500 WHERE AccountID = 2;
COMMIT;
```

Feature	THROW	RAISERROR
Introduced	SQL Server 2012	Old, still supported
Re-raise the caught error	`THROW;` with no arguments	Needs the error details copied by hand
Custom message formatting	Basic	printf style placeholders
Severity	Always 16	You choose
Statement before it	Needs a semicolon	Not required

WHY USE IT

Any runtime error aborts the batch and rolls the transaction back automatically. Tiny and safe for simple cases.

WHEN IT FAILS

You get no chance to log or clean up, and the caller just sees an error. Best used together with TRY...CATCH, not instead of it.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Savepoints**
  `SAVE TRANSACTION sp1` and `ROLLBACK TRANSACTION sp1` undo only part of a transaction.
- **Nested transactions and @@TRANCOUNT**
  COMMIT only decrements the counter. A ROLLBACK undoes everything and sets it to 0.
- **XACT_STATE()**
  Returns -1 when the transaction is doomed and can only be rolled back.

- **ACID**
  Atomicity, Consistency, Isolation, Durability. Know one example for each.
- **Retry on deadlock**
  Catch error 1205 and retry (see Q32).

ONE LINE TO REMEMBER

BEGIN TRY with BEGIN TRAN, then in CATCH: ROLLBACK if @@TRANCOUNT > 0 and THROW. Add SET XACT_ABORT ON.

Q27 Views: can you update data through a view?

IMPORTANT

Chance of being asked 50%

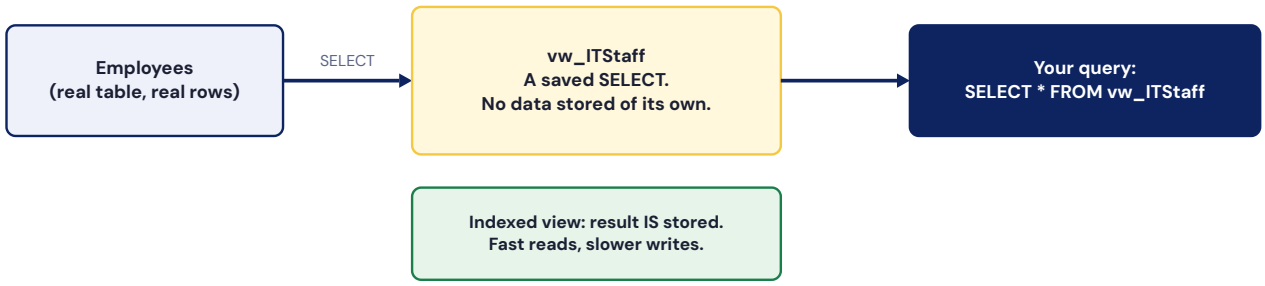
HOW THE INTERVIEWER ASKS IT

- *"What is a view? Why use one?"*
- *"Can you INSERT or UPDATE through a view?"*
- *"What is an indexed view?"*

Say this first

"A view is a saved SELECT that behaves like a virtual table. It stores no data. I use views to hide complex joins and to expose only the columns a user is allowed to see. A simple view on one table can be updated. A view with joins, aggregates or DISTINCT usually cannot."

Picture it



A normal view stores the query, not the data.

T-SQL

```
CREATE VIEW dbo.vw_ITStaff
AS
SELECT EmpID, Name, Salary, DeptID
FROM dbo.Employees
WHERE DeptID = 2
WITH CHECK OPTION;
GO

UPDATE dbo.vw_ITStaff SET Salary = 92000 WHERE EmpID = 2;     -- works
UPDATE dbo.vw_ITStaff SET DeptID = 3 WHERE EmpID = 2;         -- fails
```

The second update fails because of `WITH CHECK OPTION`: after the change, the row would no longer belong to the view (DeptID is no longer 2), and SQL Server refuses it.

Feature	Standard view	Indexed view (materialized)
Stores data	No	**Yes, the result is stored physically**
Speed	Runs the query every time	Reads the stored result, can be much faster
Requirements	None	`WITH SCHEMABINDING`, a unique clustered index, and restrictions on the query
Cost	Free	Storage, and slower writes to the base tables
Best for	Simplifying and securing access	Heavy repeated aggregations

THE INTERVIEWER'S TRAP

A view cannot have `ORDER BY` unless it also uses TOP or OFFSET. Also, `SELECT *` inside a view is frozen at creation time: columns added later to the table do not appear until you refresh the view with `sp_refreshview`. List the columns explicitly.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Rules for updating through a view**
  One base table per statement, no aggregates, GROUP BY, DISTINCT or UNION, and every required column must be available.
- **INSTEAD OF trigger on a view**
  Makes a complex view updatable by handling the insert yourself (see Q25).
- **View vs CTE vs temp table**
  A view is saved and reusable, a CTE lasts one statement, a temp table stores data (see Q21).
- **Security with views**
  Grant users access to the view and deny the base table.
- **SCHEMABINDING**
  Stops anyone from altering a base table in a way that would break the view.

ONE LINE TO REMEMBER

A view is a saved SELECT with no data of its own. Single-table views are updatable. WITH CHECK OPTION blocks changes that move a row out of the view. Indexed views store the result.

LEVEL 3 OF 4

Mid to Senior

3 to 5 years

14 questions in this level

WHAT THE INTERVIEWER IS REALLY CHECKING

Can you make a query fast and safe, read an execution plan, and reason about locks, indexes and concurrency?

HOW TO WIN THIS LEVEL

The question changes from 'can you write it' to 'can you make it fast, correct under load, and maintainable'. Answer with a process: measure, find the cause, fix one thing, measure again.

THREE HABITS FOR THIS LEVEL

- Say 'I would check the actual execution plan first' before suggesting any fix.
- Never recommend NOLOCK as a performance fix without naming the risk.
- Use real numbers from your own projects: rows, seconds, reads.

Q28 A query is slow. How do you troubleshoot it?

MOST ASKED IMPORTANT

Chance of being asked 85%

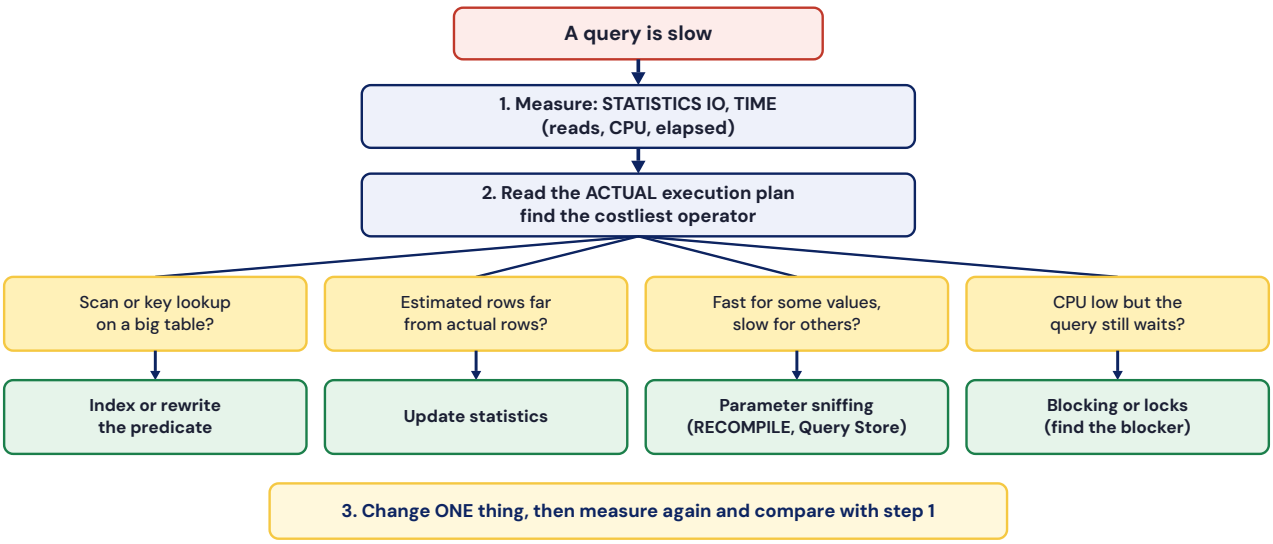
HOW THE INTERVIEWER ASKS IT

- *"A query is running slow. What will you do?"*
- *"How do you read an execution plan?"*
- *"How do you decide which index to create?"*

Say this first

"I never guess and I never add an index blindly. First I measure: logical reads, CPU and time. Then I read the actual execution plan to find the most expensive operator. Then I check the usual suspects in a fixed order: indexes, statistics, how the WHERE clause is written, parameter sniffing and blocking. I change one thing at a time and measure again."

Picture it



Follow the same path every time. It sounds calm and it is correct.

Step	What I do	What I look for	Typical fix
1. Measure	`SET STATISTICS IO, TIME ON`	Logical reads, CPU time, elapsed time	A baseline to compare against
2. Read the actual plan	Ctrl+M in SSMS, then run	Table scans, key lookups, sorts, spills, thick arrows	Aim at the costliest operator
3. Estimated vs actual rows	Hover over the operators	A big gap between the two numbers	Update statistics, fix the predicate
4. Indexes	Missing index hint, existing indexes	Scan where a seek is expected	Add or adjust an index
5. Predicate shape	Read the WHERE and JOIN	Functions on columns, implicit conversions	Rewrite so it is SARGable (Q30)
6. Parameter sniffing	Compare plans for different values	Fast for one value, slow for another	See Q36
7. Blocking and waits	`sys.dm_exec_requests`, wait type	Waiting on locks, not working	See Q32
8. Retest	Re-run and compare to step 1	Reads and time actually dropped	Keep the change, remove the rest

T-SQL

```
SET STATISTICS IO, TIME ON;

SELECT o.OrderID, o.Amount, c.Name
FROM Orders o
JOIN Customers c ON c.CustomerID = o.CustomerID
WHERE o.OrderDate >= '2026-02-01';

-- In the Messages tab, compare:  logical reads  and  CPU time
```

WAY 1 Add a covering index (when the plan shows a scan or key lookup)

T-SQL

```
CREATE NONCLUSTERED INDEX IX_Orders_OrderDate
ON Orders (OrderDate)
INCLUDE (CustomerID, Amount);
```

WHY USE IT

Turns a scan into a seek and removes the key lookups, so reads drop a lot for this query.

WHEN IT FAILS

Every index slows INSERT, UPDATE and DELETE and uses storage. If the query runs once a month, the index is not worth it. A "missing index" hint from the plan can duplicate an index you already have, so always check first.

WAY 2 Rewrite the query (when the predicate is the problem)

T-SQL

```
-- Slow: function on the column, cannot seek
WHERE YEAR(OrderDate) = 2026
-- Fast: range on the raw column
WHERE OrderDate >= '2026-01-01' AND OrderDate < '2027-01-01'
```

WHY USE IT

Costs nothing to run and needs no storage. Often gives the biggest win.

WHEN IT FAILS

Not possible when the query comes from a tool or ORM you cannot change. Then you need an index or a computed column (see Q30).

WAY 3 Fix the statistics (when estimated rows are far from actual rows)

T-SQL

```
UPDATE STATISTICS dbo.Orders WITH FULLSCAN;
```

WHY USE IT

Gives the optimizer correct row counts, so it chooses the right join type and memory grant.

WHEN IT FAILS

On a very large table a full scan of the statistics is heavy. Update statistics during a quiet window, or use a sampled update.

WHICH ONE SHOULD YOU PICK?

Measure first. If the plan shows a scan on a big table: index or predicate rewrite. If estimates are wrong: statistics. If the same procedure is sometimes fast and sometimes slow: parameter sniffing. If CPU is idle but the query is slow: blocking.

THE INTERVIEWER'S TRAP

"Just add the index suggested in the green text of the plan" is not a good answer. Say that the suggestion ignores your write load and your existing indexes, so you validate it before creating it.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **It was fast yesterday and slow today**
  See Q42 for the full scenario.
- **Estimated vs actual execution plan**
  Estimated is a prediction without running. Actual includes real row counts and warnings, which is what you want for troubleshooting.
- **How to find the top slow queries**
  Query Store (Q50) or `sys.dm_exec_query_stats`.
- **Table scan vs index seek**
  See Q29.

- **How do you read a plan**
  Operators flow from right to left. Look at cost percentage and the thickness of the arrows.
- **Tools**
  SSMS execution plans, Query Store, Extended Events. SQL Profiler is deprecated.

ONE LINE TO REMEMBER

Measure with STATISTICS IO, read the actual plan, check indexes, statistics, SARGability, sniffing, blocking. Change one thing, measure again.

Q29 Index seek vs scan, key lookup and covering index

MOST ASKED IMPORTANT

Chance of being asked 80%

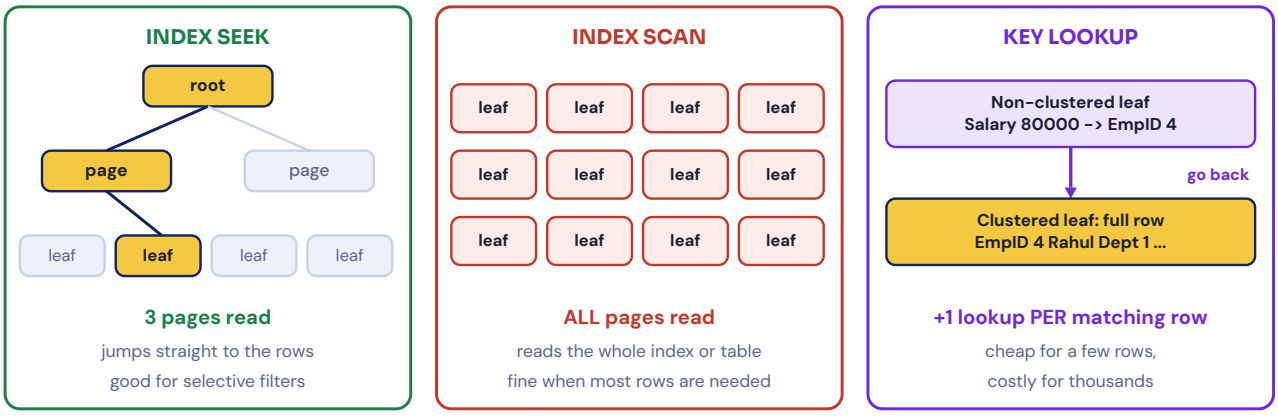
HOW THE INTERVIEWER ASKS IT

- *"What is the difference between an index seek and an index scan?"*
- *"What is a key lookup and how do you remove it?"*
- *"What is a covering index? What does INCLUDE do?"*

Say this first

"A seek jumps straight to the rows I need using the index tree. A scan reads the whole index or table. A key lookup happens when a non-clustered index finds the row but the query needs a column that is not in the index, so SQL Server goes back to the table for it. A covering index contains every column the query needs, so no lookup is needed."

Picture it



Seek, scan and key lookup are three different ways to read.

Operation	What happens	Good or bad
Index seek	Navigates the B-tree to the matching rows	**Good for selective filters**
Index scan	Reads all index pages	Fine for small tables or when most rows are needed
Table scan	Reads every page of a heap	Bad on a large table
Key lookup	One extra read in the clustered index per row found	**Costly when many rows match**

```t-sql
-- The query
SELECT OrderDate, Amount
FROM Orders
WHERE CustomerID = 1;
```

WAY 1 Leave it alone

WHY USE IT

If the filter matches only a handful of rows, a few key lookups cost almost nothing. The optimizer chooses a seek plus lookups on purpose. Do not fix what is not broken.

WHEN IT FAILS

Once the filter matches thousands of rows, each row needs a lookup and the plan gets slower. The optimizer may then switch to a full scan, which is a sign you are past the tipping point.

WAY 2 Make the index cover the query with INCLUDE

```t-sql
CREATE NONCLUSTERED INDEX IX_Orders_Customer
ON Orders (CustomerID)
INCLUDE (OrderDate, Amount);
```

WHY USE IT

CustomerID is the key used for seeking. OrderDate and Amount are stored only at the leaf level, so they add little overhead and remove the lookup entirely.

WHEN IT FAILS

The index gets wider, takes more storage and slows writes. Include only the columns your important queries need.

WAY 3 Add the column to the index key

```t-sql
CREATE NONCLUSTERED INDEX IX_Orders_Customer_Date
ON Orders (CustomerID, OrderDate);
```

WHY USE IT

Needed when you also filter or sort by the second column. The second column is then part of the sorted key, so SQL Server can seek on both and avoid a sort.

WHEN IT FAILS

Key columns are stored at every level of the tree and count toward the key size limit (900 bytes for clustered, 1700 for non-clustered). Use INCLUDE for columns that are only selected, not searched.

WAY 4 Reduce the SELECT list

WHY USE IT

Often the simplest fix. If the query only needs two columns, `SELECT *` forces lookups for all the others. Ask for only what you need.

WHEN IT FAILS

Not always possible when an application or report needs all the columns.

WHICH ONE SHOULD YOU PICK?

Few rows: ignore the lookup. Many rows and a hot query: covering index with INCLUDE. Also filter or sort by another column: add it to the key. Always: stop using SELECT *.

THE INTERVIEWER'S TRAP

"Key columns vs INCLUDE columns" is the usual follow-up. Key columns are sorted and used for seeking and ordering. INCLUDE columns are not sorted, only carried along at the leaf to avoid lookups.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Filtered index**
  An index with a WHERE clause, such as `WHERE Status = 'Open'`. Small and fast for a hot subset.
- **Column order in a composite index**
  Equality columns first, then range or sort columns. An index on (A, B) helps a filter on A, or on A and B, but not on B alone.
- **RID lookup**
  The same idea as a key lookup, but for a heap (a table without a clustered index).
- **Why does SELECT * hurt**
  It forces lookups and prevents covering indexes.
- **When does SQL Server choose a scan on purpose**
  When the filter returns a large fraction of the table.

ONE LINE TO REMEMBER

Seek = jump to rows. Scan = read everything. Key lookup = extra trip to the table. Cover the query with INCLUDE columns, but only for hot queries.

Q30 Why is my index not used? (SARGability)

LOGICAL IMPORTANT

Chance of being asked 70%

HOW THE INTERVIEWER ASKS IT

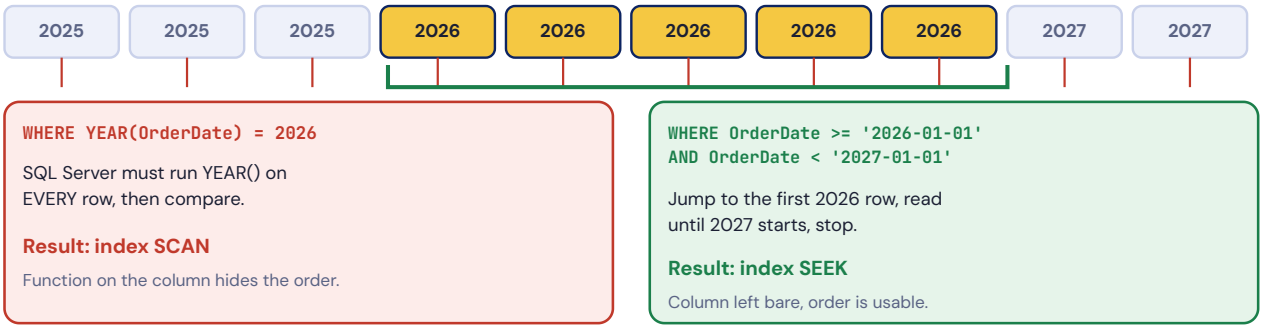
- *"The column is indexed but the query still scans. Why?"*
- *"Rewrite this query so it can use the index: `WHERE YEAR(OrderDate) = 2026`."*
- *"What is a SARGable predicate?"*

Say this first

"SARG stands for search argument. A predicate is SARGable when SQL Server can use an index seek for it. The rule is simple: keep the indexed column alone on one side of the comparison. The moment I wrap it in a function or do arithmetic on it, SQL Server has to calculate the function for every row, so it scans."

Picture it

OrderDate column, stored in sorted order inside the index



The same filter, written two ways.

Pattern	Slow (not SARGable)	Fast (SARGable)
Year filter	`WHERE YEAR(OrderDate) = 2026`	`WHERE OrderDate >= '2026-01-01' AND OrderDate < '2027-01-01'`
Starts with	`WHERE LEFT(Name, 1) = 'A'`	`WHERE Name LIKE 'A%'`
Arithmetic	`WHERE Salary + 1000 > 50000`	`WHERE Salary > 49000`
NULL replace	`WHERE ISNULL(Bonus, 0) = 0`	`WHERE Bonus = 0 OR Bonus IS NULL`
Type mismatch	`WHERE CAST(OrderID AS VARCHAR(10)) = '5'`	`WHERE OrderID = 5`
Ends with	`WHERE Name LIKE '%kumar'`	Full-text search, or a reversed computed column

WAY 1 Rewrite the predicate

```T-SQL
-- Before: scan
SELECT OrderID FROM Orders WHERE YEAR(OrderDate) = 2026;

-- After: seek
SELECT OrderID FROM Orders
WHERE OrderDate >= '2026-01-01' AND OrderDate < '2027-01-01';
```

WHY USE IT

Free, and the usual cause. A half-open date range ("from the start, up to but not including the next start") also avoids end-of-day time bugs.

WHEN IT FAILS

You can only do it when you control the query text. A reporting tool that always sends `YEAR(...)` cannot be changed.

WAY 2 Index a persisted computed column

```tsql
ALTER TABLE Orders ADD OrderYear AS YEAR(OrderDate) PERSISTED;
CREATE INDEX IX_Orders_OrderYear ON Orders (OrderYear);
-- Now WHERE OrderYear = 2026 can seek
```

WHY USE IT

Makes the expression itself indexable, so even tools that send the function can be served by modifying the table once.

WHEN IT FAILS

Extra storage and a slightly slower write. The query must use the same expression so the optimizer can match it.

WAY 3 Full-text search (for leading wildcards)

```tsql
-- LIKE '%kumar' can never seek. Use full-text instead:
SELECT * FROM Customers WHERE CONTAINS(Name, 'kumar');
```

WHY USE IT

Built for searching words inside text, and fast on large tables.

WHEN IT FAILS

Needs a full-text catalog, setup and maintenance. It matches words, not arbitrary character endings.

The silent killer: implicit conversion

If a column is VARCHAR and you compare it with an NVARCHAR value, SQL Server follows data type precedence and converts the **column** to NVARCHAR for every row. The seek turns into a scan, and there is no error. The plan shows a yellow warning. Fix it by matching the parameter type to the column type.

WHICH ONE SHOULD YOU PICK?

Rewrite first. If you cannot change the query, use an indexed computed column. For text search with a leading wildcard, use full-text.

THE INTERVIEWER'S TRAP

`OR` between two different columns often prevents a seek. Rewrite it as two queries joined with UNION ALL, each of which can use its own index. Also, the `LIKE 'A%'` seek works, but `LIKE '%A'` does not.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Why did the plan show a warning icon**
  Probably an implicit conversion or a missing index.
- **Date functions on a column**
  `DATEDIFF(DAY, OrderDate, GETDATE()) < 7` is not SARGable. Use `OrderDate >= DATEADD(DAY, -7, CAST(GETDATE() AS DATE))`.
- **Collation conflicts**
  Comparing columns with different collations can force conversions.

- **Low selectivity**
  If the filter matches most of the table, a scan is the right plan even for a SARGable predicate.

ONE LINE TO REMEMBER

Keep the indexed column bare on one side. No functions, no arithmetic, and match data types. Rewrite YEAR(col) = 2026 as a date range.

Q31 Isolation levels in SQL Server

IMPORTANT

Chance of being asked 75%

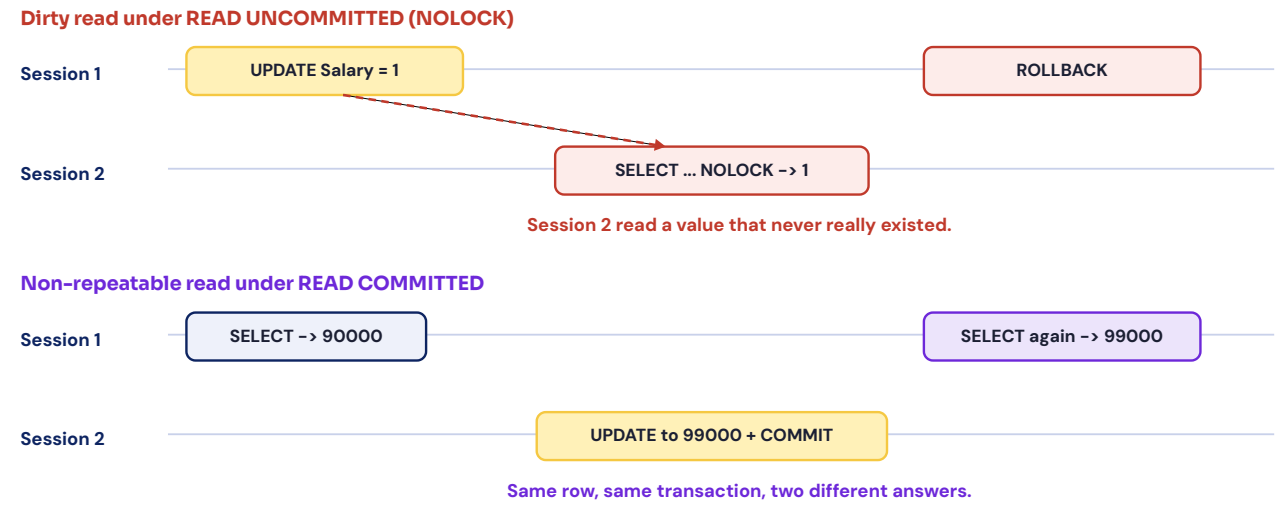
HOW THE INTERVIEWER ASKS IT

- *"What are the transaction isolation levels?"*
- *"What are dirty reads, non-repeatable reads and phantom reads?"*
- *"What is the difference between READ COMMITTED and READ COMMITTED SNAPSHOT?"*

Say this first

"Isolation levels decide how much one transaction can see of another transaction's unfinished work. Higher levels give more consistency but less concurrency. The default is READ COMMITTED. Snapshot-based levels use row versions so readers and writers do not block each other."

Picture it



Two of the three anomalies, drawn as a timeline.

Level	Dirty read	Non-repeatable read	Phantom	How it works
READ UNCOMMITTED	Possible	Possible	Possible	No shared locks (this is what NOLOCK does)
READ COMMITTED (default)	**No**	Possible	Possible	Shared locks are released right after reading
READ COMMITTED SNAPSHOT (RCSI)	No	Possible	Possible	Reads the last committed row version, readers do not block writers
REPEATABLE READ	No	No	Possible	Shared locks are held until the transaction ends
SNAPSHOT	No	No	No	One consistent view for the whole transaction, using row versions
SERIALIZABLE	No	No	No	Range locks, lowest concurrency

- **Dirty read:** reading data that another transaction has changed but not yet committed. It may be rolled back, so you read data that never existed.
- **Non-repeatable read:** reading the same row twice in one transaction and getting different values, because someone committed a change in between.
- **Phantom read:** running the same query twice and getting extra rows the second time, because someone inserted rows that match.

T-SQL

```
-- Session 1
BEGIN TRAN;
UPDATE Employees SET Salary = 1 WHERE EmpID = 1;    -- not committed yet

-- Session 2
SELECT Salary FROM Employees WITH (NOLOCK) WHERE EmpID = 1;    -- returns 1 (dirty read)

-- Session 1
ROLLBACK;    -- the salary of 1 never really existed
```

WAY 1 NOLOCK (READ UNCOMMITTED)

WHY USE IT

No locking at all, so reads never wait. People use it to make a slow report stop blocking.

WHEN IT FAILS

Dirty reads, and it can also skip rows or read the same row twice when pages split during the scan. Wrong numbers in a financial report are worse than a slow report. It hides the real problem.

WAY 2 READ COMMITTED SNAPSHOT (RCSI)

```T-SQL
ALTER DATABASE InterviewPrep SET READ_COMMITTED_SNAPSHOT ON;
```

WHY USE IT

Readers see the last committed version and do not block writers, and writers do not block readers. No dirty reads, and almost no application changes. This is the usual proper fix for read/write blocking.

WHEN IT FAILS

Uses the version store in tempdb, so it needs space and adds I/O. Each changed row carries extra bytes. Still allows non-repeatable reads.

WAY 3 SNAPSHOT isolation

```T-SQL
ALTER DATABASE InterviewPrep SET ALLOW_SNAPSHOT_ISOLATION ON;
SET TRANSACTION ISOLATION LEVEL SNAPSHOT;
```

WHY USE IT

The whole transaction sees one consistent point in time. Great for long reports that must be internally consistent.

WHEN IT FAILS

If two transactions change the same row, the later one fails with an update conflict (error 3960) and you must retry. More tempdb use than RCSI.

WHICH ONE SHOULD YOU PICK?

Default READ COMMITTED for most work. If readers and writers block each other, turn on RCSI instead of using NOLOCK. Use SNAPSHOT for consistent long reports. Use SERIALIZABLE only for strict rules, and keep the transaction short.

THE INTERVIEWER'S TRAP

Never say "I use NOLOCK to improve performance" in an interview without naming the risks. The right sentence is: "NOLOCK avoids blocking but allows dirty reads and inconsistent results, so I use RCSI instead."

MORE QUESTIONS HIDING INSIDE THIS ONE

- **How to see the current level**
  `DBCC USEROPTIONS .`
- **Lock hints**
  UPDLOCK (take an update lock early), HOLDLOCK (same as SERIALIZABLE), READPAST (skip locked rows, useful for queue tables).
- **Lock escalation**
  SQL Server may convert many row locks into one table lock (around 5000 locks) to save memory.
- **Shared, exclusive and update locks**
  Shared for reading, exclusive for writing, update as a bridge to avoid deadlocks.

- **Does RCSI change code**
  No. It changes how the default level behaves.

ONE LINE TO REMEMBER

Default is READ COMMITTED. NOLOCK = dirty reads. Use RCSI so readers do not block writers. SNAPSHOT gives a consistent view for the whole transaction.

Q32 Deadlock vs blocking

IMPORTANT

Chance of being asked 75%

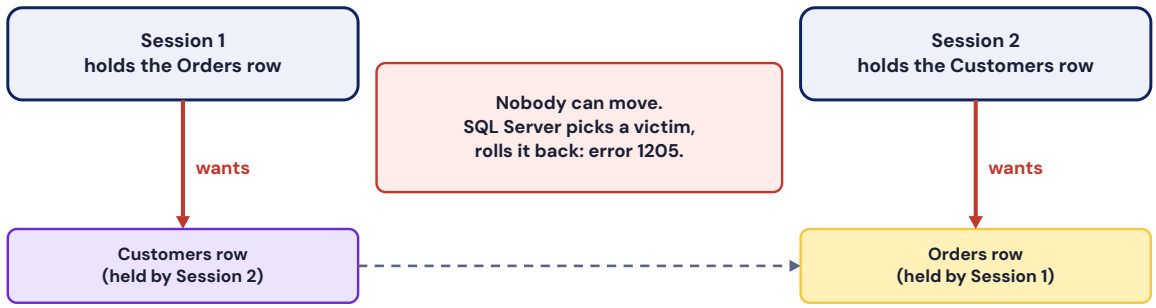
HOW THE INTERVIEWER ASKS IT

- *"What is a deadlock? How is it different from blocking?"*
- *"How do you find and prevent deadlocks?"*
- *"What error number does a deadlock victim get?"*

Say this first

"Blocking is one session waiting for a lock held by another. It ends when the first session finishes. A deadlock is a circle: session A waits for B and B waits for A, so neither can ever finish. SQL Server detects it, kills one session (the victim) with error 1205, and lets the other continue."

Picture it



Blocking = a queue that ends. Deadlock = a circle that never ends on its own.

Each session holds one thing and wants the other.

Feature	Blocking	Deadlock
What it is	One session waits for another	Two or more sessions wait for each other in a circle
Does it end by itself	Yes, when the blocker commits or rolls back	**No. SQL Server must kill one session**
Error	None, just waiting	Error 1205 for the victim
Typical cause	Long transactions, missing indexes	Different access order, long transactions

```T-SQL
-- Session 1                          -- Session 2
BEGIN TRAN;                           BEGIN TRAN;
UPDATE Orders SET Amount = 1          UPDATE Customers SET City = 'X'
 WHERE OrderID = 1;                    WHERE CustomerID = 1;
UPDATE Customers SET City = 'Y'       UPDATE Orders SET Amount = 2
 WHERE CustomerID = 1;  -- waits       WHERE OrderID = 1;     -- waits: DEADLOCK
```

WAY 1 Access objects in the same order everywhere

WHY USE IT

The most reliable prevention. If every transaction touches Orders first and Customers second, a circle cannot form.

WHEN IT FAILS

Needs discipline across all code and teams. One forgotten procedure can still deadlock.

WAY 2 Keep transactions short and add proper indexes

WHY USE IT

Locks are held for a shorter time, and good indexes lock only the rows you need instead of scanning and locking many.

WHEN IT FAILS

Does not remove the possibility. It only lowers how often it happens.

WAY 3 Use RCSI or SNAPSHOT

WHY USE IT

Readers stop taking shared locks, which removes a whole class of reader-versus-writer deadlocks (see Q31).

WHEN IT FAILS

Does not help writer-versus-writer deadlocks. Needs tempdb space.

WAY 4 Retry in the code when you get error 1205

T-SQL

```
DECLARE @try INT = 0;
WHILE @try < 3
BEGIN
    BEGIN TRY
        BEGIN TRAN;
            -- do the work here
        COMMIT;
        BREAK;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK;
        IF ERROR_NUMBER() = 1205
        BEGIN
            SET @try += 1;
            WAITFOR DELAY '00:00:01';
        END
        ELSE THROW;
    END CATCH
END;
```

WHY USE IT

Deadlocks are rare but normal in busy systems. A retry makes the user never notice.

WHEN IT FAILS

It hides a design problem if the deadlock is frequent. The work inside must be safe to repeat.

WHICH ONE SHOULD YOU PICK?

Prevent with consistent order, short transactions and indexes. Reduce with RCSI. Protect with retry logic. Investigate with the system_health Extended Events session.

THE INTERVIEWER'S TRAP

How to find them: the `system_health` Extended Events session records every deadlock graph by default, so you can open it and see both queries and the locks. Older answers mention trace flag 1222 or Profiler, which also work.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Choose which session loses**
  `SET DEADLOCK_PRIORITY LOW` on the session you are happy to sacrifice.
- **Find who is blocking right now**
  `sys.dm_exec_requests` (look at `blocking_session_id`) or `sp_who2`.
- **Kill a blocker**
  `KILL <session id>`, only after understanding the impact.
- **Lock types**
  Shared, exclusive, update, intent. Know one line for each.
- **Live-lock and latch**
  A latch is a short internal lock on a page in memory. Different from a transaction lock.

ONE LINE TO REMEMBER

Blocking waits and ends. A deadlock is a circle, SQL Server kills a victim with error 1205. Prevent with the same access order, short transactions, indexes, RCSI and retry.

Q33 Consecutive days (gaps and islands)

LOGICAL VARIANT FAMILY

Chance of being asked 60%

HOW THE INTERVIEWER ASKS IT

- *"Find users who logged in on 3 or more consecutive days."*
- *"Find the longest streak for each user."*
- *"Find missing IDs in a sequence."*

Say this first

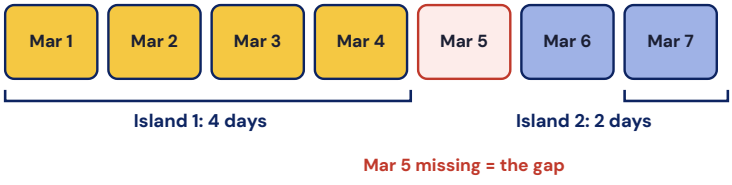
"This is the gaps and islands pattern. An island is an unbroken run of consecutive values. The trick is that if I subtract a row number from a date, every date in the same streak gives the same result, because both increase by one each step. Rows with the same difference belong to the same island, so I group by it."

Picture it

User 1 logins

LoginDate	rn	LoginDate - rn
03-01	1	02-28
03-02	2	02-28
03-03	3	02-28
03-04	4	02-28
03-06	5	03-01
03-07	6	03-01

On the calendar



GROUP BY UserID, grp gives one row per island:
start date, end date, number of days

Date minus row number gives the same value for every day in a streak.

WAY 1 Date minus ROW_NUMBER

T-SQL

```
WITH d AS (
    SELECT DISTINCT UserID, LoginDate FROM Logins),
g AS (
    SELECT UserID, LoginDate,
           DATEADD(DAY,
                   -ROW_NUMBER() OVER (PARTITION BY UserID ORDER BY LoginDate),
                   LoginDate) AS grp
    FROM d)
SELECT UserID,
       MIN(LoginDate) AS StartDate,
       MAX(LoginDate) AS EndDate,
       COUNT(*)       AS Days
FROM g
GROUP BY UserID, grp
HAVING COUNT(*) >= 3
ORDER BY UserID;
```

RESULT

UserID	StartDate	EndDate	Days
1	2026-03-01	2026-03-04	4
2	2026-03-03	2026-03-05	3

WHY USE IT

One pass, set-based and fast. It also tells you the start, end and length of every streak, so "longest streak" is just a MAX away.

WHEN IT FAILS

Dates must be unique per user, so the DISTINCT in the first CTE is essential. If the column is a datetime, cast it to DATE first. It only works when the step is exactly one day.

EXECUTION ORDER Way 1: date minus ROW_NUMBER

```
WITH d AS (SELECT DISTINCT UserID, LoginDate FROM Logins),
g AS (SELECT UserID, LoginDate,
DATEADD(DAY, -ROW_NUMBER() OVER (PARTITION BY UserID ORDER BY LoginDate), LoginDate) AS grp
FROM d)
SELECT UserID, MIN(LoginDate) AS StartDate, MAX(LoginDate) AS EndDate, COUNT(*) AS Days
FROM g GROUP BY UserID, grp HAVING COUNT(*) >= 3 ORDER BY UserID;
```

1 CTE d

SELECT DISTINCT removes repeated (user, date) pairs. Our 12 login rows have no repeats, so 12 rows remain.

2 CTE g

For each user, ROW_NUMBER numbers the dates 1, 2, 3 … Then the row number in days is subtracted from the date. Dates in an unbroken streak give the same grp value.

UserID	LoginDate	rn	grp
1	03-01	1	02-28
1	03-02	2	02-28
1	03-03	3	02-28
1	03-04	4	02-28
1	03-06	5	03-01
1	03-07	6	03-01

3 GROUP BY

Rows are grouped by (UserID, grp). 6 groups: user 1 has 2, user 2 has 2, user 3 has 2. The groups are user 1: 4 days and 2 days, user 2: 1 day and 3 days, user 3: 1 day and 1 day.

4	HAVING	`COUNT(*) >= 3` keeps the groups of 4 days (user 1) and 3 days (user 2).
5	SELECT	Returns the first date, last date and day count of each surviving group.
6	ORDER BY	Sorts by UserID.

WAY 2 LAG flag and running sum

T-SQL

```
WITH x AS (
    SELECT UserID, LoginDate,
           CASE WHEN DATEDIFF(DAY,
                      LAG(LoginDate) OVER (PARTITION BY UserID ORDER BY LoginDate),
                      LoginDate) = 1
                THEN 0 ELSE 1 END AS NewIsland
    FROM Logins),
y AS (
    SELECT *, SUM(NewIsland) OVER (PARTITION BY UserID ORDER BY LoginDate
                                   ROWS UNBOUNDED PRECEDING) AS grp
    FROM x)
SELECT UserID, MIN(LoginDate) AS StartDate, MAX(LoginDate) AS EndDate, COUNT(*) AS Days
FROM y
GROUP BY UserID, grp
HAVING COUNT(*) >= 3;
```

WHY USE IT

You decide what "continues" means, so it also handles rules like "a gap of at most 2 days is still the same streak". Very flexible.

WHEN IT FAILS

More code and two window passes. Slightly harder to explain in an interview than the first way.

WAY 3 Self join for exactly 3 days

T-SQL

```
SELECT DISTINCT l1.UserID
FROM Logins l1
JOIN Logins l2 ON l2.UserID = l1.UserID
              AND l2.LoginDate = DATEADD(DAY, 1, l1.LoginDate)
JOIN Logins l3 ON l3.UserID = l1.UserID
              AND l3.LoginDate = DATEADD(DAY, 2, l1.LoginDate);
```

WHY USE IT

Easy to read for a fixed streak length of 3. Works without window functions.

WHEN IT FAILS

Every extra day needs another join, so a 7-day streak needs 7 joins. It cannot report the streak length or its dates.

WHICH ONE SHOULD YOU PICK?

Default: date minus ROW_NUMBER. If the continuity rule is custom: LAG with a running sum. If asked for exactly 3 and no windows: self join.

THE INTERVIEWER'S TRAP

Forgetting DISTINCT: if a user logs in twice on the same day, ROW_NUMBER moves one step while the date does not, and the streak breaks. Always de-duplicate the dates first.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Longest streak per user**
  `SELECT UserID, MAX(Days)` over the island query. Our data: user 1 has 4.
- **Missing numbers in a sequence (gaps)**
  `LEAD(ID) OVER (ORDER BY ID) - ID > 1` shows where the gap starts.
- **Consecutive available seats**
  Same idea with seat numbers instead of dates.
- **Three consecutive wins in a row**
  Same trick, or a self join with three aliases.
- **Date ranges where a status stayed the same**
  `ROW_NUMBER() OVER (ORDER BY d) - ROW_NUMBER() OVER (PARTITION BY status ORDER BY d)` as the group key.

ONE LINE TO REMEMBER

Consecutive values minus ROW_NUMBER give a constant. Group by that constant to get each island. De-duplicate dates first.

Q34 Recursive CTE: employee hierarchy

VARIANT FAMILY

Chance of being asked 60%

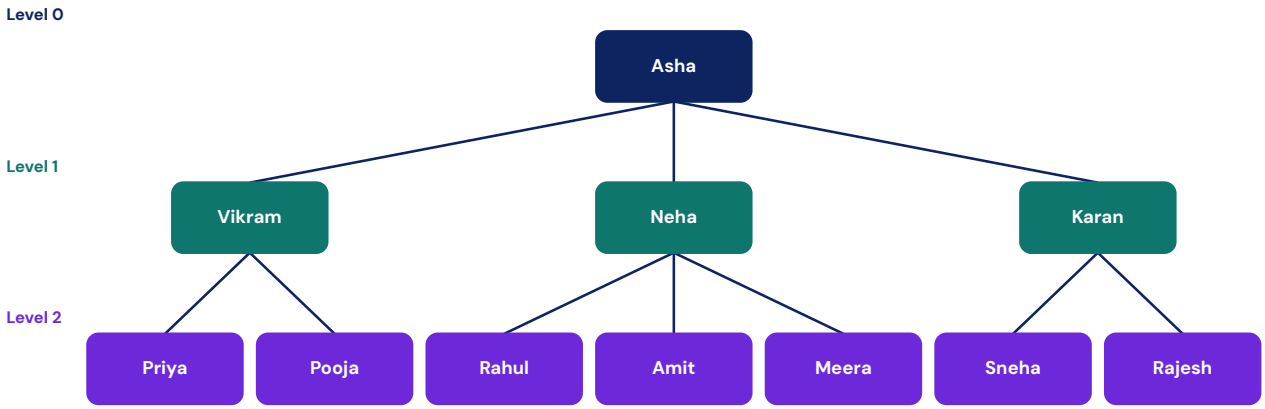
HOW THE INTERVIEWER ASKS IT

- *"Show the full reporting hierarchy of the company."*
- *"Find everyone who reports to a given manager, directly or indirectly."*
- *"Generate a list of dates or numbers without a table."*

Say this first

"A recursive CTE has two parts joined by UNION ALL. The anchor query returns the starting rows, for example the top person. The recursive part joins the CTE to the table again to find the next level, and it keeps repeating until no new rows come back."

Picture it



Level 0 = anchor query. Level 1 and 2 = each pass of the recursive part.

The recursive part walks down one level at a time.

WAY 1 Recursive CTE

T-SQL

```
WITH Org AS (
    -- anchor: the top of the tree
    SELECT EmpID, Name, ManagerID, 0 AS Lvl,
           CAST(Name AS VARCHAR(500)) AS Path
    FROM Employees
    WHERE ManagerID IS NULL
    UNION ALL
    -- recursive part: the next level of reports
    SELECT e.EmpID, e.Name, e.ManagerID, o.Lvl + 1,
           CAST(o.Path + ' > ' + e.Name AS VARCHAR(500))
    FROM Employees e
    JOIN Org o ON e.ManagerID = o.EmpID)
SELECT Lvl, Path FROM Org
ORDER BY Path
OPTION (MAXRECURSION 100);
```

RESULT

```
Lvl  Path
---  --------------------
0    Asha
1    Asha > Karan
2    Asha > Karan > Rajesh
2    Asha > Karan > Sneha
1    Asha > Neha
2    Asha > Neha > Amit
2    Asha > Neha > Meera
2    Asha > Neha > Rahul
1    Asha > Vikram
2    Asha > Vikram > Pooja
2    Asha > Vikram > Priya
```

WHY USE IT

Handles any depth with one query, and returns the level and path as a bonus. This is the answer interviewers expect.

WHEN IT FAILS

By default SQL Server stops at 100 levels (error 530), which is a protection against infinite loops caused by cycles. The anchor and recursive columns must have identical data types, so the CAST on both sides is not optional.

EXECUTION ORDER Way 1: recursive CTE

```
WITH Org AS (
SELECT EmpID, Name, ManagerID, 0 AS Lvl, CAST(Name AS VARCHAR(500)) AS Path
FROM Employees WHERE ManagerID IS NULL
UNION ALL
SELECT e.EmpID, e.Name, e.ManagerID, o.Lvl + 1, CAST(o.Path + ' > ' + e.Name AS VARCHAR(500))
FROM Employees e JOIN Org o ON e.ManagerID = o.EmpID)
SELECT Lvl, Path FROM Org ORDER BY Path;
```

1	Anchor	Runs once. `ManagerID IS NULL` finds only Asha. Org = {Asha}, level 0.
2	Recursive pass 1	Joins Employees to the rows produced by the LAST step only (Asha). Everyone whose ManagerID is Asha's EmpID is added: Vikram, Neha, Karan. Level 1.
3	Recursive pass 2	Joins Employees to those 3 rows. Added: Priya and Pooja (under Vikram), Rahul, Amit and Meera (under Neha), Sneha and Rajesh (under Karan). 7 rows, level 2.

4	`Recursive pass 3`	Joins Employees to those 7 rows. Nobody reports to them, so 0 rows come back and the recursion stops.
5	`Outer SELECT`	Org now holds 1 + 3 + 7 = 11 rows. They are listed with their Lvl and Path, sorted by Path.

WAY 2 Fixed self joins

```tsql
SELECT e1.Name AS Boss, e2.Name AS Level1, e3.Name AS Level2
FROM Employees e1
LEFT JOIN Employees e2 ON e2.ManagerID = e1.EmpID
LEFT JOIN Employees e3 ON e3.ManagerID = e2.EmpID
WHERE e1.ManagerID IS NULL;
```

WHY USE IT

No recursion needed, simple to read when you know the depth is small and fixed.

WHEN IT FAILS

Works only for the number of levels you wrote. A new level in the data means a new join. Not suitable for real organisations.

WAY 3 WHILE loop with a temp table

```tsql
SELECT EmpID, Name, 0 AS Lvl INTO #org FROM Employees WHERE ManagerID IS NULL;
WHILE @@ROWCOUNT > 0
    INSERT INTO #org
    SELECT e.EmpID, e.Name, o.Lvl + 1
    FROM Employees e
    JOIN #org o ON e.ManagerID = o.EmpID
    WHERE e.EmpID NOT IN (SELECT EmpID FROM #org);
```

WHY USE IT

Procedural and easy to step through in a debugger. Works in places where recursive CTEs are not allowed.

WHEN IT FAILS

Slower, more code, and the NOT IN check grows with every loop. Use only when a CTE is truly impossible.

WHICH ONE SHOULD YOU PICK?

Recursive CTE for hierarchies of unknown depth. Self joins only when the depth is fixed and tiny. Also mention `hierarchyid` as a built-in data type for very large trees.

THE INTERVIEWER'S TRAP

If the data has a cycle (A reports to B, B reports to A) the recursion never ends and SQL Server raises error 530 after 100 levels. Mention that you can track the Path and add `WHERE o.Path NOT LIKE '%' + e.Name + '%'` as a cycle guard.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **All reports under one manager**
  Anchor with `WHERE EmpID = 3` . Our data: Rahul, Amit and Meera report to Neha.

- **Level of every employee**
  The `Lvl` column above.
- **Generate a calendar of dates**
  Anchor with a start date, add one day in the recursive part, stop with a WHERE. For more than 100 rows use `OPTION (MAXRECURSION 0)`.
- **Generate numbers 1 to 1000**
  `SELECT 1 AS n UNION ALL SELECT n + 1 FROM x WHERE n < 1000`, with MAXRECURSION 1000.
- **Split a comma separated string**
  Today use `STRING_SPLIT`. A recursive CTE is the old way.

ONE LINE TO REMEMBER

Recursive CTE = anchor UNION ALL recursive part. Match the data types, watch for cycles, and remember the default limit of 100 levels.

Q35 MERGE: insert, update and delete in one statement

IMPORTANT

Chance of being asked 55%

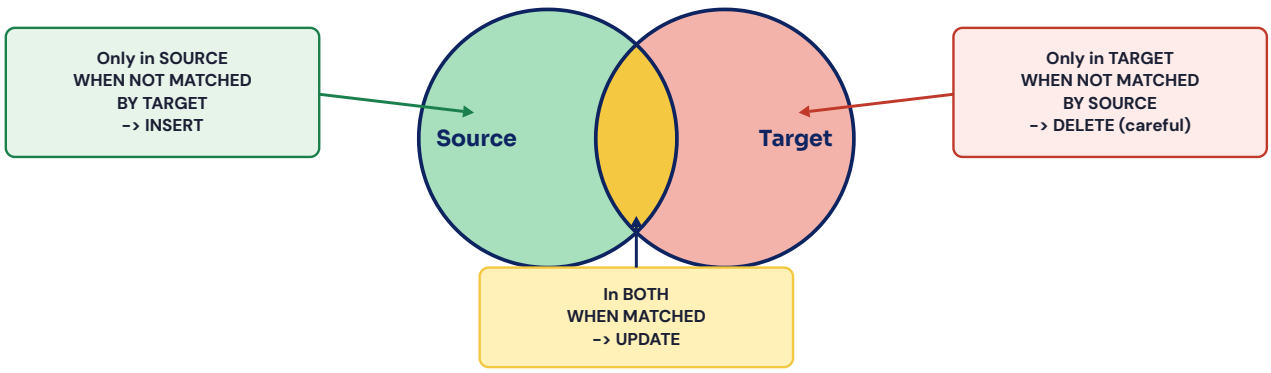
HOW THE INTERVIEWER ASKS IT

- *"How do you do an upsert (update if it exists, insert if not)?"*
- *"What is MERGE? What are its risks?"*
- *"How would you load a dimension table incrementally?"*

Say this first

"MERGE compares a source and a target on a key, then updates the matching rows, inserts the new ones, and can delete the extra ones, all in one statement. It is popular in ETL. But it has a history of bugs and concurrency traps, so many teams prefer separate UPDATE and INSERT statements inside a transaction."

Picture it



Three groups of rows, three different actions.

WAY 1 MERGE

T-SQL

```
MERGE dbo.DimCustomer WITH (HOLDLOCK) AS t
USING stg.Customer AS s
   ON t.CustomerID = s.CustomerID
WHEN MATCHED AND t.City <> s.City THEN
     UPDATE SET t.City = s.City
WHEN NOT MATCHED BY TARGET THEN
     INSERT (CustomerID, Name, City) VALUES (s.CustomerID, s.Name, s.City)
WHEN NOT MATCHED BY SOURCE THEN
     DELETE
OUTPUT $action, inserted.CustomerID;
```

WHY USE IT

One statement, one pass, and `OUTPUT $action` tells you exactly what happened to each row. Very compact for loading dimension tables.

WHEN IT FAILS

Without `HOLDLOCK` there are race conditions that can cause duplicate key errors under concurrency. Several historical bugs exist. And `WHEN NOT MATCHED BY SOURCE THEN DELETE` deletes every target row missing from the source, which wipes the table if the source arrives empty.

WAY 2 Separate UPDATE and INSERT in a transaction

T-SQL

```
BEGIN TRAN;
UPDATE t SET t.City = s.City
FROM dbo.DimCustomer t
JOIN stg.Customer s ON s.CustomerID = t.CustomerID
WHERE t.City <> s.City;

INSERT INTO dbo.DimCustomer (CustomerID, Name, City)
SELECT s.CustomerID, s.Name, s.City
FROM stg.Customer s
WHERE NOT EXISTS (SELECT 1 FROM dbo.DimCustomer t WHERE t.CustomerID = s.CustomerID);
COMMIT;
```

WHY USE IT

Each step is simple, predictable and easy to debug. Production teams often choose this for safety.

WHEN IT FAILS

Two passes over the data, so it is slower on very large loads. You must remember the transaction and the right filters.

WAY 3 Single row upsert pattern

```tsql
BEGIN TRAN;
UPDATE dbo.DimCustomer WITH (UPDLOCK, SERIALIZABLE)
SET City = @City WHERE CustomerID = @ID;

IF @@ROWCOUNT = 0
    INSERT INTO dbo.DimCustomer (CustomerID, Name, City) VALUES (@ID, @Name, @City);
COMMIT;
```

WHY USE IT

The classic safe upsert for one row at a time, such as an API call.

WHEN IT FAILS

Without the `UPDLOCK, SERIALIZABLE` hints, two sessions can both see zero rows and both insert, causing a duplicate key. It is for single rows only.

WHICH ONE SHOULD YOU PICK?

Bulk ETL where you control concurrency: MERGE with HOLDLOCK and a filtered DELETE. Safety first or a team that distrusts MERGE: UPDATE then INSERT in a transaction. Single record: the UPDATE with hints, then INSERT if nothing updated.

THE INTERVIEWER'S TRAP

Be honest about the risks. Saying "I use MERGE everywhere" worries a senior interviewer. Saying "MERGE is useful, but I protect it with HOLDLOCK and I never leave an unfiltered NOT MATCHED BY SOURCE" earns trust.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **SCD Type 1 vs Type 2**
  Type 1 overwrites the old value. Type 2 keeps history by expiring the old row and inserting a new one.
- **The OUTPUT clause**
  Returns the changed rows from INSERT, UPDATE, DELETE and MERGE, great for auditing.
- **Difference between MERGE and an INSERT with a NOT EXISTS check**
  INSERT only adds. MERGE can also update and delete.
- **Can MERGE update the same target row twice**
  No. It raises an error if a target row matches several source rows.

ONE LINE TO REMEMBER

MERGE = upsert in one statement, but protect it with HOLDLOCK and never leave NOT MATCHED BY SOURCE unfiltered. Many teams prefer UPDATE plus INSERT.

Q36 Parameter sniffing

IMPORTANT

Chance of being asked 55%

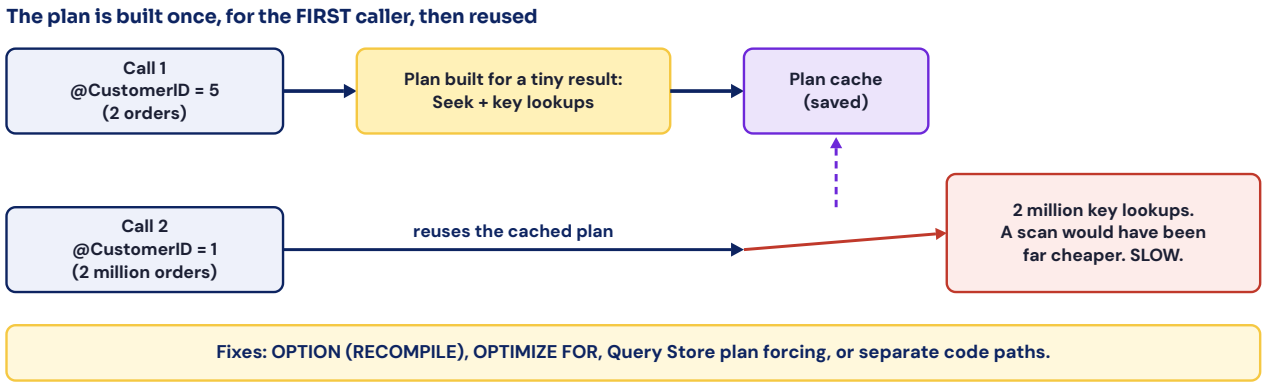
HOW THE INTERVIEWER ASKS IT

- *"What is parameter sniffing? How do you fix it?"*
- *"A procedure is fast in SSMS but slow from the application. Why?"*
- *"The same procedure is fast for one customer and very slow for another. Why?"*

Say this first

"When a stored procedure runs for the first time, SQL Server looks at the parameter values, builds the best plan for those values, and caches it. Every later call reuses that plan. That is sniffing, and it is a feature. It becomes a problem when the first value was unusual, such as a customer with 2 orders, and the plan is then terrible for a customer with 2 million."

Picture it



The first caller decides the plan for everyone.

T-SQL

```
CREATE PROCEDURE dbo.usp_OrdersByCustomer @CustomerID INT
AS
    SELECT OrderID, Amount, OrderDate
    FROM Orders
    WHERE CustomerID = @CustomerID;
GO

EXEC dbo.usp_OrdersByCustomer @CustomerID = 5;    -- tiny customer: plan = seek + lookups
EXEC dbo.usp_OrdersByCustomer @CustomerID = 1;    -- huge customer reuses the same plan
```

WAY 1 OPTION (RECOMPILE)

T-SQL

```
SELECT OrderID, Amount, OrderDate
FROM Orders
WHERE CustomerID = @CustomerID
OPTION (RECOMPILE);
```

WHY USE IT

A fresh plan for the actual value every time, so the plan is always right for that call. The simplest fix.

WHEN IT FAILS

Compiling costs CPU on every run, and there is no plan reuse. Bad for a query that runs hundreds of times per second.

WAY 2 OPTIMIZE FOR

T-SQL

```
OPTION (OPTIMIZE FOR (@CustomerID = 1));     -- build the plan for a big customer
-- or
OPTION (OPTIMIZE FOR UNKNOWN);               -- use average statistics
```

WHY USE IT

A stable plan that does not depend on who called first. FOR UNKNOWN avoids the "unlucky first caller" problem.

WHEN IT FAILS

The plan is tuned for the value you chose, so it can be poor for others. FOR UNKNOWN uses average density, which is mediocre for everybody.

WAY 3 Query Store plan forcing

T-SQL

```
EXEC sp_query_store_force_plan @query_id = 42, @plan_id = 7;
```

WHY USE IT

No code change. You pick the good plan from history and pin it, which is ideal for a vendor application you cannot edit.

WHEN IT FAILS

A forced plan can become wrong when the data changes. You must monitor it for forcing failures.

WAY 4 Separate the paths (or use dynamic SQL)

T-SQL

```
IF @CustomerID IN (SELECT TOP (50) CustomerID FROM BigCustomers)
    EXEC dbo.usp_OrdersForBigCustomer @CustomerID;
ELSE
    EXEC dbo.usp_OrdersForSmallCustomer @CustomerID;
```

WHY USE IT

Each branch has its own plan, tuned for its own data shape. Predictable and explicit.

WHEN IT FAILS

More code to maintain, and you must know the data shape. Overkill for a small problem.

WHICH ONE SHOULD YOU PICK?

Start with OPTION (RECOMPILE) if the query is not very frequent. If it is hot, use OPTIMIZE FOR or Query Store forcing. On SQL Server 2022 (compatibility level 160), Parameter Sensitive Plan optimization can create more than one plan automatically for some patterns.

THE INTERVIEWER'S TRAP

"Why is it fast in SSMS and slow in the app?" Two different cached plans, because the session SET options differ (the classic one is ARITHABORT). The query is not different, the plan is. Compare the two plans and test with the app's settings.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **How do you prove it is sniffing**
  Compare the plan's compiled parameter value with the runtime value, and test with OPTION (RECOMPILE).
- **Clear one plan from cache**
  `DBCC FREEPROCCACHE (plan_handle)` for a single plan. Never clear the whole cache on production.
- **Simple vs forced parameterization**
  How SQL Server turns literals into parameters for ad-hoc queries.
- **Local variable trick**
  Copying the parameter into a local variable has the same effect as OPTIMIZE FOR UNKNOWN.

ONE LINE TO REMEMBER

Sniffing = plan built for the first caller's values and reused. Fix with RECOMPILE, OPTIMIZE FOR, Query Store forcing, or split paths. It is a feature, not a bug.

Q37 Index fragmentation: REBUILD vs REORGANIZE

IMPORTANT

Chance of being asked 50%

HOW THE INTERVIEWER ASKS IT

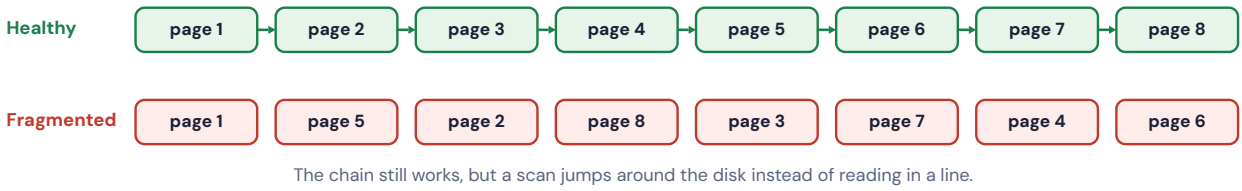
- *"What is index fragmentation and how do you fix it?"*
- *"REBUILD or REORGANIZE: how do you choose?"*
- *"What is FILLFACTOR?"*

Say this first

"Fragmentation means the logical order of an index no longer matches the physical order, usually because of page splits, so scans do more work. I check it with a dynamic management function. As a rule of thumb, I reorganize between 5 and 30 percent and rebuild above 30 percent. But I only care for indexes with enough pages to matter."

Picture it

Logical order vs physical order of index pages



The chain still works, but a scan jumps around the disk instead of reading in a line.

How it happens: a page split



Both pages are only half full, and the new page may land far away.

REORGANIZE (5 to 30 percent): tidy in place, online.

REBUILD (above 30 percent): fresh index + new statistics.

Fragmentation is about order, not about empty space only.

Feature	REORGANIZE	REBUILD
When	About 5 to 30 percent fragmentation	Above about 30 percent
Online	Always	Offline by default. Online with `ONLINE = ON` in editions that support it
Statistics	Not updated	**Updated with a full scan**
Interruptible	Yes, progress is kept	No, a cancelled rebuild rolls back
Log use	Light	Heavier

T-SQL

```
SELECT OBJECT_NAME(ips.object_id) AS TableName, i.name AS IndexName,
       ips.avg_fragmentation_in_percent AS FragPct, ips.page_count
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') AS ips
JOIN sys.indexes i ON i.object_id = ips.object_id AND i.index_id = ips.index_id
WHERE ips.avg_fragmentation_in_percent > 5
  AND ips.page_count > 1000;

ALTER INDEX IX_Orders_Customer ON Orders REORGANIZE;
ALTER INDEX IX_Orders_Customer ON Orders REBUILD WITH (FILLFACTOR = 90, ONLINE = ON);
```

WAY 1 REORGANIZE

WHY USE IT

Always online, light on resources, and safe to stop. Good for moderate fragmentation on a busy system.

WHEN IT FAILS

Does not update statistics and cannot change FILLFACTOR. For heavy fragmentation it does less than a rebuild.

WAY 2 REBUILD

WHY USE IT

Creates a fresh compact index, updates its statistics with a full scan, and lets you change FILLFACTOR. The strongest fix.

WHEN IT FAILS

Offline by default (blocking), uses more log and tempdb space, and cannot be cancelled without rolling back. ONLINE needs a supporting edition.

WAY 3 Do not defrag everything every night

WHY USE IT

Small indexes (under about 1000 pages) rarely benefit, and SSD storage makes fragmentation less painful. Updating statistics often matters more than rebuilding.

WHEN IT FAILS

Ignoring a heavily fragmented big index that serves range scans will cost you. Measure before deciding.

THE INTERVIEWER'S TRAP

Mention FILLFACTOR and page splits: a lower fill factor leaves free space on each page so inserts do not split pages, at the price of a bigger index. Also say that fragmentation hurts mainly scans, not single-row seeks.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Internal vs external fragmentation**
  Internal is wasted space inside pages. External is out-of-order pages.
- **Fragmented heap**
  `ALTER TABLE ... REBUILD`.
- **Maintenance tooling**
  Many teams use the community Ola Hallengren maintenance scripts instead of writing their own.
- **Update statistics after REORGANIZE**
  Yes, do it separately.
- **What causes page splits**
  Random inserts into a full page, such as GUID keys (see Q44).

ONE LINE TO REMEMBER

Check with sys.dm_db_index_physical_stats. REORGANIZE at 5 to 30 percent, REBUILD above 30 percent, ignore tiny indexes. Rebuild also updates statistics.

Q38 Dynamic SQL and SQL injection

IMPORTANT

Chance of being asked 50%

HOW THE INTERVIEWER ASKS IT

- *"What is dynamic SQL? When do you use it?"*
- *"What is the difference between EXEC and sp_executesql?"*
- *"How do you prevent SQL injection in T-SQL?"*

Say this first

"Dynamic SQL builds a query as a string and runs it. I use it when something that cannot be a parameter changes, such as a table name, a column list or an ORDER BY column. The dangerous way is concatenating user values into the string. The safe way is sp_executesql with real parameters, and QUOTENAME for object names."

WAY 1 EXEC with string concatenation (dangerous)

T-SQL

```
DECLARE @Name VARCHAR(50) = 'x'' OR 1=1 --';
DECLARE @sql VARCHAR(MAX) =
  'SELECT * FROM Employees WHERE Name = ''' + @Name + '''';
EXEC (@sql);      -- the final text: ... WHERE Name = 'x' OR 1=1 --'
```

WHY USE IT

Easy to write, which is the only reason people use it.

WHEN IT FAILS

The input closes the string and adds `OR 1=1`, so the query returns every row. An attacker can add `; DROP TABLE ...`. Each different value also creates a different plan, which wastes memory.

WAY 2 sp_executesql with parameters (safe)

T-SQL

```
DECLARE @Name VARCHAR(50) = 'Neha';
EXEC sp_executesql
     N'SELECT * FROM Employees WHERE Name = @Name',
     N'@Name VARCHAR(50)',
     @Name = @Name;
```

WHY USE IT

The value is passed as data and never parsed as code, so injection is impossible. The plan is reused for different values.

WHEN IT FAILS

Parameters cannot replace identifiers, so you still cannot pass a table or column name this way.

WAY 3 QUOTENAME for object names

```tsql
DECLARE @Col SYSNAME = 'Salary';
DECLARE @sql NVARCHAR(MAX) =
  N'SELECT Name, ' + QUOTENAME(@Col) + N' FROM Employees ORDER BY ' + QUOTENAME(@Col);
EXEC sp_executesql @sql;
```

WHY USE IT

QUOTENAME wraps the name in square brackets and escapes any bracket inside it, so a malicious name stays a harmless identifier.

WHEN IT FAILS

It makes the name safe but does not check that the name is allowed. Validate against a whitelist (`sys.columns`) so users cannot read columns they should not see.

WAY 4 Avoid dynamic SQL: the optional-filter pattern

```tsql
SELECT * FROM Employees
WHERE (@DeptID IS NULL OR DeptID = @DeptID)
  AND (@MinSalary IS NULL OR Salary >= @MinSalary)
OPTION (RECOMPILE);
```

WHY USE IT

No strings, so no injection and easy to read. The RECOMPILE removes the filters that are NULL from the plan.

WHEN IT FAILS

Without RECOMPILE the cached plan can be poor for other filter combinations. Very complex searches may still deserve dynamic SQL.

WHICH ONE SHOULD YOU PICK?

Prefer a static query. If the shape must change: sp_executesql for values and QUOTENAME with a whitelist for names. Never concatenate user input.

THE INTERVIEWER'S TRAP

`EXEC (@sql)` and `EXEC sp_executesql @sql` look similar, but only the second accepts parameters. Also remember that permissions: the caller needs rights on the underlying objects when running dynamic SQL inside a procedure, because ownership chaining does not apply.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Dynamic PIVOT**
  Build the column list with QUOTENAME and STRING_AGG (see Q22).
- **Dynamic ORDER BY**
  Validate the column name, QUOTENAME it, add ASC or DESC from a fixed list.
- **Why use SET NOCOUNT ON**
  Stops the extra row-count messages, which saves network round trips.
- **How do applications prevent injection**
  Parameterized queries or stored procedures. Never string building.

- **Least privilege**
  The application login should have only the permissions it needs.

ONE LINE TO REMEMBER

Never concatenate user input. Use sp_executesql with parameters for values and QUOTENAME with a whitelist for names.

Q39 Median salary

LOGICAL

Chance of being asked 45%

HOW THE INTERVIEWER ASKS IT

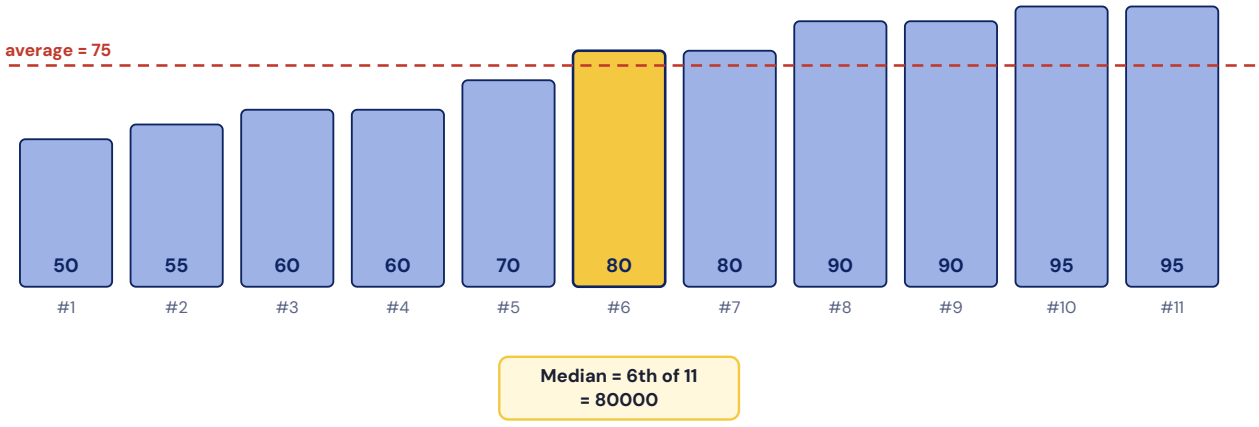
- *"Find the median salary."*
- *"Find the median salary of each department."*
- *"Find the median without using PERCENTILE functions."*

Say this first

"The median is the middle value after sorting. With an odd number of rows it is the middle row. With an even number it is the average of the two middle rows. SQL Server has PERCENTILE_CONT for this, but it is available only as a window function, so I add DISTINCT or use a subquery."

Picture it

Salaries sorted low to high (in thousands). The median is the middle bar.



With an EVEN count (a department with 4 people) the median is the average of the two middle values.

Median = the middle value of the sorted list.

WAY 1 PERCENTILE_CONT

T-SQL

```
SELECT DISTINCT DeptID,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY Salary)
           OVER (PARTITION BY DeptID) AS MedianSalary
FROM Employees
ORDER BY DeptID;
```

RESULT

```
DeptID  MedianSalary
------  ------------
1       85000
2       85000
3       60000
```

WHY USE IT

Shortest. It interpolates, so for an even number of rows it averages the two middle ones. Works per group with PARTITION BY.

WHEN IT FAILS

In SQL Server it exists only in the window form, hence DISTINCT. Needs SQL Server 2012 or later. It sorts the data, which is heavy on very large tables. PERCENTILE_DISC returns an actual existing value instead of an average.

EXECUTION ORDER Way 1: PERCENTILE_CONT per department

```
SELECT DISTINCT DeptID,
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY Salary) OVER (PARTITION BY DeptID) AS MedianSalary
FROM Employees
ORDER BY DeptID;
```

1 FROM — Employees: 11 rows.

2 SELECT (window) — Rows are split into partitions by DeptID. In each partition the salaries are sorted and the median is found. PERCENTILE_CONT(0.5) averages the two middle values when the count is even. The same value is attached to every row of the partition.

DeptID	Sorted salaries	Median
1	60000, 80000, 90000, 95000	85000
2	55000, 80000, 90000, 95000	85000
3	50000, 60000, 70000	60000

3 DISTINCT — 11 rows (each carrying its department median) collapse to 3 unique rows.

4 ORDER BY — Sorted by DeptID.

WAY 2 ROW_NUMBER from both ends

T-SQL

```
WITH s AS (
    SELECT Salary,
           ROW_NUMBER() OVER (ORDER BY Salary)      AS asc_rn,
           ROW_NUMBER() OVER (ORDER BY Salary DESC) AS desc_rn
    FROM Employees)
SELECT AVG(1.0 * Salary) AS Median
FROM s
WHERE asc_rn IN (desc_rn - 1, desc_rn, desc_rn + 1);
```

RESULT

```
Median
------
80000.000000
```

WHY USE IT

No special function. The middle row has equal numbers from both ends. For an even count the two middle rows differ by one. Works on any version that has window functions.

WHEN IT FAILS

The logic is clever but hard to read on the first look. Ties are fine because row numbers are unique, but per-group medians need PARTITION BY on both numbers.

WAY 3 OFFSET FETCH with a count

T-SQL

```
SELECT AVG(1.0 * Salary) AS Median
FROM (SELECT Salary
      FROM Employees
      ORDER BY Salary
      OFFSET (SELECT (COUNT(*) - 1) / 2 FROM Employees) ROWS
      FETCH NEXT (SELECT 2 - COUNT(*) % 2 FROM Employees) ROWS ONLY) AS m;
```

WHY USE IT

Sorts once and picks only the one or two middle rows. Efficient and works for the whole table.

WHEN IT FAILS

Hard to read, needs SQL Server 2012 or later, and awkward for per-group medians.

WHICH ONE SHOULD YOU PICK?

Default: PERCENTILE_CONT. Mention the ROW_NUMBER trick if they ask "without PERCENTILE".

THE INTERVIEWER'S TRAP

The median of 11 salaries is the 6th value, which is 80000. A plain average of those same salaries is 75000, close here, but one very high salary would pull the average up and leave the median unchanged. Say clearly why the median is better for salaries: outliers do not pull it up.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Median per department**
  PARTITION BY DeptID, as shown. Our data: 85000, 85000 and 60000.

- **Mode (most frequent salary)**
  `SELECT TOP 1 WITH TIES Salary FROM Employees GROUP BY Salary ORDER BY COUNT(*) DESC .`
- **90th percentile**
  `PERCENTILE_CONT(0.9) .`
- **Average excluding the highest and lowest**
  Filter with `Salary > (SELECT MIN(Salary) ...) AND Salary < (SELECT MAX(Salary) ...)` and average the rest.

ONE LINE TO REMEMBER

Median = middle value. PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY col) OVER (...) in SQL Server, with DISTINCT. Alternative: ROW_NUMBER from both ends.

Q40 Date logic questions

LOGICAL

Chance of being asked 50%

HOW THE INTERVIEWER ASKS IT

- *"Get the first and last day of the current month."*
- *"Find orders placed in March 2026."*
- *"Calculate a person's age in years."*

Say this first

"For date ranges I always use a half-open range: the start date included, the next start date excluded. For first and last days I use EOMONTH and DATEFROMPARTS. And I remember that DATEDIFF counts boundaries crossed, not full years or days elapsed."

Picture it

A DATETIME column. Order at 2026-03-31 14:00 must be in March.



BETWEEN loses the whole last day when the column has a time part.

Task	T-SQL
Last day of the current month	`EOMONTH(GETDATE())`
First day of the current month	`DATEFROMPARTS(YEAR(GETDATE()), MONTH(GETDATE()), 1)`
First day of month, older versions	`DATEADD(MONTH, DATEDIFF(MONTH, 0, GETDATE()), 0)`
Orders in the last 7 days	`OrderDate >= DATEADD(DAY, -7, CAST(GETDATE() AS DATE))`
Day name	`DATENAME(WEEKDAY, OrderDate)`
Exact age in years	`DATEDIFF(YEAR, dob, GETDATE()) - CASE WHEN DATEADD(YEAR, DATEDIFF(YEAR, dob, GETDATE()), dob) > GETDATE() THEN 1 ELSE 0 END`

WAY 1 BETWEEN with dates (the trap)

T-SQL

```
-- Looks right, is wrong for datetime columns
WHERE OrderDate BETWEEN '2026-03-01' AND '2026-03-31'
```

WHY USE IT

Reads naturally and works when the column is a DATE with no time part.

WHEN IT FAILS

If the column is DATETIME, an order at 2026-03-31 14:00 is excluded, because '2026-03-31' means midnight. BETWEEN includes both ends, which also makes ranges overlap if you chain them.

WAY 2 Half-open range (the correct way)

T-SQL

```
WHERE OrderDate >= '2026-03-01'
  AND OrderDate <  '2026-04-01'
```

WHY USE IT

Correct for DATE, DATETIME and DATETIME2, and it can use an index seek. Next month starts exactly where this one stops, so there is no gap and no overlap.

WHEN IT FAILS

Slightly longer to type. You must compute the next start date, which is easy with `DATEADD(MONTH, 1, @start)`.

WAY 3 YEAR() and MONTH() functions

```tsql
WHERE YEAR(OrderDate) = 2026 AND MONTH(OrderDate) = 3
```

WHY USE IT

Very easy to read, and fine for tiny tables.

WHEN IT FAILS

Functions on the column stop index seeks (see Q30). Slow on large tables.

THE INTERVIEWER'S TRAP

`DATEDIFF(YEAR, '2025-12-31', '2026-01-01')` returns **1**, although only one day has passed. DATEDIFF counts how many year boundaries are crossed. That is why a simple DATEDIFF is wrong for age, and the CASE correction in the table is needed.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Year-over-year sales**
  Group by month, then compare to the same month last year with `LAG(Sales, 12)` or a self join on `DATEADD(YEAR, -1, ...)`.
- **Customers active every month this year**
  `HAVING COUNT(DISTINCT MONTH(OrderDate)) = MONTH(GETDATE())`.
- **Week number**
  `DATEPART(ISO_WEEK, d)` is consistent. `DATEPART(WEEK, d)` depends on `SET DATEFIRST`.
- **Generate a calendar table**
  A recursive CTE (see Q34).
- **Business days between two dates**
  Count days with DATEDIFF, subtract weekends and a holiday table.

ONE LINE TO REMEMBER

Use a half-open date range, not BETWEEN, for datetime columns. DATEDIFF counts boundaries, not elapsed time. EOMONTH gives the last day of the month.

Q41 Normalization, and when to denormalize

IMPORTANT

Chance of being asked 50%

HOW THE INTERVIEWER ASKS IT

- *"What is normalization? Explain 1NF, 2NF and 3NF."*
- *"When would you denormalize?"*
- *"What is the difference between OLTP and OLAP schema design?"*

Say this first

"Normalization organises tables so each fact is stored once. The easy line to remember is: every non-key column depends on the key, the whole key, and nothing but the key. That gives fewer update mistakes. Denormalization breaks that rule on purpose to make reads faster, mostly in reporting."

Picture it

Not 3NF: DeptName repeats

EmpID	Name	DeptID	DeptName
1	Asha	2	IT
2	Vikram	2	IT
5	Priya	2	IT
3	Neha	1	Sales

Rename IT once = update 3 rows, easy to miss one.

split →

3NF: each fact once

EmpID	Name	DeptID
1	Asha	2
2	Vikram	2
5	Priya	2
3	Neha	1

DeptID	DeptName
2	IT
1	Sales

Fix the repeated DeptName by giving departments their own table.

Normal form	Rule	Example of a violation and the fix
1NF	One value per cell. No repeating groups	A Phones column holding '98000, 97000'. Move phones into a child table with one row per phone
2NF	1NF, and no column depends on only part of a composite key	OrderItems(OrderID, ProductID, ProductName). ProductName depends only on ProductID. Move it to a Products table
3NF	2NF, and no column depends on another non-key column	Employees(EmpID, DeptID, DeptName). DeptName depends on DeptID, not EmpID. Move it to Departments

WAY 1 Normalized design (OLTP)

WHY USE IT

Each fact lives in one place, so an update touches one row and you avoid update, insert and delete anomalies. Best for systems with many small writes, like orders and payments.

WHEN IT FAILS

More joins for reading, and a report that needs data from many tables can get slow.

WAY 2 Denormalized design (reporting tables)

WHY USE IT

Fewer joins, so read queries are much faster. Useful for dashboards and read-heavy workloads.

WHEN IT FAILS

Duplicated data can fall out of sync, so you need a refresh process. Updates become more expensive.

WAY 3 Star schema (analytics)

WHY USE IT

One central fact table (sales) surrounded by dimension tables (date, product, customer). Simple for BI tools, fast for aggregation, and easy to understand.

WHEN IT FAILS

It needs an ETL process to build and keep it updated, and it is a poor fit for transactional systems.

Picture it



Facts = numbers you add up. Dimensions = the labels you group by.

For reporting, numbers go in a fact table and descriptions in dimension tables.

WHICH ONE SHOULD YOU PICK?

OLTP system: normalize to 3NF. Reporting or BI: build a star schema or denormalized tables. Always say how you will keep duplicated data in sync.

THE INTERVIEWER'S TRAP

Do not say "denormalization is bad". It is a deliberate trade: faster reads for harder writes. The clean answer is "normalize for correctness first, then denormalize only where measurements show a need".

MORE QUESTIONS HIDING INSIDE THIS ONE

- **BCNF**
  A stricter version of 3NF. Every determinant must be a candidate key.
- **Insert, update and delete anomalies**
  Problems caused by repeating facts. Give one example of each.
- **Surrogate key vs natural key**
  A generated number vs a real-world value like email (see Q44).
- **Star schema vs snowflake schema**
  Snowflake normalizes the dimensions into more tables. Star keeps them flat.
- **Is a computed total column a violation**
  Technically yes, but it is a common, deliberate choice for performance.

ONE LINE TO REMEMBER

The key, the whole key, and nothing but the key. Normalize for OLTP, denormalize or use a star schema for reporting.

LEVEL 4 OF 4

Senior

5+ years

10 questions in this level

WHAT THE INTERVIEWER IS REALLY CHECKING

Can you diagnose production problems, design for scale, and justify trade-offs?

HOW TO WIN THIS LEVEL

Senior rounds are scenario-led. There is rarely one right answer. They score your reasoning, the trade-offs you name, and how you would prove the fix worked.

THREE HABITS FOR THIS LEVEL

- Structure every answer as: what I check first, what I conclude, what I change, how I prove it.
- Name the trade-off of every recommendation (cost, risk, complexity).
- Mention monitoring. Fixing it once is not enough, you must catch it earlier next time.

Q42 It was fast yesterday and 40 seconds today. What changed?

MOST ASKED IMPORTANT

Chance of being asked 70%

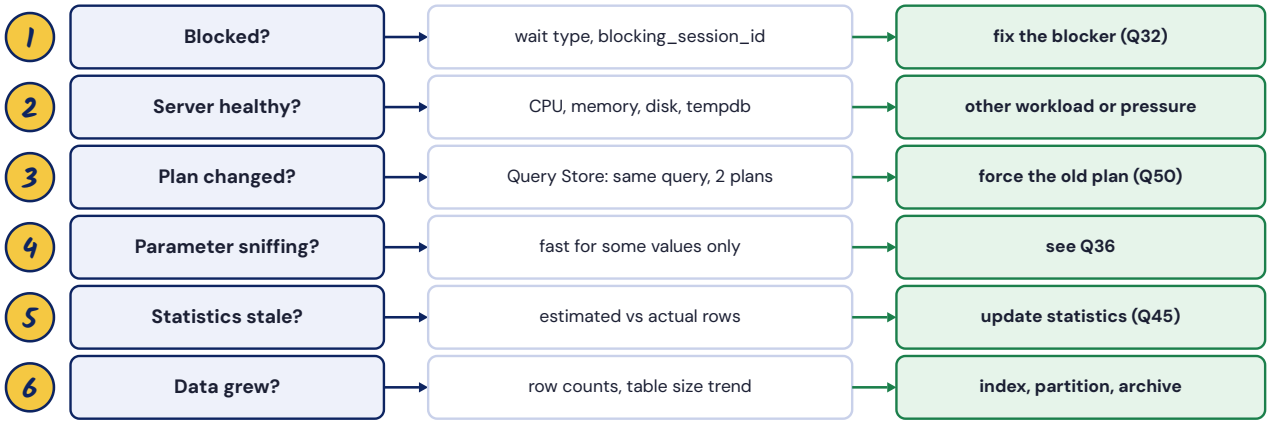
HOW THE INTERVIEWER ASKS IT

- *"A report ran in 2 seconds yesterday and takes 40 seconds today. Nothing was deployed. What do you do?"*
- *"How do you find a performance regression in production?"*

Say this first

"Something changed even if nobody deployed. I do not guess. I split the problem in three: is the query waiting, is it running a different plan, or is the data different? I check waits and blocking first, then compare today's plan with yesterday's plan in Query Store, then look at statistics and data volume."

Picture it



Go down the list in order. Stop at the first thing that explains the slowdown.

Step	Check	Tool	Likely cause and fix
1. Is it blocked?	Wait type, blocking_session_id	`sys.dm_exec_requests`	Long transaction upstream. Fix the blocker (Q32)
2. Server healthy?	CPU, memory, disk latency, tempdb	DMVs, Perfmon	Another workload, memory pressure
3. Plan changed?	Compare plans for the same query_id	Query Store	Plan regression. Force the old plan (Q50)
4. Sniffing?	Fast for some parameters only	Actual plan	See Q36
5. Statistics stale?	Estimated vs actual rows	Plan, `sys.dm_db_stats_properties`	Update statistics (Q45)
6. Data volume jumped?	Row counts, size trend	Table sizes	Index, partitioning or archive
7. Something changed?	Compatibility level, patch, index job	Server and agent logs	Roll back or retest

```T-SQL
-- Top queries by duration in the last day, with plan counts
SELECT TOP (10) q.query_id, qt.query_sql_text,
       COUNT(DISTINCT p.plan_id)            AS PlanCount,
       AVG(rs.avg_duration) / 1000.0        AS AvgMs
FROM sys.query_store_query q
JOIN sys.query_store_query_text qt  ON qt.query_text_id = q.query_text_id
JOIN sys.query_store_plan p         ON p.query_id = q.query_id
JOIN sys.query_store_runtime_stats rs ON rs.plan_id = p.plan_id
JOIN sys.query_store_runtime_stats_interval i
     ON i.runtime_stats_interval_id = rs.runtime_stats_interval_id
WHERE i.start_time > DATEADD(DAY, -1, SYSUTCDATETIME())
GROUP BY q.query_id, qt.query_sql_text
ORDER BY AvgMs DESC;
```

WAY 1 Look at waits first

WHY USE IT

If the query is waiting on locks, no index or rewrite will help. Wait type tells you whether to chase CPU, I/O, memory or blocking.

WHEN IT FAILS

Waits describe the symptom, not always the cause. A PAGEIOLATCH wait may mean a missing index or a slow disk.

WAY 2 Compare plans in Query Store

WHY USE IT

The best evidence: you can show the old fast plan and the new slow plan side by side and prove that the plan changed.

WHEN IT FAILS

Query Store must have been enabled before the incident. On older versions or if it is off, you only have the current plan.

WAY 3 Check statistics and data growth

WHY USE IT

Statistics that have not been updated after a big load give the optimizer wrong row estimates, which leads to wrong join types and memory spills.

WHEN IT FAILS

Updating statistics also invalidates cached plans, which can hide the evidence. Capture the bad plan first.

WHICH ONE SHOULD YOU PICK?

Order: blocked? plan changed? stale statistics? Server pressure? Capture evidence before you fix, then fix the root cause, then add monitoring so you hear about it before users do.

THE INTERVIEWER'S TRAP

Do not restart the service or clear the plan cache as the first step. It destroys the evidence and the problem comes back next week.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **What if Query Store is off**
  Use `sys.dm_exec_query_stats` and the plan cache, then turn Query Store on.
- **Only slow at month end**
  Look for data volume, a nightly job, or a scheduled report competing for resources.
- **Slow only for one user**
  Check their parameter values, SET options and permissions-related row filters.
- **Recently upgraded**
  Compare compatibility level and the cardinality estimator version.

ONE LINE TO REMEMBER

Waits first, then plan comparison, then statistics and data. Capture evidence before fixing. Do not restart blindly.

Q43 Indexing strategy for a big table

IMPORTANT

Chance of being asked 65%

HOW THE INTERVIEWER ASKS IT

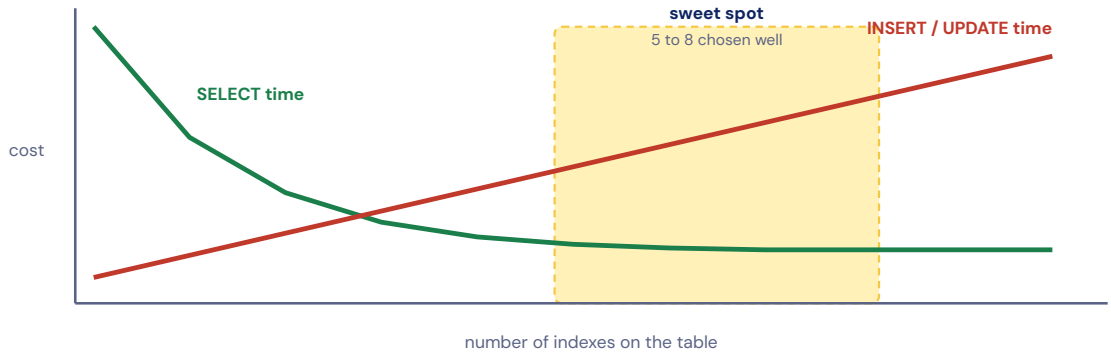
- *"How would you design indexes for a table with 500 million rows?"*
- *"How many indexes are too many?"*
- *"How do you find unused and missing indexes?"*

Say this first

"I start from the workload, not from the table. I list the most frequent and most expensive queries, design a small number of indexes that serve them, and then check usage statistics to remove the ones nobody reads. Every index speeds up reads and slows down writes, so the goal is the fewest indexes that cover the important queries."

Picture it

Every extra index: reads get faster, writes get slower (illustrative)



The shape is what matters, not the numbers.

Rule	Why
Equality columns first, then range or sort columns	The index can seek on equality and then read a range in order
Put the most selective filter column first, when queries filter on it	Narrows the search fastest
Use INCLUDE for columns that are only selected	Avoids lookups without widening the key
Filtered index for a small hot subset	`WHERE Status = 'Open'` stays tiny
Keep the clustered key narrow, unique and increasing	Every nonclustered index carries a copy of it (Q44)
Review usage regularly	`sys.dm_db_index_usage_stats` shows reads vs writes

T-SQL

```
-- Indexes that cost writes but are never read since the last restart
SELECT OBJECT_NAME(i.object_id) AS TableName, i.name AS IndexName,
       us.user_seeks, us.user_scans, us.user_lookups, us.user_updates
FROM sys.indexes i
LEFT JOIN sys.dm_db_index_usage_stats us
       ON us.object_id = i.object_id AND us.index_id = i.index_id
      AND us.database_id = DB_ID()
WHERE OBJECTPROPERTY(i.object_id, 'IsUserTable') = 1
  AND i.type_desc = 'NONCLUSTERED'
  AND ISNULL(us.user_seeks + us.user_scans + us.user_lookups, 0) = 0;
```

WAY 1 Design from the workload

WHY USE IT

Indexes match real queries, so each one earns its cost. This is how senior engineers work.

WHEN IT FAILS

Needs a representative workload. A trace taken on a quiet day misses the month-end report.

WAY 2 Follow the missing index DMVs

T-SQL

```
SELECT TOP (10) d.statement, d.equality_columns, d.inequality_columns, d.included_columns,
       s.user_seeks, s.avg_user_impact
FROM sys.dm_db_missing_index_details d
JOIN sys.dm_db_missing_index_groups g ON g.index_handle = d.index_handle
JOIN sys.dm_db_missing_index_group_stats s ON s.group_handle = g.index_group_handle
ORDER BY s.user_seeks * s.avg_user_impact DESC;
```

WHY USE IT

Free hints from the optimizer about what it wished it had. Good starting point.

WHEN IT FAILS

Suggests overlapping indexes, ignores column order subtleties, ignores write cost, and resets on restart. Treat it as a hint, not a to-do list.

WAY 3 Remove unused and duplicate indexes

WHY USE IT
Less storage, faster inserts, updates and deletes, and shorter maintenance windows.

WHEN IT FAILS
Usage counters reset on restart, so an index used only quarterly can look unused. Check uptime and keep a script to recreate the index.

WHICH ONE SHOULD YOU PICK?
Workload first, DMVs as hints, and a review every quarter. For a huge table also consider partitioning (Q47) and columnstore (Q48).

THE INTERVIEWER'S TRAP
"More indexes make everything faster" is the wrong answer. Say that each index slows every write and uses memory, and that 5 to 8 well-chosen indexes usually beat 20 random ones.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Index on (A, B) vs two indexes on A and B**
  The composite serves A, and A with B. Two single indexes can be combined but less efficiently.
- **Column order**
  Equality first, range last. Our query `WHERE CustomerID = 1 AND OrderDate >= x` fits (CustomerID, OrderDate).
- **Unique indexes**
  They enforce a rule and also help the optimizer.
- **Online index creation**
  `WITH (ONLINE = ON)` where the edition supports it.
- **Indexed views**
  Materialise an aggregate. Fast reads, slower writes, strict rules.

ONE LINE TO REMEMBER
Design indexes from the workload, use DMVs as hints only, remove unused ones, and count the write cost of every index.

Q44 Choosing the clustered index key: IDENTITY vs GUID

IMPORTANT

Chance of being asked 55%

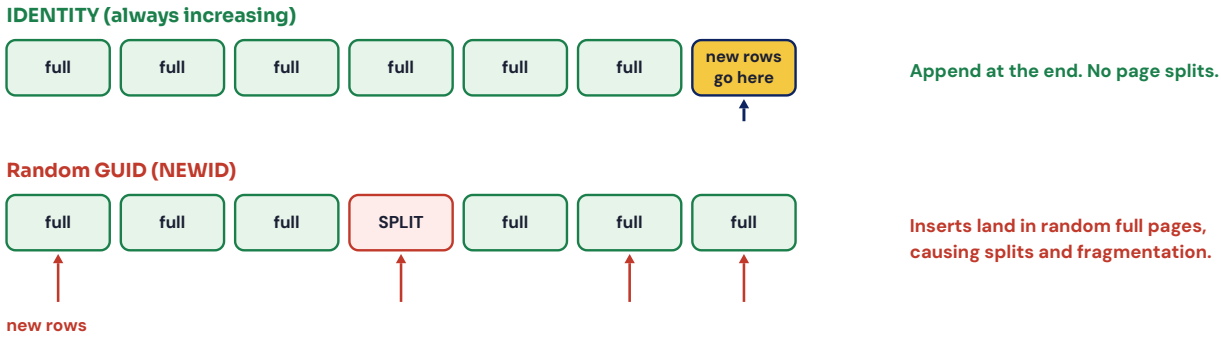
HOW THE INTERVIEWER ASKS IT

- *"What makes a good clustered index key?"*
- *"Is a GUID a good primary key?"*
- *"What is the difference between NEWID and NEWSEQUENTIALID?"*

Say this first

"A good clustered key is narrow, unique, static and ever-increasing. Narrow, because every nonclustered index stores a copy of it. Increasing, because new rows go to the end of the table and pages do not split. An INT or BIGINT IDENTITY fits perfectly. A random GUID does not."

Picture it



Why random keys cost more than increasing keys.

Key	Size	Order	Effect
INT IDENTITY	4 bytes	Increasing	**Compact, no page splits, best default**
BIGINT IDENTITY	8 bytes	Increasing	Use when 2 billion rows is possible
NEWID() GUID	16 bytes	Random	**Page splits and heavy fragmentation, larger nonclustered indexes**
NEWSEQUENTIALID() GUID	16 bytes	Mostly increasing	Fine for the clustered key, only as a column default
Natural key such as email	Variable	Random	Wide and can change, poor clustered key

WAY 1 INT or BIGINT IDENTITY clustered key

```t-sql
CREATE TABLE dbo.Orders2 (
    OrderID BIGINT IDENTITY(1,1) NOT NULL,
    CustomerID INT NOT NULL,
    CONSTRAINT PK_Orders2 PRIMARY KEY CLUSTERED (OrderID));
```

WHY USE IT

Small, fast and sequential. Inserts append at the end, so the table stays compact.

WHEN IT FAILS

Not globally unique, so merging data from several servers can collide. Values can have gaps after rollbacks or restarts. Do not expose it to users if guessing IDs is a security concern.

WAY 2 GUID primary key, clustered on a different column

T-SQL

```
CREATE TABLE dbo.Customers2 (
    CustomerGuid UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
    CONSTRAINT PK_Customers2 PRIMARY KEY NONCLUSTERED (CustomerGuid));
CREATE CLUSTERED INDEX CIX_Customers2 ON dbo.Customers2 (CreatedAt);
```

WHY USE IT

You keep the GUID for distributed systems and uniqueness across servers, while the table itself stays ordered by time.

WHEN IT FAILS

Two indexes to maintain. Lookups by GUID go through a nonclustered index plus a key lookup.

WAY 3 NEWSEQUENTIALID() as default

T-SQL

```
CREATE TABLE dbo.Events (
    EventID UNIQUEIDENTIFIER NOT NULL DEFAULT NEWSEQUENTIALID() PRIMARY KEY);
```

WHY USE IT

GUIDs that increase, so page splits mostly go away while uniqueness is kept.

WHEN IT FAILS

Allowed only as a column default, not in a query. The sequence can restart after a Windows restart, and it is guessable, so do not use it where predictability is a risk.

WHICH ONE SHOULD YOU PICK?

Default to BIGINT IDENTITY. If you need global uniqueness, use a GUID key but cluster on another column, or use NEWSEQUENTIALID. Avoid random GUIDs as the clustered key.

THE INTERVIEWER'S TRAP

A common mix-up: primary key does not mean clustered index. By default a primary key creates a clustered index, but you can choose otherwise, as the GUID example shows.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Heap vs clustered table**
  A heap has no clustered index, so rows are stored unordered and lookups use RID. Heaps suit staging loads.
- **Why keep the clustered key narrow**
  It is stored in every nonclustered index.
- **IDENTITY gaps**
  Caused by rollbacks, deleted rows and failed inserts. Not a bug.
- **SEQUENCE object vs IDENTITY**
  SEQUENCE is shared across tables and can be reserved ahead of insert.

ONE LINE TO REMEMBER

Narrow, unique, static, increasing. IDENTITY is the default. Random GUIDs fragment, so cluster on something else or use NEWSEQUENTIALID.

Q45 Statistics and cardinality estimation

IMPORTANT

Chance of being asked 55%

HOW THE INTERVIEWER ASKS IT

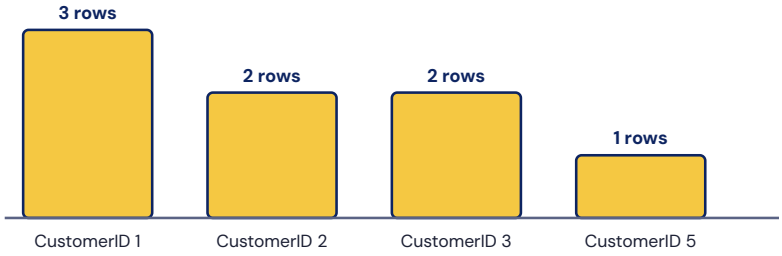
- *"What are statistics and why do they matter?"*
- *"When are statistics updated automatically?"*
- *"What is cardinality estimation?"*

Say this first

"Statistics are small summaries of how values are distributed in a column: a histogram plus density information. The optimizer uses them to guess how many rows each step will return, which is called cardinality estimation. Good guesses give good plans. Bad guesses give wrong join types, wrong memory grants and spills to tempdb."

Picture it

What statistics store: a histogram of Orders.CustomerID



The optimizer reads this to guess: CustomerID = 1 will return about 3 rows.

Stale statistics = wrong guess = wrong join type, wrong memory grant, tempdb spill.

Statistics are a summary of how the values are spread.

T-SQL

```
DBCC SHOW_STATISTICS ('dbo.Orders', IX_Orders_Customer);

SELECT s.name, sp.last_updated, sp.rows, sp.rows_sampled, sp.modification_counter
FROM sys.stats s
CROSS APPLY sys.dm_db_stats_properties(s.object_id, s.stats_id) sp
WHERE s.object_id = OBJECT_ID('dbo.Orders');
```

WAY 1 Rely on auto-update statistics

WHY USE IT

Turned on by default and needs no work. On SQL Server 2016 and later with compatibility 130+, the threshold scales with table size.

WHEN IT FAILS

Even the scaled threshold can be slow to fire on huge tables, and the default sampling can miss skewed values. Large data loads can leave statistics stale until the next trigger.

WAY 2 Update statistics manually after big changes

```T-SQL
UPDATE STATISTICS dbo.Orders WITH FULLSCAN;
-- or one statistic only
UPDATE STATISTICS dbo.Orders IX_Orders_Customer WITH SAMPLE 30 PERCENT;
```

WHY USE IT

Right after an ETL load you know the data changed, so you refresh immediately instead of waiting.

WHEN IT FAILS

FULLSCAN on a very large table is heavy. It also forces recompilation of queries that use it.

WAY 3 Asynchronous auto-update

```T-SQL
ALTER DATABASE InterviewPrep SET AUTO_UPDATE_STATISTICS_ASYNC ON;
```

WHY USE IT

The query that triggers the update does not wait. It uses the old statistics while the update runs in the background.

WHEN IT FAILS

That triggering query still runs with the stale statistics, so it can be slow once.

WHICH ONE SHOULD YOU PICK?

Keep auto-update on. Add a manual update after bulk loads. For skewed data on big tables, use filtered statistics or a higher sample.

THE INTERVIEWER'S TRAP

Index rebuild updates statistics with a full scan, but REORGANIZE does not (Q37). Many teams rebuild by habit and forget that the statistics update is a big part of the benefit.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Histogram**
  Up to 200 steps showing value ranges and row counts.
- **Estimated vs actual row gap**
  The first clue of bad statistics.
- **Legacy vs new cardinality estimator**
  Compatibility level 120 and above uses the newer estimator, which changes some plans.
- **Table variable estimates**
  Historically a fixed guess of 1 row. Deferred compilation in 2019 improves this.
- **Auto-create statistics**
  SQL Server creates single-column statistics when a query filters on a column.

ONE LINE TO REMEMBER

Statistics drive row estimates and plan quality. Auto-update is on by default. Update manually after bulk loads. Big estimate gaps mean bad statistics.

Q46 Recovery models, backups and point-in-time restore

MOST ASKED IMPORTANT Chance of being asked 55%

HOW THE INTERVIEWER ASKS IT

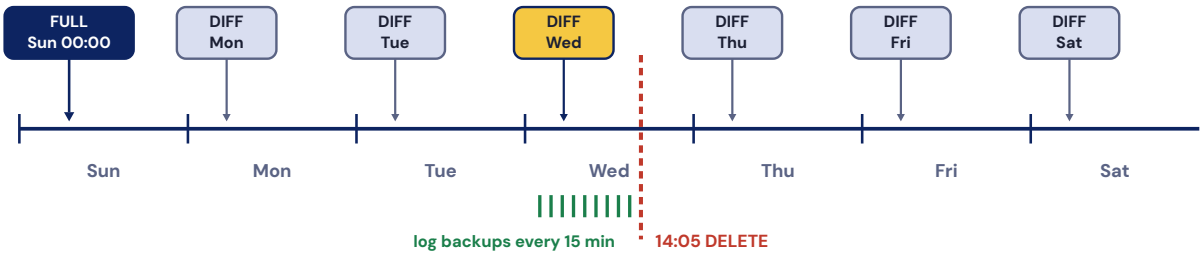
- *"What are the recovery models?"*
- *"What types of backup are there? How do you restore to a point in time?"*
- *"The log file keeps growing. Why?"*

Say this first

"The recovery model decides how much transaction log is kept. SIMPLE reclaims the log automatically, so I can only restore to the last full or differential backup. FULL keeps the log until I back it up, so I can restore to any point in time. BULK_LOGGED is FULL with minimal logging for bulk operations."

Picture it

A week of backups, and a mistaken DELETE on Wednesday at 14:05



Restore: FULL (Sun) -> DIFF (Wed 00:00) -> log backups ... STOPAT 14:04:59
The last log restore uses WITH STOPAT and RECOVERY. Everything else uses NORECOVERY.

The restore chain: one full, one differential, then logs up to the moment before the mistake.

Model	Log backups	Point-in-time restore	Typical use
SIMPLE	Not possible	No	Development, easily rebuilt data
FULL	Required	Yes	Production OLTP
BULK_LOGGED	Required	Not inside a bulk operation	Short window of big loads, then back to FULL

Backup type	What it contains
Full	The entire database
Differential	Everything changed since the last full backup
Transaction log	The log since the last log backup

```T-SQL
-- Restore to just before a mistaken DELETE at 14:05
RESTORE DATABASE Shop FROM DISK = 'D:\bk\Shop_full.bak'
    WITH NORECOVERY, REPLACE;
RESTORE DATABASE Shop FROM DISK = 'D:\bk\Shop_diff.bak'
    WITH NORECOVERY;
RESTORE LOG Shop FROM DISK = 'D:\bk\Shop_log1.trn'
    WITH NORECOVERY;
RESTORE LOG Shop FROM DISK = 'D:\bk\Shop_log2.trn'
    WITH STOPAT = '2026-10-15T14:04:59', RECOVERY;
```

WAY 1 Full + differential + log backups

WHY USE IT

The standard production plan: weekly full, daily differential, log backups every 5 to 15 minutes. Fast restores and a small data-loss window (RPO).

WHEN IT FAILS

A broken log chain (a missed log backup, or switching to SIMPLE) makes later logs unusable. You must test restores, because an untested backup is only a hope.

WAY 2 SIMPLE recovery with frequent full backups

WHY USE IT

No log management at all. Good for dev, test, or a copy of data that can be reloaded.

WHEN IT FAILS

Everything since the last backup is lost on failure, and no point-in-time restore.

WAY 3 BULK_LOGGED for a big load

WHY USE IT

Minimal logging makes large imports and index rebuilds much faster and keeps the log small.

WHEN IT FAILS

A log backup that contains a bulk operation cannot be restored to a point in time inside it. Switch back to FULL and take a log backup right after.

Why does my log keep growing?

In FULL recovery the log cannot be reused until a log backup runs. Check `log_reuse_wait_desc` in `sys.databases`: LOG_BACKUP means you need log backups, ACTIVE_TRANSACTION means a long open transaction, REPLICATION or AVAILABILITY_REPLICA means a subscriber or replica is behind. Shrinking the log is not the fix.

WHICH ONE SHOULD YOU PICK?

Production with important data: FULL with regular log backups. Always agree RPO (how much data can we lose) and RTO (how long can we be down) with the business first, then choose backups to fit.

THE INTERVIEWER'S TRAP

`COPY_ONLY` backups do not break the differential base or the log chain. Also, never answer "backup to the same disk": a backup on the same drive is lost together with the database.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **RESTORE WITH NORECOVERY vs RECOVERY**
  NORECOVERY keeps the database restoring so more backups can follow. RECOVERY brings it online.
- **Tail-log backup**
  The last log backup before a restore, to avoid losing the most recent transactions.
- **Verify a backup**
  `RESTORE VERIFYONLY` checks readability, but only a real test restore proves it.
- **Backup compression and checksum**
  Use `WITH COMPRESSION, CHECKSUM`.
- **What if the master database is lost**
  Restore it in single-user mode from a backup.

ONE LINE TO REMEMBER

SIMPLE no log backups, FULL point-in-time with log backups, BULK_LOGGED for big loads. Full + differential + logs, and test restores. Log growing means no log backup or an open transaction.

Q47 Table partitioning

IMPORTANT

Chance of being asked 50%

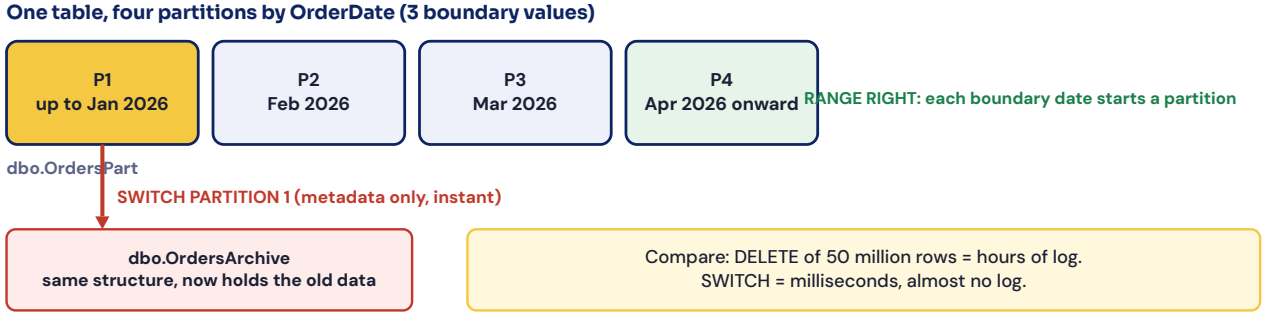
HOW THE INTERVIEWER ASKS IT

- *"What is table partitioning? When do you use it?"*
- *"Is partitioning a performance feature?"*
- *"How do you archive old data quickly?"*

Say this first

"Partitioning splits one large table into smaller pieces by a column value, usually a date, while it still looks like one table. Its biggest benefit is manageability: I can switch an old partition out in a second instead of deleting millions of rows. Query speedup is a possible bonus through partition elimination, but it is not guaranteed."

Picture it



Switching a partition moves it without copying rows.

```T-SQL
CREATE PARTITION FUNCTION pf_OrderDate (DATE)
    AS RANGE RIGHT FOR VALUES ('2026-02-01', '2026-03-01', '2026-04-01');
CREATE PARTITION SCHEME ps_OrderDate
    AS PARTITION pf_OrderDate ALL TO ([PRIMARY]);

CREATE TABLE dbo.OrdersPart (
    OrderID BIGINT NOT NULL, OrderDate DATE NOT NULL, Amount MONEY,
    CONSTRAINT PK_OrdersPart PRIMARY KEY CLUSTERED (OrderDate, OrderID))
ON ps_OrderDate (OrderDate);
```

WAY 1 Partition switching for archiving

```T-SQL
ALTER TABLE dbo.OrdersPart SWITCH PARTITION 1 TO dbo.OrdersArchive;
```

WHY USE IT

A metadata-only change, so moving millions of rows takes milliseconds and generates almost no log.

WHEN IT FAILS

The target table must have an identical structure, indexes and constraints on the same filegroup. It is strict, and one mismatch fails the switch.

WAY 2 Partition elimination for queries

WHY USE IT

A query with a filter on the partition column reads only the matching partitions, so scans touch less data.

WHEN IT FAILS

Queries that do not filter on the partition column read all partitions, and can even be slower. The partition column must be part of the clustered key and every unique index (aligned indexes).

WAY 3 Delete in batches (no partitioning)

```T-SQL
WHILE 1 = 1
BEGIN
    DELETE TOP (50000) FROM dbo.Orders WHERE OrderDate < '2025-01-01';
    IF @@ROWCOUNT = 0 BREAK;
END;
```

WHY USE IT

Works on any table and edition, with no design change.

WHEN IT FAILS

Slow, heavy on the log and blocks if not batched. Fine for modest volumes, poor for hundreds of millions of rows.

WHICH ONE SHOULD YOU PICK?

Huge tables with a time-based lifecycle: partition by date and switch old data out. Small or medium tables: indexes and batching are enough.

THE INTERVIEWER'S TRAP

"Partitioning makes queries faster" is only half true. State that it is mainly for maintenance, and that performance gains depend on the query filtering on the partition key.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **RANGE LEFT vs RANGE RIGHT**
  Which side of the boundary the boundary value belongs to. RIGHT is common for dates.
- **Sliding window**
  Split a new partition in at the front, switch the oldest one out at the back.
- **Partitioned view**
  A UNION ALL view across several tables. The old way, still used.
- **Does partitioning need Enterprise**
  Since SQL Server 2016 SP1 it is available in Standard, but check your version.

ONE LINE TO REMEMBER

Partitioning is a manageability feature first. Switch old partitions out instantly. Put the partition column in the key, and filter on it.

Q48 Columnstore vs rowstore indexes

IMPORTANT

Chance of being asked 50%

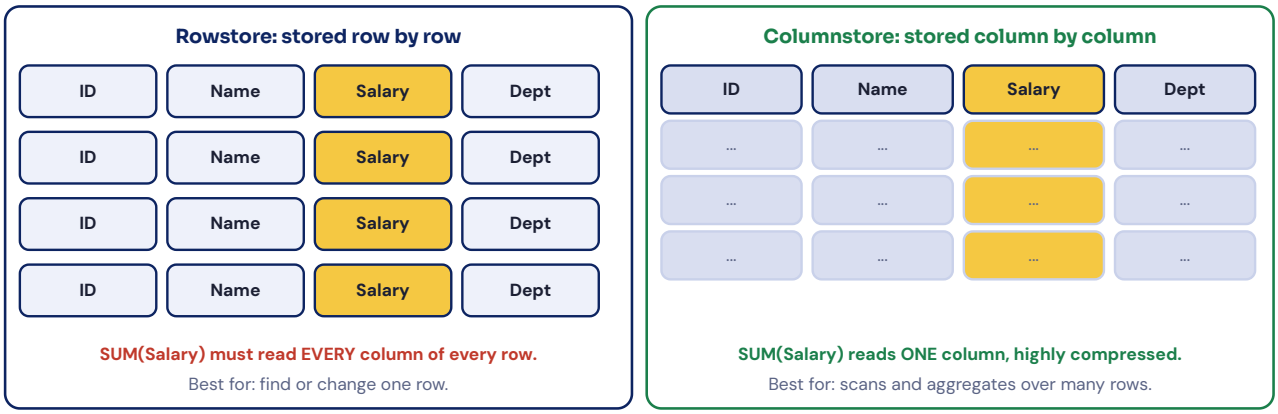
HOW THE INTERVIEWER ASKS IT

- *"What is a columnstore index? When would you use it?"*
- *"Why is columnstore fast for reports?"*
- *"Why is it bad for single-row lookups?"*

Say this first

"A rowstore index stores whole rows together, which is great for finding or changing one row. A columnstore index stores each column separately and compresses it heavily, which is great for scanning millions of rows and aggregating a few columns. That is why transactional tables use rowstore and big reporting tables use columnstore."

Picture it



The same data, stored two ways.

Feature	Rowstore (B-tree)	Columnstore
Best for	OLTP: seeks, small updates	Analytics: scans and aggregates
Compression	Modest	**Very high, often 5 to 10 times**
Reads	Whole rows	Only the columns requested
Batch mode	Mostly row mode	Batch mode processing, much faster
Single-row lookup	**Excellent**	**Poor**
Frequent small updates	Fine	Costly (delta store, then compressed)

```T-SQL
-- Clustered columnstore for a fact table
CREATE CLUSTERED COLUMNSTORE INDEX CCI_FactSales ON dbo.FactSales;

-- Nonclustered columnstore for reporting on an OLTP table (real-time analytics)
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_Orders
ON dbo.Orders (OrderDate, CustomerID, Amount);
```

WAY 1 Clustered columnstore on a fact or history table

WHY USE IT

Huge compression and batch mode make aggregation queries run many times faster, and the table takes far less space.

WHEN IT FAILS

Updates and deletes are heavier because rows go through a delta store, and point lookups on a key are slow unless you add a nonclustered rowstore index.

WAY 2 Nonclustered columnstore on an OLTP table

WHY USE IT

Gives real-time analytics on live data, without a separate warehouse, while the clustered rowstore still serves transactions.

WHEN IT FAILS

Adds write overhead to the busy table. Test the load carefully.

WAY 3 Stay with rowstore

WHY USE IT
Keeps the design simple for OLTP workloads dominated by seeks and small writes.

WHEN IT FAILS
Large aggregations scan far more data than they need and run slowly as the table grows.

WHICH ONE SHOULD YOU PICK?
Warehouse, fact tables, big append-only history: clustered columnstore. Mixed workloads: nonclustered columnstore. Pure OLTP: rowstore.

THE INTERVIEWER'S TRAP
Do not say columnstore is "faster" in general. It is faster for scans and aggregates on many rows. It is slower for lookups of single rows. Also mention that row groups hold about one million rows, and trickle inserts leave small row groups that need maintenance.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Delta store**
  A small rowstore where new rows wait until enough rows exist to compress.
- **Columnstore archival compression**
  Even smaller storage for cold data, with more CPU.
- **Segment elimination**
  Min and max values per segment let the engine skip segments, like partition elimination.
- **Batch mode on rowstore**
  Available from 2019 for some queries.

ONE LINE TO REMEMBER
Rowstore for OLTP, columnstore for analytics. High compression and batch mode, poor for single-row lookups.

Q49 High availability and disaster recovery options

IMPORTANT

Chance of being asked 50%

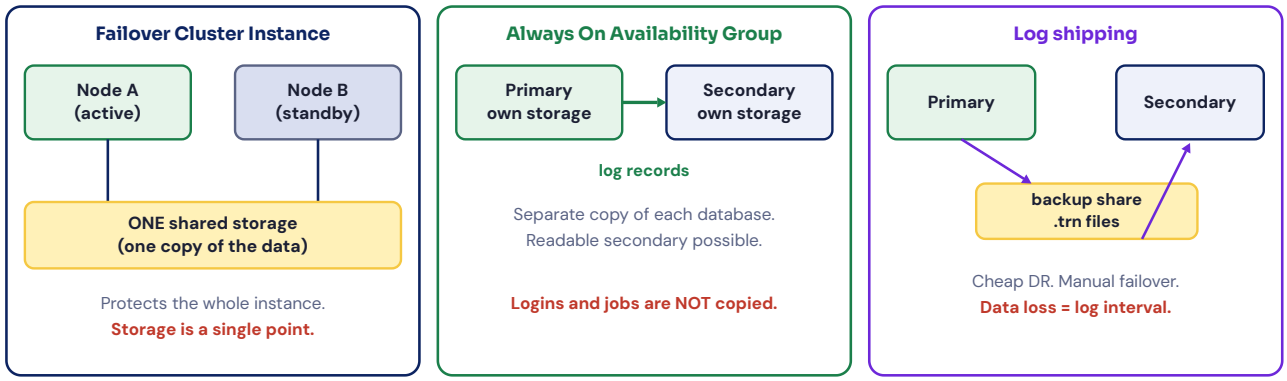
HOW THE INTERVIEWER ASKS IT

- *"What HA and DR options does SQL Server have?"*
- *"Difference between Always On Availability Groups and Failover Cluster Instances?"*
- *"What are RPO and RTO?"*

Say this first

"I first ask the business two numbers: RPO, how much data we can afford to lose, and RTO, how long we can be down. Then I pick the technology. Failover cluster instances protect against a server failure with shared storage. Availability groups protect databases with copies on other servers. Log shipping is the simple, cheap DR option."

Picture it



Three common designs. They solve different problems.

Option	Protects	Failover	Notes
Failover Cluster Instance (FCI)	The whole instance	Automatic, minutes	Shared storage, one copy of data
Always On Availability Group	Selected databases	Automatic with synchronous replica	Separate copies, readable secondaries
Log shipping	A database	Manual	Cheap and simple, data loss equals the log interval
Database mirroring	A database	Automatic	**Deprecated, replaced by availability groups**
Replication	Selected data	None	For distributing data, not for HA
Backup and restore	Everything	Manual	The final safety net

WAY 1 Always On Availability Group

WHY USE IT

Each replica has its own copy of the data, secondaries can serve read-only queries and backups, and failover can be automatic with no data loss on a synchronous replica.

WHEN IT FAILS

Does not copy logins, SQL Agent jobs and other server-level objects. You must keep them in sync. Needs a cluster and careful network design. Synchronous commit adds latency to every write.

WAY 2 Failover Cluster Instance

WHY USE IT

Protects the entire instance, including jobs and logins, and the application connects to one name.

WHEN IT FAILS

Shared storage is a single point of failure, there is only one copy of the data, and secondaries cannot be read.

WAY 3 Log shipping

WHY USE IT

Easy, cheap, works across editions, and a good DR copy in another region.

WHEN IT FAILS

Manual failover and potential data loss up to the log interval. The secondary is not available for reading while logs restore, unless in standby mode.

WHICH ONE SHOULD YOU PICK?

RPO of zero and automatic failover: AG with synchronous replica. Instance-level protection: FCI. Cheap DR with a few minutes of data loss acceptable: log shipping. Combine them, and never skip backups.

THE INTERVIEWER'S TRAP

HA is not backup. An availability group copies your accidental DELETE to every replica within seconds. Say that clearly, and mention that backups plus point-in-time restore (Q46) protect against human mistakes.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Synchronous vs asynchronous commit**
  Synchronous waits for the replica to harden the log, no data loss, higher latency. Asynchronous is faster, with possible data loss.
- **Basic availability groups**
  Standard edition, one database, limited features.
- **Readable secondary**
  Offload reports and backups, but beware of version store overhead.
- **Quorum and witness**
  Cluster voting that decides which side stays alive in a network split.

ONE LINE TO REMEMBER

Know RPO and RTO first. FCI protects the instance, AG protects databases with separate copies, log shipping is cheap DR. HA is not backup.

Q50 Query Store and plan regression

IMPORTANT

Chance of being asked 45%

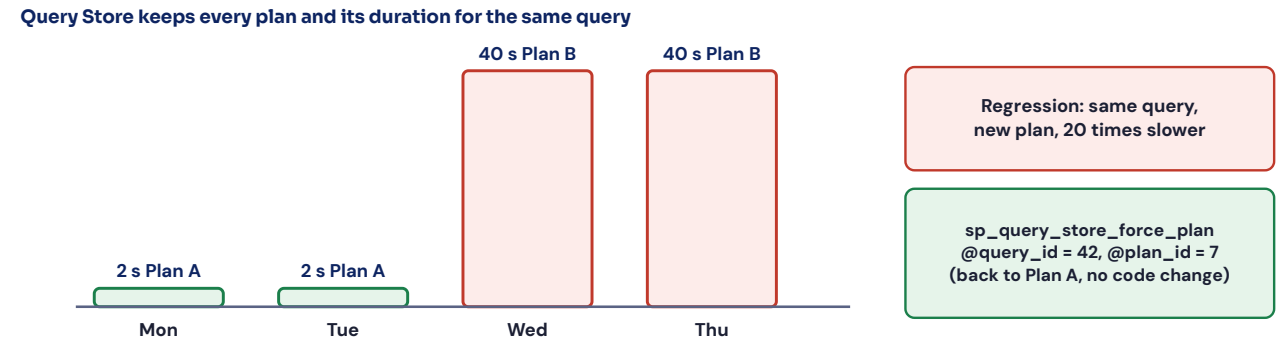
HOW THE INTERVIEWER ASKS IT

- *"What is Query Store? How does it help?"*
- *"How do you fix a query that suddenly got a bad plan?"*
- *"What is automatic plan correction?"*

Say this first

"Query Store is like a flight recorder for queries. It keeps the query text, every plan, and runtime statistics per time interval inside the database. When a query gets slower I can open the history, see that it switched from plan A to plan B, and force plan A. It survives restarts, which the plan cache does not."

Picture it



Same query_id, two plans, very different durations.

T-SQL

```
ALTER DATABASE InterviewPrep SET QUERY_STORE = ON
    (OPERATION_MODE = READ_WRITE, QUERY_CAPTURE_MODE = AUTO,
     MAX_STORAGE_SIZE_MB = 1024, INTERVAL_LENGTH_MINUTES = 30);

-- Force the good plan
EXEC sp_query_store_force_plan @query_id = 42, @plan_id = 7;
-- Undo it
EXEC sp_query_store_unforce_plan @query_id = 42, @plan_id = 7;
```

WAY 1 Manual plan forcing

WHY USE IT

Quick, safe and needs no code change. You choose the good plan using real duration data.

WHEN IT FAILS

If the schema or data changes, the forced plan can fail or become bad. Check `sys.query_store_plan` for forcing failures.

WAY 2 Automatic plan correction (SQL Server 2017+)

```T-SQL
ALTER DATABASE InterviewPrep SET AUTOMATIC_TUNING (FORCE_LAST_GOOD_PLAN = ON);
```

WHY USE IT

SQL Server detects a regression itself and reverts to the last good plan, then verifies that it helped.

WHEN IT FAILS

Needs Query Store on and reasonable history. It only handles plan regressions, not missing indexes or bad queries.

WAY 3 Fix the root cause (index, statistics, rewrite)

WHY USE IT

Forcing a plan hides the problem. A permanent fix removes it for everyone.

WHEN IT FAILS

Takes longer and may need a deployment. Often combine: force the plan now, fix properly later.

WHICH ONE SHOULD YOU PICK?

Turn Query Store on everywhere. During an incident force the good plan, then fix the root cause and unforce.

THE INTERVIEWER'S TRAP

Query Store is on by default for new databases in SQL Server 2022, but not for upgraded ones. Also, a full Query Store goes read-only, so size it and set cleanup.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Plan cache vs Query Store**
  The plan cache is in memory and is cleared on restart. Query Store is persistent.
- **Query Store hints**
  Add hints to a query without changing the code (2022).
- **Wait stats in Query Store**
  Available from 2017, shows what a query waits on.
- **Tracking regressions**
  The built-in "Regressed Queries" report in SSMS.

ONE LINE TO REMEMBER

Query Store records queries, plans and runtime stats. Force the last good plan during an incident, automatic tuning can do it for you, then fix the root cause.

Q51 tempdb contention and usage

IMPORTANT

Chance of being asked 40%

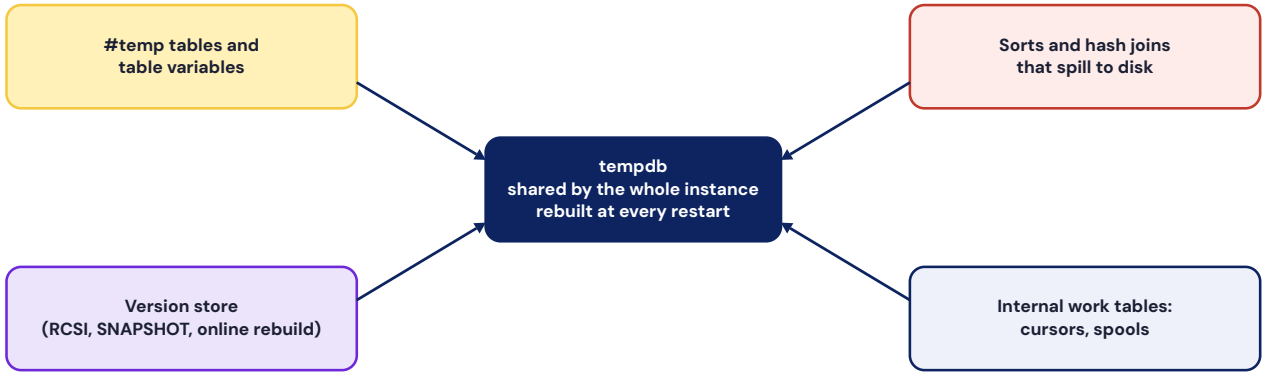
HOW THE INTERVIEWER ASKS IT

- *"What is tempdb used for? Why does it become a bottleneck?"*
- *"How many tempdb data files should you have?"*
- *"What fills up tempdb?"*

Say this first

"tempdb is a shared scratch space for the whole instance. It holds temp tables, table variables, sorts and hash spills, row versions for snapshot isolation, and internal work tables. Because everyone shares it, it can become a bottleneck. It is recreated at every restart, so nothing in it survives."

Picture it



Four kinds of work share one scratch database.

Use of tempdb	Example
User objects	`#temp` tables, table variables
Internal objects	Sorts, hash joins and spills, spools
Version store	RCSI and SNAPSHOT isolation (Q31), online index rebuilds
Other	Cursors, triggers, sort during index creation

T-SQL

```
-- What is using tempdb right now?
SELECT TOP (10) s.session_id,
       (u.user_objects_alloc_page_count - u.user_objects_dealloc_page_count) * 8 / 1024 AS UserMB,
       (u.internal_objects_alloc_page_count - u.internal_objects_dealloc_page_count) * 8 / 1024 AS InternalMB
FROM sys.dm_db_session_space_usage u
JOIN sys.dm_exec_sessions s ON s.session_id = u.session_id
ORDER BY UserMB + InternalMB DESC;
```

WAY 1 Add equally sized data files

WHY USE IT

Allocation pages (PFS, GAM, SGAM) become hot spots when many sessions create temp objects at once. More files spread that load. A common starting point is 1 file per core, up to 8, then add more only if contention continues.

WHEN IT FAILS

Files of different sizes get unequal use because of proportional fill. Too many files can hurt. Newer versions of SQL Server set this up better by default.

WAY 2 Reduce what goes into tempdb

WHY USE IT

Fix spills with better statistics and memory grants, avoid huge temp tables, and keep transactions short so the version store can clean up.

WHEN IT FAILS

Some workloads legitimately need a lot of tempdb, such as big sorts or index builds.

WAY 3 Size and place tempdb well

WHY USE IT

Pre-size the files so they do not auto-grow in the middle of the day, and put them on fast storage. Monitor free space.

WHEN IT FAILS

Pre-sizing only moves the problem if the workload really needs more space than you have.

WHICH ONE SHOULD YOU PICK?

Find who uses it with the DMVs, fix the heavy queries first, then size and add files if allocation contention is the proof. Do not blindly add files.

THE INTERVIEWER'S TRAP

Memory-optimized tempdb metadata (SQL Server 2019) removes metadata contention for heavy temp table use. Mention it if the interviewer asks about the newest options. Also note that table variables live in tempdb as well, not in memory only.

MORE QUESTIONS HIDING INSIDE THIS ONE

- **Why is tempdb full**
  A long open transaction with snapshot isolation, a runaway query, or a huge sort.
- **Temp table vs table variable**
  See Q21.
- **PAGELATCH waits on 2:1:1 or 2:1:3**
  Classic signs of allocation contention.
- **Can you back up tempdb**
  No. It is rebuilt at startup.

- **Accelerated database recovery**
  Uses a persistent version store in the user database.

ONE LINE TO REMEMBER

tempdb is shared scratch space. Watch the version store and spills. Equal-sized pre-grown files reduce allocation contention.

Last-night revision sheet

The 24 highest-probability questions, each with the one line you must remember. If you only have one evening, read this page, then open the full answer for any line that feels unfamiliar.

Q	Question	Chance	One line to remember	Page
Q1	Find the second highest salary	95%	Second highest means second highest DISTINCT value. For ties, DENSE_RANK is the safe window function. ROW_NUMBER and RANK both go wrong.	6
Q2	Types of JOINs in SQL Server	95%	INNER keeps matches, LEFT keeps all of the left table. A WHERE filter on the right table turns a LEFT JOIN into an INNER JOIN. Put it in ON.	12
Q4	DELETE vs TRUNCATE vs DROP	90%	DELETE = choose rows, logged, triggers fire. TRUNCATE = all rows, fast, resets identity, CAN be rolled back in SQL Server. DROP = the table is gone.	17
Q14	ROW_NUMBER vs RANK vs DENSE_RANK	90%	ROW_NUMBER = unique. RANK = ties share, then gaps. DENSE_RANK = ties share, no gaps. For 100, 100, 90: ROW_NUMBER 1,2,3. RANK 1,1,3. DENSE_RANK 1,1,2.	40
Q5	WHERE vs HAVING	85%	WHERE filters rows before grouping. HAVING filters groups after aggregation. Aggregates are not allowed in WHERE.	19
Q6	Find duplicate records	85%	GROUP BY + HAVING COUNT(*) > 1 finds the value. COUNT(*) OVER (PARTITION BY ...) finds the full rows. ROW_NUMBER finds the extras.	22
Q7	Clustered vs non-clustered index	85%	Clustered = the table, sorted by key, only one. Non-clustered = separate structure with a pointer, many. PK is a constraint, not the same as a clustered index.	25
Q16	Delete duplicates but keep one row	85%	Number the rows in each duplicate group with ROW_NUMBER, delete where rn > 1. Preview first, delete in a transaction, and decide which copy survives.	46
Q28	A query is slow. How do you troubleshoot it?	85%	Measure with STATISTICS IO, read the actual plan, check indexes, statistics, SARGability, sniffing, blocking. Change one thing, measure again.	77
Q8	Stored procedure vs function	80%	Function: returns a value, usable in SELECT, cannot change data. Procedure: does anything, called with EXEC, cannot sit inside a SELECT.	26
Q9	Primary key vs unique key	80%	One primary key per table, no NULL. Many unique keys, and SQL Server allows exactly ONE NULL in a unique column.	28
Q15	Top N per group (top 3 earners in each department)	80%	Top N per group = PARTITION BY group, ORDER BY the measure, filter the rank. DENSE_RANK keeps ties, ROW_NUMBER gives exactly N, CROSS APPLY TOP is the fast SQL Server way.	42
Q29	Index seek vs scan, key lookup and covering index	80%	Seek = jump to rows. Scan = read everything. Key lookup = extra trip to the table. Cover the query with INCLUDE columns, but only for hot queries.	80
Q17	Employees who earn more than their manager	75%	Self join: the same table twice with two aliases, matching ManagerID to EmpID. LEFT JOIN when the top person must still appear.	49
Q21	CTE vs temp table vs table variable	75%	CTE = named query for one statement, not stored. #Temp = real table in tempdb with statistics. @Table = lighter, not rolled back.	61

Q	Question	Chance	One line to remember	Page
Q31	**Isolation levels in SQL Server**	75%	Default is READ COMMITTED. NOLOCK = dirty reads. Use RCSI so readers do not block writers. SNAPSHOT gives a consistent view for the whole transaction.	**85**
Q32	**Deadlock vs blocking**	75%	Blocking waits and ends. A deadlock is a circle, SQL Server kills a victim with error 1205. Prevent with the same access order, short transactions, indexes, RCSI and retry.	**88**
Q3	**How many rows will each join return?**	70%	Count matches as left x right per value. NULL never matches NULL. Then add unmatched rows for the outer side.	**15**
Q10	**Logical order of query execution**	70%	FROM, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, TOP. Aliases are born in SELECT, so WHERE cannot see them.	**29**
Q11	**UNION vs UNION ALL (and INTERSECT, EXCEPT)**	70%	UNION removes duplicates (slower). UNION ALL keeps them (faster). Columns match by position, not name.	**32**
Q18	**Customers who never placed an order**	70%	Anti-join: NOT EXISTS is safest, LEFT JOIN IS NULL is readable. NOT IN returns nothing if the subquery contains a single NULL.	**52**
Q19	**Running total (cumulative sum)**	70%	SUM(x) OVER (ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW). Always write ROWS, add a tie-breaker.	**55**
Q30	**Why is my index not used? (SARGability)**	70%	Keep the indexed column bare on one side. No functions, no arithmetic, and match data types. Rewrite YEAR(col) = 2026 as a date range.	**82**
Q42	**It was fast yesterday and 40 seconds today. What changed?**	70%	Waits first, then plan comparison, then statistics and data. Capture evidence before fixing. Do not restart blindly.	**118**

MarutiNovaTech

Data and Technology Training Institute, Jaipur

Learn SQL Server, Power BI, Python, Microsoft Fabric and Azure Data Engineering with live batches taught by Deepankar Sir.

INSTAGRAM	**@datawithdeepankar**
YOUTUBE	**@deepankarpathak983**

Practise the queries, not just the answers.
All the best for your interview.

Probabilities are estimates based on a review of public interview guides and interview-prep content. They are meant for planning your revision, not as a guarantee.